

Módulo 6 - Seguridad con Spring Security y JWT

TEORÍA

Contents

Punto 6.1 - Introducción a Spring Security	2
Objetivos de aprendizaje	2
Bloque 1 - Qué es Spring Security	2
Bloque 2 - La cadena de filtros	4
Bloque 3 - Configuración moderna	5
Bloque 4 - Autenticación en detalle	7
Bloque 5 - Ecosistema de Spring Security	9
Resumen de la teoría	10
Punto 6.2 - Autenticación con usuarios en memoria	10
Objetivos de aprendizaje	10
Bloque 1 - El modelo de usuario en Spring Security	10
Bloque 2 - Usuarios en memoria	12
Bloque 3 - Cifrado de contraseñas	14
Bloque 4 - Integración con la cadena de filtros	16
Bloque 5 - Testing y buenas prácticas	17
Resumen de la teoría	19
Punto 6.3 - Usuarios en base de datos	19
Objetivos de aprendizaje	19
Bloque 1 - Por qué usuarios en base de datos	19
Bloque 2 - UserDetailsService personalizado	21
Bloque 3 - Integración con la configuración	24
Bloque 4 - Registro de usuarios	26
Bloque 5 - Buenas prácticas y tests	27
Resumen de la teoría	29
Punto 6.4 - Autorización por roles	29
Objetivos de aprendizaje	29
Bloque 1 - Autorización por URL vs por método	29
Bloque 2 - hasRole, hasAuthority, hasAnyRole	30
Bloque 3 - @PreAuthorize y @Secured	32
Bloque 4 - Acceder al usuario autenticado	33
Bloque 5 - Buenas prácticas y errores	35
Resumen de la teoría	37
Punto 6.5 - Introducción a JWT	37
Objetivos de aprendizaje	37
Bloque 1 - Qué es un JWT	37
Bloque 2 - Estructura de un JWT	39
Bloque 3 - La firma	40
Bloque 4 - Cuándo usar JWT	41
Bloque 5 - Errores y buenas prácticas	42
Resumen de la teoría	44
Punto 6.6 - Generación de tokens JWT	44
Objetivos de aprendizaje	44
Bloque 1 - La librería JJWT	44
Bloque 2 - La clave secreta	45

Bloque 3 - Generar un token	47
Bloque 4 - El endpoint de login	49
Bloque 5 - Refresh tokens y buenas prácticas	52
Resumen de la teoría	53
Punto 6.7 - Filtro JWT	53
Objetivos de aprendizaje	53
Bloque 1 - Por qué un filtro personalizado	54
Bloque 2 - Extraer el token	55
Bloque 3 - Construir el Authentication	56
Bloque 4 - Registrar el filtro	58
Bloque 5 - Diagnóstico y buenas prácticas	59
Resumen de la teoría	61
Punto 6.8 - Protección de endpoints con JWT	61
Objetivos de aprendizaje	61
Bloque 1 - La configuración completa	61
Bloque 2 - Endpoints públicos y privados	63
Bloque 3 - Autorización con JWT	64
Bloque 4 - El flujo completo	66
Bloque 5 - Diagnóstico y buenas prácticas	67
Resumen de la teoría	69
Punto 6.9 - Testing de seguridad	69
Objetivos de aprendizaje	69
Bloque 1 - Por qué testear la seguridad	69
Bloque 2 - @WithMockUser y @WithUserDetails	71
Bloque 3 - SecurityMockMvcRequestPostProcessors	72
Bloque 4 - Tests de autenticación y autorización	73
Bloque 5 - Buenas prácticas y errores	74
Resumen de la teoría	76

Baseline: Java 17, Spring Boot 3.5.16 y Maven Wrapper 3.9.16. En los ejemplos JWT se usa JWT 0.13.0. Este módulo parte del proyecto final de M5 y mantiene la estructura pedagógica del curso: cada punto desarrolla primero los conceptos y después los aplica en la guía práctica.

Punto 6.1 - Introducción a Spring Security

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es Spring Security y qué problema resuelve.
2. Describir qué ocurre al añadir spring-boot-starter-security al proyecto.
3. Diferenciar autenticación de autorización y relacionarlas con los códigos HTTP 401 y 403.
4. Explicar qué es la cadena de filtros de Spring Security y por qué el orden de los filtros importa.
5. Configurar SecurityFilterChain como bean, que es el enfoque utilizado en Spring Boot 3.x.
6. Identificar el papel de AuthenticationManager, UserDetailsService y PasswordEncoder en el proceso de autenticación.
7. Reconocer las principales alternativas del ecosistema de Spring Security y las herramientas de prueba que utilizaremos más adelante.
8. Diagnosticar los errores más habituales al comenzar a proteger una API con Spring Security.

Bloque 1 - Qué es Spring Security

T1.1 - Definición y propósito

Spring Security es el framework de seguridad del ecosistema Spring. Su función no consiste simplemente en mostrar un formulario de acceso: proporciona la infraestructura que permite decidir **quién está haciendo una petición** y **si esa identidad puede realizar la operación solicitada**. Estas dos preguntas corresponden a dos responsabilidades diferentes:

- **Autenticación:** comprobar la identidad del cliente. Por ejemplo, verificar un nombre de usuario y una contraseña o validar un token firmado.
- **Autorización:** decidir qué puede hacer una identidad que ya ha sido autenticada. Por ejemplo, permitir que un administrador gestione usuarios, pero impedir esa operación a un usuario normal.

Sin una capa de seguridad, cualquier cliente capaz de llegar a la API podría invocar los endpoints que estuvieran publicados. En un sistema administrativo esto sería inaceptable: un ciudadano no debe poder modificar expedientes ajenos y un usuario sin funciones de gestión no debe poder acceder a operaciones administrativas.

Spring Security se coloca delante de los controladores y aplica estas decisiones antes de que la petición llegue a la lógica de negocio. Para ello ofrece mecanismos de autenticación muy distintos —usuario y contraseña, JWT, OAuth2, certificados, LDAP, SAML— y varias formas de expresar reglas de autorización —roles, authorities, expresiones y seguridad a nivel de método—.

Una idea importante desde el principio es que Spring Security no obliga a utilizar un único mecanismo. El framework proporciona piezas combinables. En este módulo empezaremos con HTTP Basic y usuarios sencillos, después persistiremos las identidades y terminaremos utilizando JWT en una API stateless.

T1.2 - El “susto” inicial

La primera experiencia con Spring Security suele sorprender. Basta con añadir el starter de seguridad para que una aplicación que hasta ese momento era pública pase a estar protegida por defecto.

Si antes funcionaba:

```
curl -i http://localhost:8080/api/v1/alumnos
```

al incorporar `spring-boot-starter-security` la misma llamada dejará de obtener libremente los datos y recibirá un rechazo de autenticación. Spring Boot crea además, para ese estado inicial de desarrollo, un usuario llamado `user` y genera una contraseña aleatoria que aparece en el log de arranque, con un mensaje similar a:

```
Using generated security password: 3fa2b9e1-8c5d-4a7e-9f2b-1d4e5c6a7b8d
```

La contraseña es provisional y cambia cuando se reinicia la aplicación. No es una configuración que debamos conservar, sino una forma segura de evitar que la incorporación del starter deje accidentalmente toda la aplicación abierta.

Este comportamiento refleja una filosofía importante: Spring Security prefiere partir de **denegar el acceso** y permitir después de forma explícita aquello que deba ser público. En una aplicación real es más seguro descubrir que falta abrir un endpoint que descubrir que un endpoint sensible quedó accesible por defecto.

T1.3 - Autenticación vs autorización

Autenticación y autorización suelen confundirse porque ambas forman parte de la seguridad, pero responden a preguntas distintas.

La **autenticación** responde a «¿quién eres?». Un usuario presenta credenciales y el sistema intenta verificar que corresponden a una identidad válida. Si una ruta requiere autenticación y la petición no aporta credenciales válidas, la respuesta habitual es **401 Unauthorized**.

La **autorización** responde a «sabiendo quién eres, ¿tienes permiso para hacer esto?». El usuario puede estar correctamente autenticado y, aun así, carecer del rol o permiso exigido. En ese caso la respuesta adecuada es **403 Forbidden**.

Podemos resumirlo así:

Situación	Resultado típico
No existe una identidad autenticada válida para una ruta protegida	401 Unauthorized
Existe una identidad autenticada, pero no posee el permiso requerido	403 Forbidden
La identidad está autenticada y autorizada	La petición continúa hacia el controlador

Esta distinción será visible durante todo el módulo. Cuando lleguemos a JWT, el token permitirá reconstruir la identidad autenticada; las reglas de URL y método seguirán siendo las encargadas de decidir si esa identidad puede acceder al recurso.

Pregunta de reflexión: ¿Qué diferencia hay entre un 401 y un 403? ¿Cuándo devuelve el servidor cada uno?

Respuesta razonada: Un 401 indica que la petición no ha aportado una identidad válida para un recurso que exige autenticación: faltan credenciales o son inválidas. Un 403 aparece cuando la identidad ya está autenticada, pero no posee el rol o permiso necesario para realizar la operación.

Bloque 2 - La cadena de filtros

T2.1 - Qué es un filtro HTTP

En una aplicación Java web, un filtro es un componente que puede interceptar una petición antes de que llegue al servlet y una respuesta antes de que salga hacia el cliente. Los filtros se encadenan: cada uno inspecciona o transforma la petición y decide si continúa hacia el siguiente elemento de la cadena.

Cuando se han ejecutado los filtros correspondientes, la petición puede llegar al `DispatcherServlet` de Spring MVC, que localiza el controlador adecuado.

Los filtros no existen únicamente para seguridad. Se utilizan habitualmente para tareas transversales como:

- registrar peticiones;
- añadir información de auditoría;
- gestionar CORS;
- comprimir respuestas;
- incorporar o verificar información de seguridad.

Spring Security aprovecha precisamente este mecanismo. En vez de introducir todas las decisiones de seguridad dentro de los controladores, coloca una infraestructura delante de ellos. Así, la misma política puede proteger muchos endpoints sin repetir código en cada método.

Además de seguridad, el mecanismo de filtros se utiliza para responsabilidades transversales que deben ejecutarse antes o después del controlador: logging de peticiones, auditoría de quién accede a qué, tratamiento de CORS y, en algunos servidores, compresión de respuestas. La idea importante es que un filtro no sustituye al `DispatcherServlet`: actúa antes de él y decide si la petición continúa por la cadena.

En una cadena cada filtro tiene una responsabilidad limitada. Si deja continuar la petición, invoca al siguiente elemento; sólo al final se alcanza Spring MVC y el controlador correspondiente. Spring Security aprovecha precisamente este mecanismo estándar del mundo Servlet/Jakarta para colocar varias piezas especializadas en lugar de concentrar toda la seguridad en una única clase monolítica.

T2.2 - La cadena de filtros de Spring Security

Spring Security añade su propia cadena especializada. Cada filtro realiza una responsabilidad concreta y el conjunto forma el proceso de seguridad de la petición.

Entre los componentes que conviene reconocer están:

- `SecurityContextHolderFilter`, encargado de preparar el contexto de seguridad utilizado durante la petición;
- filtros de autenticación, como los asociados a HTTP Basic o usuario/contraseña;
- `ExceptionTranslationFilter`, que participa en la traducción de determinados fallos de seguridad a respuestas HTTP;
- `AuthorizationFilter`, que aplica las reglas de autorización antes de permitir que la petición alcance el recurso protegido;
- otros filtros asociados a CORS, CSRF, sesión o logout, que se activan o configuran según las necesidades de la aplicación.

No necesitamos memorizar toda la lista interna. Lo importante es entender que Spring Security no funciona como una llamada aislada al principio del controlador. La petición atraviesa una serie de etapas, cada una con una responsabilidad clara.

Esta arquitectura explica también por qué más adelante podremos insertar nuestro propio `JwtAuthenticationFilter`: no reemplazaremos Spring Security, sino que añadiremos una etapa capaz

de interpretar un token Bearer y construir la identidad que utilizarán después las reglas de autorización.

T2.3 - Orden de los filtros principales

El orden es esencial. Una simplificación útil del recorrido es:

1. **Preparación del contexto.** Se inicializan las estructuras necesarias para tratar la seguridad de la petición.
2. **Autenticación.** Los filtros que reconocen credenciales intentan construir una identidad autenticada.
3. **Autorización.** Con la identidad disponible —o con una petición anónima— se comprueba si la ruta admite el acceso.
4. **Tratamiento de fallos y continuación.** Los componentes correspondientes convierten los fallos en respuestas adecuadas o permiten que la petición alcance Spring MVC.

Si un filtro de autenticación no consigue establecer una identidad y la ruta exige autenticación, la autorización terminará rechazando la petición. Si la identidad existe pero carece del rol requerido, el rechazo se produce por falta de permisos.

Por eso el diagnóstico debe hacerse pensando en capas. Ante una petición rechazada hay que preguntarse primero si se obtuvo una identidad y después si esa identidad cumplía la regla de autorización.

Los logs de Spring Security resultan especialmente útiles para seguir este recorrido. En la parte práctica activaremos temporalmente el nivel DEBUG para observar qué filtros intervienen y qué decisión toma la cadena.

Pregunta de reflexión: ¿Por qué la seguridad se implementa con una cadena de filtros y no con un único filtro monolítico?

Respuesta razonada: Porque cada filtro puede asumir una responsabilidad concreta —cargar el contexto, autenticar, traducir excepciones o autorizar— y ejecutarse en un orden definido. Esta separación permite componer, sustituir y diagnosticar mecanismos sin concentrar toda la seguridad en una única pieza difícil de mantener.

Bloque 3 - Configuración moderna

T3.1 - WebSecurityConfigurerAdapter (deprecado)

Durante años fue habitual configurar Spring Security extendiendo `WebSecurityConfigurerAdapter` y sobrescribiendo métodos como `configure(HttpSecurity)`. Muchos tutoriales antiguos siguen mostrando ese enfoque:

```
// Enfoque histórico: no usar en Spring Boot 3.x
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
            .anyRequest().authenticated()
            .and()
            .httpBasic();
    }
}
```

Es importante conocerlo porque aparecerá en documentación y ejemplos antiguos, pero **no es código válido para nuestra versión del curso**. `WebSecurityConfigurerAdapter` fue deprecado antes de Spring Boot 3 y ya no forma parte del enfoque actual.

El cambio no es simplemente sintáctico. El modelo moderno favorece la composición mediante beans en lugar de obligarnos a extender una clase base. Esto permite declarar de forma explícita las piezas de la configuración y facilita combinarlas y probarlas.

Por tanto, cuando encontremos un tutorial que extienda `WebSecurityConfigurerAdapter`, no copiaremos ese código. Conservaremos el concepto que pretende enseñar y lo expresaremos con el modelo basado en `SecurityFilterChain`.

T3.2 - SecurityFilterChain como bean

En Spring Boot 3.x configuramos la seguridad web declarando un bean de tipo SecurityFilterChain:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Aquí aparecen varias piezas importantes:

- @Configuration indica que la clase declara configuración de Spring.
- @EnableWebSecurity hace explícito que la clase participa en la seguridad web. En Spring Boot, la presencia del bean SecurityFilterChain ya permite la configuración automática necesaria, pero mantener la anotación hace visible la intención de la clase.
- @Bean registra el SecurityFilterChain en el contexto.
- HttpSecurity es el objeto sobre el que expresamos las reglas.
- authorizeHttpRequests abre la DSL moderna de autorización.
- httpBasic(Customizer.withDefaults()) activa HTTP Basic.
- http.build() construye la cadena configurada.

La ventaja respecto al antiguo modelo por herencia es que cada pieza se declara explícitamente como configuración componible. Más adelante añadiremos reglas de rutas, CORS, política de sesión, manejadores de errores y nuestro filtro JWT sobre esta misma base.

La configuración moderna es composicional. @Configuration declara la clase como fuente de beans; @Bean registra el SecurityFilterChain; HttpSecurity es el objeto de configuración sobre el que se encadenan reglas; y http.build() materializa finalmente la cadena que Spring Security aplicará a las peticiones.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

@EnableWebSecurity hace explícita la intención de configurar seguridad web. Spring Boot puede detectar el bean SecurityFilterChain sin esa anotación, pero el enfoque basado en beans es el reemplazo conceptual del antiguo WebSecurityConfigurerAdapter: composición en lugar de herencia, con una configuración más fácil de combinar y probar.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

T3.3 - Métodos principales de la configuración

Dentro del SecurityFilterChain utilizaremos varias operaciones de HttpSecurity. Un ejemplo de reglas por URL es:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
)
```

Las llamadas más relevantes del módulo son:

- `requestMatchers(...)`: selecciona rutas o grupos de rutas;
- `permitAll()`: permite acceso sin autenticación;
- `authenticated()`: exige una identidad autenticada;
- `hasRole("ADMIN")`: exige el rol indicado;
- `hasAuthority(...)`: exige una authority concreta;
- `anyRequest()`: aplica la regla a cualquier petición no capturada antes.

El orden de las reglas importa porque las coincidencias específicas deben quedar expresadas antes de la regla general.

También aparecerán:

- `httpBasic(...)`, para el mecanismo temporal de HTTP Basic;
- `csrf(...)`, para configurar CSRF;
- `cors(...)`, para integrar la política CORS con la seguridad;
- `sessionManagement(...)`, que más adelante configuraremos como STATELESS;
- `addFilterBefore(...)`, que utilizaremos para colocar el filtro JWT en la cadena.

La configuración del punto 6.1 será todavía sencilla. El objetivo no es adelantar JWT, sino aprender cómo se expresa una política web moderna para poder evolucionarla de forma controlada durante los siguientes puntos.

Pregunta de reflexión: ¿Por qué Spring Security abandonó `WebSecurityConfigurerAdapter` en favor de una configuración basada en beans?

Respuesta razonada: Porque el enfoque basado en beans favorece la composición frente a la herencia: cada pieza de seguridad se declara explícitamente y puede combinarse con otras configuraciones sin depender de una clase base. Además, hace más visible qué componentes utiliza realmente la aplicación.

Bloque 4 - Autenticación en detalle

T4.1 - AuthenticationManager

`AuthenticationManager` es la interfaz central que recibe un intento de autenticación y devuelve, si las credenciales son válidas, un objeto `Authentication` autenticado.

Podemos pensar el flujo de usuario y contraseña de esta manera:

1. Un filtro obtiene las credenciales de la petición.
2. Construye un objeto `Authentication` todavía no autenticado.
3. Lo entrega al `AuthenticationManager`.
4. El gestor delega la verificación en uno o varios `AuthenticationProvider`.
5. Si las credenciales son válidas, se obtiene un `Authentication` autenticado.
6. Esa identidad se coloca en el contexto de seguridad para el resto de la petición.

La implementación habitual es `ProviderManager`, que coordina distintos providers. Para usuario y contraseña, uno de los providers habituales es `DaoAuthenticationProvider`, que utiliza un `UserDetailsService` para localizar el usuario y un `PasswordEncoder` para comprobar la contraseña.

En nuestro proyecto no construiremos manualmente toda esta infraestructura. Spring Boot puede ensamblarla a partir de los beans que declaremos. En el punto 6.6 utilizaremos el `AuthenticationManager` durante el login para validar las credenciales iniciales antes de emitir los tokens JWT.

Conviene separar desde ahora los dos conceptos: el `AuthenticationManager` valida las credenciales de login; cuando trabajemos con un Bearer token ya emitido, nuestro filtro JWT verificará el token y reconstruirá la identidad para esa petición.

Una consecuencia práctica es que la aplicación no debería mezclar responsabilidades: el `AuthenticationManager` coordina la autenticación, pero no carga directamente usuarios ni cifra contraseñas.

Esa separación permite cambiar la fuente de identidades —memoria en 6.2, base de datos en 6.3— sin reescribir el componente que orquesta la autenticación.

Cuando necesitemos usarlo desde código propio, por ejemplo en `AuthService`, Spring puede exponer el `AuthenticationManager` construido a partir de su configuración. El servicio le entrega un `UsernamePasswordAuthenticationToken` con `username` y `contraseña`. Si todo es correcto, el resultado contiene el principal y sus `authorities`; si no, se produce una excepción de autenticación que debe traducirse a un 401 estable. En ningún caso debemos comparar manualmente la contraseña en `AuthService`: reutilizamos el mismo mecanismo de Spring Security para que la política sea única.

T4.2 - UserDetailsService

`UserDetailsService` representa la operación de cargar un usuario por su nombre:

```
public interface UserDetailsService {
    UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException;
}
```

Su responsabilidad es **cargar** la información de la identidad, no verificar por sí mismo la contraseña. El `UserDetails` resultante contiene el nombre de usuario, el hash de contraseña, roles o `authorities` y los flags de estado de la cuenta.

Spring Security incluye implementaciones como `InMemoryUserDetailsManager` y `JdbcUserDetailsManager`, y también permite implementar un servicio propio. En el punto 6.2 utilizaremos usuarios en memoria para entender el mecanismo sin introducir todavía persistencia. En 6.3 sustituiremos esa solución por un `UserDetailsService` que cargará nuestras entidades `Usuario` y `Rol` desde la base de datos.

Esta interfaz es uno de los puntos de desacoplamiento más importantes de Spring Security: la cadena de autenticación no necesita saber si el usuario procede de una lista en memoria, una tabla SQL o cualquier otra fuente. Sólo necesita un contrato capaz de cargar `UserDetails`.

T4.3 - PasswordEncoder

Las contraseñas no deben almacenarse en texto plano. `PasswordEncoder` define las operaciones utilizadas para transformar y verificar credenciales:

```
public interface PasswordEncoder {
    String encode(CharSequence rawPassword);
    boolean matches(CharSequence rawPassword, String encodedPassword);
}
```

`encode` se utiliza cuando hay que almacenar una contraseña nueva. `matches` se utiliza cuando el usuario presenta una contraseña y queremos comprobar si corresponde al hash guardado.

No se descifra la contraseña almacenada. Los algoritmos de hashing de contraseñas están diseñados precisamente para evitar esa operación. La comprobación consiste en ejecutar el algoritmo apropiado sobre la contraseña presentada y comparar de forma segura el resultado.

Durante el módulo veremos `BCrypt` y también utilizaremos el `DelegatingPasswordEncoder` que ofrece Spring Security para permitir que el hash indique qué algoritmo lo produjo. Otras opciones del ecosistema incluyen `PBKDF2` y `Argon2`. `NoOpPasswordEncoder`, que no protege la contraseña, no debe utilizarse como solución real.

El contrato de `PasswordEncoder` separa dos operaciones que no deben confundirse:

```
String encode(CharSequence rawPassword);
boolean matches(CharSequence rawPassword, String encodedPassword);
```

`encode` se usa al registrar o cambiar una contraseña; `matches` durante la autenticación. La aplicación no descifra el hash almacenado: comprueba si la contraseña presentada produce una coincidencia válida según el algoritmo.

`BCrypt` es adaptativo: su coste puede aumentarse para hacer más cara la fuerza bruta. Spring también ofrece `Argon2PasswordEncoder`, `Pbkdf2PasswordEncoder` y `DelegatingPasswordEncoder`, que permite convivir con varios esquemas identificados mediante un prefijo. `NoOpPasswordEncoder` no protege la contraseña y no debe usarse como solución de producción. La regla del módulo es constante: nunca guardar la contraseña en texto plano ni comparar hashes manualmente.

El contrato de `PasswordEncoder` separa dos operaciones que no deben confundirse:

```
String encode(CharSequence rawPassword);  
boolean matches(CharSequence rawPassword, String encodedPassword);
```

BCrypt es adaptativo: su coste puede aumentarse para hacer más cara la fuerza bruta. Spring también ofrece Argon2PasswordEncoder, Pbkdf2PasswordEncoder y DelegatingPasswordEncoder, que permite convivir con varios esquemas identificados mediante un prefijo. NoOpPasswordEncoder no protege la contraseña y no debe usarse como solución de producción. La regla del módulo es constante: nunca guardar la contraseña en texto plano ni comparar hashes manualmente. > **Pregunta de reflexión:** ¿Por qué no se guarda la contraseña en texto plano? ¿Qué diferencia hay entre encode y matches?

Respuesta razonada: Guardar contraseñas en texto plano convertiría una fuga de la base de datos en una exposición inmediata de credenciales. encode transforma una contraseña en una representación protegida para almacenarla; matches comprueba si una contraseña presentada corresponde con el hash guardado sin necesitar recuperar la contraseña original.

Bloque 5 - Ecosistema de Spring Security

T5.1 - Autenticación en memoria vs base de datos

Para aprender el proceso podemos almacenar usuarios en memoria. InMemoryUserDetailsManager permite declarar un conjunto pequeño de usuarios directamente en configuración y es muy útil en ejercicios, prototipos o ciertos tests. Sin embargo, esos usuarios no constituyen una solución de gestión real: no representan un sistema de registro persistente y su definición depende de la configuración de la aplicación.

En una aplicación de producción lo habitual es persistir las identidades. Podemos utilizar esquemas predefinidos o, como haremos en este curso, nuestras propias entidades y un UserDetailsService personalizado. Eso permite registrar usuarios, modificar sus datos, gestionar roles y conservar el estado entre reinicios.

El recorrido del curso utiliza ambas estrategias deliberadamente. En 6.2 tendremos usuarios en memoria porque es la forma más clara de aislar los conceptos de UserDetails, roles y contraseñas. En 6.3 cambiaremos esa fuente por base de datos sin reemplazar el resto de la arquitectura de seguridad.

T5.2 - OAuth2 y JWT

Spring Security soporta mecanismos mucho más amplios que el login local con usuario y contraseña.

OAuth2 es un protocolo de autorización delegada. Permite que una aplicación obtenga acceso limitado a recursos gestionados por otro sistema sin compartir directamente la contraseña del usuario con la aplicación cliente. Spring Security ofrece soporte tanto para clientes OAuth2 como para resource servers.

JWT no es un protocolo de login, sino un formato de token firmado capaz de transportar claims. En una API REST puede utilizarse para representar una identidad y sus permisos sin mantener una sesión HTTP tradicional en el servidor. A partir de 6.5 estudiaremos su estructura y después implementaremos la emisión y validación de tokens.

El ecosistema incluye asimismo SAML, muy utilizado en federación empresarial, y autenticación mediante certificados X.509, habitual en escenarios que requieren certificados de cliente.

En este curso nos concentraremos en el recorrido usuario/contraseña → JWT porque permite estudiar de forma visible cómo se conectan autenticación, autorización y una API stateless.

T5.3 - Testing de seguridad

La seguridad necesita tests específicos. No basta con comprobar que el controlador funciona cuando todo está permitido; también debemos probar que los accesos que **deben fallar** efectivamente son rechazados.

Spring Security proporciona herramientas como:

- @WithMockUser, para ejecutar un test con una identidad simulada y unos roles determinados;
- @WithUserDetails, para utilizar un usuario cargado desde un UserDetailsService;
- SecurityMockMvcRequestPostProcessors, que permiten aplicar información de seguridad a peticiones MockMvc.

Las estudiaremos con detalle en 6.9. Antes de llegar allí también realizaremos comprobaciones manuales con `curl`, porque son una forma directa de observar la diferencia entre una ruta pública, una petición sin credenciales, una identidad válida y una identidad sin permisos.

Pregunta de reflexión: ¿Qué diferencia hay entre OAuth2 y JWT? ¿Cuándo usarías cada uno?

Respuesta razonada: OAuth2 es un protocolo de autorización delegada que define actores y flujos para conceder acceso; JWT es un formato de token firmado que puede utilizarse dentro o fuera de OAuth2. Usaría OAuth2 cuando interviene autorización delegada o un proveedor de identidad, y JWT cuando necesito transportar claims firmados entre cliente y API en un diseño adecuado.

Resumen de la teoría

- Spring Security proporciona autenticación y autorización para las peticiones de la aplicación.
- Al añadir `spring-boot-starter-security`, Spring Boot protege las rutas por defecto y crea una credencial provisional de desarrollo.
- Autenticación y autorización son procesos diferentes: la ausencia de identidad válida suele producir 401; la falta de permisos de una identidad autenticada produce 403.
- Spring Security trabaja mediante una cadena ordenada de filtros.
- `WebSecurityConfigurerAdapter` pertenece al enfoque histórico y ya no debe utilizarse en Spring Boot 3.x.
- La configuración moderna declara un bean `SecurityFilterChain` sobre `HttpSecurity`.
- `AuthenticationManager` coordina el intento de autenticación; `UserDetailsService` carga la identidad; `PasswordEncoder` protege y comprueba la contraseña.
- Los usuarios pueden proceder de memoria o de una base de datos, sin cambiar el contrato que consume Spring Security.
- OAuth2, JWT, SAML y X.509 forman parte del ecosistema más amplio.
- Spring Security aporta utilidades específicas de testing que utilizaremos en el punto 6.9.

Punto 6.2 - Autenticación con usuarios en memoria

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es un `UserDetails` y qué información contiene.
2. Configurar usuarios en memoria con `InMemoryUserDetailsManager`.
3. Cifrar contraseñas con `BCryptPasswordEncoder` y entender por qué es necesario.
4. Definir usuarios con roles y authorities.
5. Autenticarse con HTTP Basic y verificar el acceso a distintos endpoints.
6. Diagnosticar y resolver los errores más comunes al configurar usuarios.

Bloque 1 - El modelo de usuario en Spring Security

T1.1 - Qué es un UserDetails

`UserDetails` es la representación de seguridad de una identidad. Contiene nombre de usuario, contraseña codificada, authorities y varios indicadores de estado de la cuenta. Spring Security trabaja con este contrato en lugar de depender directamente de nuestra entidad JPA.

Separar la entidad de dominio de la representación de seguridad evita acoplar toda la aplicación a una interfaz del framework. En M6 terminaremos utilizando `UsuarioPrincipal` como adaptador entre `Usuario` y `UserDetails`.

`UserDetails` es la representación que Spring Security consume, no necesariamente la entidad JPA del dominio. Contiene nombre de usuario, contraseña codificada, authorities y banderas de estado. Mantener un adaptador evita que la infraestructura de seguridad obligue a deformar la entidad `Usuario`.

En una aplicación real el principal puede necesitar además el identificador interno del usuario. Por eso más adelante `UsuarioPrincipal` encapsula los datos de seguridad que necesitamos sin exponer la contraseña ni acoplar el resto del código al modelo de persistencia.

El contrato incluye no sólo usuario, contraseña y authorities, sino también el estado de la cuenta. Sus métodos permiten expresar si la cuenta está expirada, bloqueada, si las credenciales han

caducado o si el usuario está habilitado:

```
String getUsername();
String getPassword();
Collection<? extends GrantedAuthority> getAuthorities();
boolean isAccountNonExpired();
boolean isAccountNonLocked();
boolean isCredentialsNonExpired();
boolean isEnabled();
```

Spring aporta además una implementación llamada `org.springframework.security.core.userdetails.User`.

Es útil para los usuarios didácticos en memoria, pero no debe confundirse con nuestra futura entidad JPA Usuario. Esa distinción explica por qué el módulo puede cambiar la forma de almacenar identidades sin cambiar el contrato que consume Spring Security.

```
String getUsername();
String getPassword();
Collection<? extends GrantedAuthority> getAuthorities();
boolean isAccountNonExpired();
boolean isAccountNonLocked();
boolean isCredentialsNonExpired();
boolean isEnabled();
```

T1.2 - Qué es un UserDetailsService

UserDetailsService carga un UserDetails por nombre. Su método principal, `loadUserByUsername`, debe devolver la identidad o lanzar `UsernameNotFoundException`. La interfaz no sabe si los datos vienen de memoria, JPA, LDAP u otro sistema.

Esa neutralidad permite sustituir la implementación sin cambiar el login. Durante 6.2 la fuente de datos será una colección en memoria; en 6.3 la misma operación consultará la base de datos.

UserDetailsService tiene un único método central, `loadUserByUsername`. Ese contrato parece pequeño, pero establece una frontera clara entre autenticación y almacenamiento. La implementación puede buscar en una colección, JPA, LDAP u otro sistema sin cambiar al consumidor.

Una implementación correcta distingue “usuario no encontrado” mediante `UsernameNotFoundException` y devuelve authorities consistentes con las reglas. No debe autenticar manualmente ni devolver DTOs de API. Su salida está pensada para Spring Security.

El método esencial es:

```
UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException;
```

Si el usuario existe, devuelve un UserDetails; si no existe, comunica el caso mediante `UsernameNotFoundException`. La interfaz no valida contraseñas. Su responsabilidad es localizar y describir la identidad; la comparación de credenciales la realiza el proveedor de autenticación junto con el `PasswordEncoder`.

Esta separación permite que en 6.2 la implementación sea `InMemoryUserDetailsManager` y que en 6.3 podamos reemplazarla por una implementación que consulte `UsuarioRepository`. El resto del flujo de autenticación no necesita saber si los datos proceden de memoria, JDBC, JPA, LDAP u otro origen.

El método esencial es:

```
UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException;
```

T1.3 - Roles y authorities

Spring Security toma decisiones sobre `GrantedAuthority`. Un rol es una convención construida sobre authorities cuyo nombre suele llevar el prefijo `ROLE_`. Por eso `hasRole("ADMIN")` busca internamente `ROLE_ADMIN`, mientras `hasAuthority("ADMIN")` exige exactamente esa cadena.

Conviene fijar una única convención. En el proyecto almacenaremos roles con nombres como `ROLE_USER`, `ROLE_GESTOR` y `ROLE_ADMIN`, y los transformaremos directamente en authorities.

En Spring Security, un rol es una convención sobre una authority. `hasRole("ADMIN")` busca normalmente la authority `ROLE_ADMIN`; `hasAuthority("ADMIN")` busca literalmente `ADMIN`. Confundir ambas formas produce 403 aparentemente inexplicables.

Elegiremos una convención y la mantendremos desde la base de datos hasta los tests. La entidad

rol puede almacenar ADMIN, mientras el adaptador transforma a ROLE_ADMIN; lo importante es que la frontera sea explícita y consistente.

Ejemplo.

```
new SimpleGrantedAuthority("ROLE_ADMIN")
```

GrantedAuthority es la unidad que Spring Security utiliza realmente al decidir permisos. Los roles son una convención construida encima de esas authorities. Por eso estas dos formas no son equivalentes:

```
.roles("ADMIN")
.authorities("ADMIN")
```

La primera genera ROLE_ADMIN; la segunda crea literalmente ADMIN. De la misma forma, hasRole("ADMIN") busca ROLE_ADMIN, mientras hasAuthority("ADMIN") busca exactamente ADMIN.

Los roles resultan cómodos para categorías amplias como USER, GESTOR o ADMIN. Las authorities pueden representar permisos más finos, por ejemplo READ_ALUMNOS. El punto importante no es elegir siempre una sola API, sino usar una convención coherente desde la creación del usuario hasta las reglas y los tests.

```
.roles("ADMIN")
.authorities("ADMIN")
```

Los roles resultan cómodos para categorías amplias como USER, GESTOR o ADMIN. Las authorities pueden representar permisos más finos, por ejemplo READ_ALUMNOS. El punto importante no es elegir siempre una sola API, sino usar una convención coherente desde la creación del usuario hasta las reglas y los tests. > **Pregunta de reflexión:** ¿Qué diferencia hay entre un rol y un permiso? ¿Cuándo usarías cada uno?

Respuesta razonada: Un rol agrupa una función de negocio amplia, como ADMIN o GESTOR; un permiso representa una capacidad más concreta, como LEER_EXPEDIENTES. Los roles simplifican políticas generales y los permisos permiten granularidad cuando varias funciones comparten o difieren en operaciones concretas.

Bloque 2 - Usuarios en memoria

T2.1 - InMemoryUserDetailsManager

InMemoryUserDetailsManager almacena objetos UserDetails dentro del proceso. Es rápido de configurar y permite experimentar con varios usuarios y roles sin crear todavía tablas ni repositorios. Su limitación es precisamente esa: el estado vive en memoria. Al reiniciar, sólo existen los usuarios declarados en configuración. Por eso lo usaremos como etapa didáctica y lo eliminaremos antes del estado final.

InMemoryUserDetailsManager implementa UserDetailsService y permite declarar identidades de forma local. Es adecuado para el experimento porque Spring recibe un componente real, no un mock, y podemos observar la misma maquinaria de autenticación que después utilizará la base de datos.

Los usuarios deben construirse con contraseñas codificadas y roles deliberados. El objetivo no es crear “cuentas de producción”, sino disponer de casos contrastables —por ejemplo USER y ADMIN— para demostrar autorización.

La implementación puede declararse como bean y recibir varios UserDetails ya contruidos:

```
@Bean
UserDetailsService users(PasswordEncoder encoder) {
    UserDetails ana = User.builder()
        .username("ana")
        .password(encoder.encode("ana123"))
        .roles("USER")
        .build();
    UserDetails admin = User.builder()
        .username("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN", "USER")
        .build();
    return new InMemoryUserDetailsManager(ana, admin);
}
```

Spring recibe aquí un UserDetailsService real. La diferencia con una base de datos está en

dónde viven los usuarios, no en el contrato de autenticación. Al reiniciar, el estado vuelve a ser el declarado en configuración; no existe alta dinámica ni persistencia. Esa limitación es deliberada en 6.2 y desaparece en 6.3.

La implementación puede declararse como bean y recibir varios `UserDetails` ya contruidos:

```
@Bean
UserService users(PasswordEncoder encoder) {
    UserDetails ana = User.builder()
        .username("ana")
        .password(encoder.encode("ana123"))
        .roles("USER")
        .build();
    UserDetails admin = User.builder()
        .username("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN", "USER")
        .build();
    return new InMemoryUserDetailsManager(ana, admin);
}
```

T2.2 - Por qué usar usuarios en memoria

Una etapa intermedia simple reduce variables. Si queremos entender por qué un ADMIN entra y un USER recibe 403, es más fácil comenzar con usuarios estáticos que introducir a la vez JPA, relaciones muchos-a-muchos y carga de authorities.

El valor de la técnica es aislar el mecanismo. Una vez comprobado, trasladaremos las identidades a base de datos sin cambiar la semántica de roles ni la forma de autorizar.

Reducir dependencias durante un experimento ayuda a aislar causas. Si al mismo tiempo introdujéramos JPA, relaciones de roles, inicialización y reglas de acceso, un 401 podría tener demasiadas explicaciones. Los usuarios en memoria permiten aprender primero el contrato de Spring Security.

El experimento tiene una fecha de caducidad. Una vez introducida la persistencia, conservar simultáneamente la fuente in-memory crearía dos orígenes de verdad y podría autenticar usuarios que ya no existen en la base de datos.

Usar memoria permite estudiar autenticación y autorización sin introducir todavía tablas, repositorios, transacciones y relaciones JPA. Si un usuario USER recibe 403 en un endpoint ADMIN, sabemos que el experimento se concentra en las reglas de seguridad y no en una consulta defectuosa a la base de datos.

Este mecanismo es apropiado para desarrollo, demostraciones y determinados tests. No es una solución para producción: los cambios requieren modificar configuración/código, los usuarios no se administran de forma dinámica y el estado no representa un censo persistente. Por eso el módulo lo usa como una etapa controlada: primero entendemos `UserDetailsService` y las authorities; después conservamos esos conceptos y cambiamos únicamente el origen de datos.

T2.3 - Definir roles vs authorities

Al crear un usuario podemos utilizar `.roles("ADMIN")` o `.authorities("ROLE_ADMIN")`. La primera forma añade el prefijo `ROLE_`; la segunda utiliza literalmente los valores suministrados.

Mezclar ambas sin conocer la diferencia genera fallos difíciles de ver: el usuario parece tener un permiso, pero la expresión busca otro nombre. En el módulo verificaremos las authorities reales que llegan al `SecurityContext`.

Los roles expresan categorías amplias; las authorities pueden modelar permisos más finos. `roles("ADMIN")` añade automáticamente el prefijo `ROLE_`, mientras `authorities(...)` conserva exactamente el texto suministrado. Esta diferencia debe entenderse antes de mezclar APIs.

Un diseño sencillo del curso usa roles porque encajan con USER/GESTOR/ADMIN. Si en el futuro aparecieran permisos como `EXPEDIENTE_LEER` y `EXPEDIENTE_APROBAR`, sería natural representarlos como authorities independientes y componerlos en los roles.

Ejemplo.

```
User.withUsername("ana")
    .password(encoder.encode("clave-de-ejemplo"))
    .roles("USER")
    .build();
```

Spring ofrece ambas APIs porque roles y permisos pueden modelar necesidades distintas. Por ejemplo:

```
User.builder()
    .username("admin")
    .password(encoded)
    .roles("ADMIN", "USER")
    .build();
```

produce ROLE_ADMIN y ROLE_USER. Si en cambio se llama a `.authorities("READ_ALUMNOS")`, no se añade ningún prefijo.

El error típico consiste en guardar ADMIN, construir luego una authority ADMIN y proteger la ruta con `hasRole("ADMIN")`: la regla buscará ROLE_ADMIN y devolverá 403. En este curso usamos roles para USER/GESTOR/ADMIN y mantenemos visible el momento en el que se añade el prefijo. Si el dominio necesitara capacidades granulares, esas capacidades podrían expresarse como authorities independientes.

Spring ofrece ambas APIs porque roles y permisos pueden modelar necesidades distintas. Por ejemplo:

```
User.builder()
    .username("admin")
    .password(encoded)
    .roles("ADMIN", "USER")
    .build();
```

produce ROLE_ADMIN y ROLE_USER. Si en cambio se llama a `.authorities("READ_ALUMNOS")`, no se añade ningún prefijo.

El error típico consiste en guardar ADMIN, construir luego una authority ADMIN y proteger la ruta con `hasRole("ADMIN")`: la regla buscará ROLE_ADMIN y devolverá 403. En este curso usamos roles para USER/GESTOR/ADMIN y mantenemos visible el momento en el que se añade el prefijo. Si el dominio necesitara capacidades granulares, esas capacidades podrían expresarse como authorities independientes. > **Pregunta de reflexión:** ¿Qué diferencia hay entre `.roles("ADMIN")` y `.authorities("ADMIN")`? ¿Qué authority se crea en cada caso?

Respuesta razonada: `.roles("ADMIN")` aplica la convención de Spring Security y crea la authority ROLE_ADMIN. `.authorities("ADMIN")` crea exactamente ADMIN, sin prefijo. Por eso `hasRole("ADMIN")` y `hasAuthority("ADMIN")` no consultan necesariamente el mismo valor.

Bloque 3 - Cifrado de contraseñas

T3.1 - Por qué no se guardan contraseñas en texto plano

Una base de datos comprometida no debe revelar las contraseñas originales. Por eso se almacenan hashes calculados con algoritmos diseñados para contraseñas, deliberadamente costosos y con sal aleatoria. El objetivo no es “cifrar y descifrar”, sino verificar una entrada sin recuperar el secreto original.

Guardar texto plano o usar hashes rápidos como SHA-256 sin un esquema específico de password hashing es una mala práctica. Spring Security nos obliga además a declarar cómo se codifican las credenciales.

Una contraseña es un secreto verificable, no un dato que la aplicación necesite recuperar. Por eso se almacena mediante una función de derivación resistente a fuerza bruta, no mediante cifrado reversible. Si la base de datos se filtra, el atacante no debería obtener inmediatamente las contraseñas originales.

También hay que evitar la falsa seguridad de aplicar SHA-256 directamente. Un hash rápido favorece al atacante. BCrypt incorpora sal y un coste configurable precisamente para encarecer intentos masivos.

Una contraseña no debe tratarse como un dato reversible. El servidor necesita comprobarla, no recuperarla. Por eso se almacena el resultado de una función específica de password hashing. Si una base de datos se filtra, guardar texto plano expone inmediatamente todas las credenciales; usar un hash rápido como SHA-256 sin un esquema de contraseñas facilita probar millones de candidatos.

Los algoritmos de password hashing incorporan sal y un coste deliberado. La sal evita que dos usuarios con la misma contraseña tengan necesariamente el mismo hash y dificulta el uso de tablas precalculadas. El coste ralentiza cada intento y penaliza los ataques masivos. Spring Security

hace explícita esta decisión mediante `PasswordEncoder`, lo que obliga a declarar cómo se codifican y comparan las credenciales.

T3.2 - BCryptPasswordEncoder

BCrypt incorpora sal y un factor de coste. Dos usuarios con la misma contraseña pueden terminar con hashes distintos, y `matches` puede comprobar ambos correctamente. El coste configurable dificulta ataques masivos de fuerza bruta.

No compararemos strings de hashes de forma manual. El servicio de registro llamará a `PasswordEncoder.encode(...)` y el proveedor de autenticación utilizará el mismo encoder al validar el login.

BCryptPasswordEncoder genera hashes con sal y coste. El parámetro de coste aumenta el trabajo requerido; subirlo mejora resistencia pero también consume CPU en cada autenticación. La elección debe equilibrar seguridad y capacidad del sistema.

El punto pedagógico importante es observar que `encode("clave")` no produce un valor fijo. Para verificar se usa `matches`. Esto explica por qué un test que espera una cadena BCrypt exacta está mal diseñado; debe comprobar que el encoder reconoce la contraseña correcta y rechaza otra.

BCrypt genera una sal aleatoria e incorpora en el propio hash la información necesaria para comprobarlo posteriormente. Por eso dos llamadas a `encode` con la misma contraseña pueden producir cadenas diferentes y, aun así, ambas validar correctamente con `matches`.

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String hash = encoder.encode("ana123");
boolean ok = encoder.matches("ana123", hash);
```

Su factor de coste es adaptable: aumentar el coste incrementa el trabajo tanto para el servidor como para un atacante que pruebe contraseñas por fuerza bruta. El objetivo no es elegir una cifra universal, sino ajustar el coste al hardware y al contexto. En ningún caso se debe comparar manualmente `encode(raw).equals(hash)`, porque la sal hace que esa lógica sea incorrecta.

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String hash = encoder.encode("ana123");
boolean ok = encoder.matches("ana123", hash);
```

El ejemplo completo de BCrypt muestra tanto la codificación como la verificación; no debe compararse un hash regenerado como si fuera determinista:

```
PasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

String hash = passwordEncoder.encode("miContraseña");
// hash es algo como: $2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy

boolean coincide = passwordEncoder.matches("miContraseña", hash);
// coincide es true
```

T3.3 - DelegatingPasswordEncoder

DelegatingPasswordEncoder guarda un identificador de algoritmo junto al hash, por ejemplo `{bcrypt}`. Esto permite que una aplicación reconozca hashes creados con esquemas distintos y pueda migrarlos gradualmente.

Para un curso acumulativo es una opción especialmente útil: mantiene la configuración flexible sin obligarnos a codificar BCrypt directamente en cada clase. El bean de `PasswordEncoder` será compartido por registro y autenticación.

DelegatingPasswordEncoder añade un prefijo como `{bcrypt}` que permite identificar el algoritmo usado por cada hash. Esa capacidad facilita migraciones graduales: cuentas antiguas y nuevas pueden coexistir mientras se actualizan credenciales.

Aunque el proyecto del curso puede funcionar con un encoder BCrypt directo, conocer el delegado evita asumir que todos los hashes del mundo tienen el mismo formato. El principio es mantener la elección del algoritmo en una frontera configurable, no dispersa por servicios y controladores.

Ejemplo.

```
boolean ok = passwordEncoder.matches(passwordEnClaro, hashPersistido);

Los prefijos permiten identificar el algoritmo que produjo cada hash:
{bcrypt}$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
{argon2}$argon2id$v=19$m=65536,t=3,p=4$...
```

```
{noop}contraseñaEnTextoPlano
```

El bean recomendado para esta estrategia es:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

Pregunta de reflexión: ¿Por qué BCrypt es más seguro que un hash simple como SHA-256? ¿Qué aporta el salt?

Respuesta razonada: BCrypt está diseñado para contraseñas: es deliberadamente costoso y permite aumentar el factor de trabajo. Además incorpora un salt aleatorio, de modo que la misma contraseña no produce siempre el mismo hash y se dificultan tablas precalculadas y ataques masivos. SHA-256 directo es demasiado rápido y no resuelve por sí solo esas propiedades.

Bloque 4 - Integración con la cadena de filtros

T4.1 - Cómo se conecta el UserDetailsService con la autenticación

Cuando llega una autenticación de usuario/contraseña, Spring Security busca un proveedor capaz de procesarla. Ese proveedor carga el usuario mediante UserDetailsService, compara la contraseña con PasswordEncoder y, si todo es correcto, crea un Authentication autenticado con sus authorities.

El controlador no necesita conocer estos detalles. Esa separación nos permitirá reutilizar exactamente el mismo mecanismo desde el login JWT de 6.6.

Cuando llega una autenticación de usuario/contraseña, un proveedor carga el UserDetails, obtiene su hash y consulta al PasswordEncoder. Si coincide, construye una autenticación válida con las authorities. UserDetailsService no recibe la contraseña en claro para decidir manualmente.

Este recorrido explica por qué tres piezas deben ser coherentes: username, hash y authorities. Si el usuario se carga pero el encoder no coincide, hay 401; si autentica pero no tiene el rol requerido, habrá 403.

En el flujo habitual Spring usa un DaoAuthenticationProvider. Ese proveedor recibe una petición de autenticación, llama al UserDetailsService para cargar la identidad y después usa el PasswordEncoder para verificar la contraseña presentada frente a la codificada. Si ambas piezas están registradas como beans, Spring puede montar el proveedor y el AuthenticationManager que las coordina.

La separación de responsabilidades es la clave: UserDetailsService carga; PasswordEncoder verifica; el provider combina ambas tareas y produce un Authentication autenticado cuando todo coincide. Por eso el cambio de memoria a base de datos en 6.3 no requiere reescribir el filtro HTTP Basic ni la semántica de autorización: cambia la implementación que responde a loadUserByUsername.

T4.2 - HTTP Basic y el flujo de autenticación

HTTP Basic transporta usuario:password codificado en Base64 en cada petición. Base64 no es cifrado, así que sólo debe utilizarse sobre HTTPS. En M6 se usa como herramienta temporal para observar el flujo de autenticación antes de introducir JWT.

Cuando el filtro Basic acepta las credenciales, el SecurityContext queda poblado para el resto de la petición. Más adelante retiraremos HTTP Basic y el contexto será creado a partir del Bearer token.

HTTP Basic transporta usuario y contraseña codificados en Base64 en cada petición; Base64 no cifra nada. En producción sólo tendría sentido sobre TLS y, aun así, no es el mecanismo final de este módulo. Lo usamos porque hace visible el flujo de autenticación con pocas piezas.

Al migrar a JWT desactivaremos Basic. Ese cambio debe verificarse: una cabecera Basic que antes autenticaba ya no debe convertirse accidentalmente en una puerta alternativa cuando la arquitectura final exige Bearer.

El recorrido de una petición Basic permite ver todas las piezas conectadas: el cliente envía Authorization: Basic ...; BasicAuthenticationFilter decodifica usuario y contraseña; crea un UsernamePasswordAuthenticationToken todavía no autenticado y lo entrega al AuthenticationManager. Éste delega en DaoAuthenticationProvider, que carga al usuario mediante UserDetailsService y compara la contraseña con PasswordEncoder.

Si la comparación es correcta, vuelve un `Authentication` autenticado y el filtro lo guarda en el `SecurityContext`. Después la petición llega a `AuthorizationFilter`, que aplica las reglas. Sólo si la identidad posee los permisos adecuados continúa hasta el controlador. Si faltan credenciales o son inválidas, el flujo termina con 401 y la cabecera propia de HTTP Basic.

T4.3 - Ver el `SecurityContext` en el controlador

Una vez autenticado, Spring permite acceder a la identidad mediante `Authentication`, `Principal` o anotaciones como `@AuthenticationPrincipal`. El objeto contiene el principal y las authorities que se utilizarán en autorización.

Consultar el propio perfil es una buena forma de comprobar el resultado del pipeline: la respuesta debe reflejar username, email y roles obtenidos de la identidad actual, no valores enviados por el cliente.

Tras autenticar, Spring guarda el `Authentication` en el `SecurityContextHolder` durante la petición. Desde un controlador puede inyectarse `Authentication` o `Principal`, y desde una regla puede consultarse mediante expresiones de seguridad.

El contexto no debe convertirse en un almacén global de negocio. Se usa para saber quién realiza la operación; los datos del dominio se obtienen a partir de un identificador fiable. En modo stateless el contexto se reconstruye para cada petición a partir del token.

Ejemplo.

Authorization: Basic ... -> proveedor -> UserDetailsService -> PasswordEncoder -> SecurityContext

El contexto puede consultarse directamente cuando una capa de infraestructura lo necesita:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();
```

En un controlador es más explícito recibir la autenticación como parámetro:

```
@GetMapping("/perfil")
public Map<String, Object> perfil(Authentication auth) {
    return Map.of(
        "usuario", auth.getName(),
        "authorities", auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .toList()
    );
}
```

Pregunta de reflexión: ¿Por qué es mejor inyectar `Authentication` como parámetro que usar `SecurityContextHolder` directamente?

Respuesta razonada: Inyectar `Authentication` hace explícita la dependencia del método respecto a la identidad actual y facilita lectura y prueba. `SecurityContextHolder` es útil en infraestructura o cuando no puede inyectarse el contexto, pero introduce una dependencia estática/global más difícil de aislar.

Bloque 5 - Testing y buenas prácticas

T5.1 - Probar la autenticación con curl

curl permite inspeccionar códigos, cabeceras y cuerpo sin depender de una interfaz gráfica. En la etapa Basic usaremos `-u usuario:password`; en la etapa JWT enviaremos `Authorization: Bearer <token>`.

La prueba correcta incluye al menos tres casos: sin credenciales, credenciales erróneas y credenciales válidas. Sólo el tercero debe alcanzar el recurso protegido.

`curl -u usuario:clave` permite comparar tres escenarios: credenciales ausentes, incorrectas y correctas. La observación no debe limitarse al status; conviene comprobar qué endpoint se alcanza, qué identidad aparece y qué respuesta recibe un rol insuficiente.

Separar esos casos previene falsos positivos. Si “cualquier autenticado” obtiene 200 en una ruta que debía ser ADMIN, el login funciona pero la autorización está mal. El test debe expresar ambas propiedades.

Con curl, el flag `-u` construye la cabecera Basic por nosotros:

```
curl -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

También puede enviarse manualmente una cabecera `Authorization: Basic <base64(usuario:contraseña)>`, aunque `-u` es menos propenso a errores. Con `-v` se pueden inspeccionar las cabeceras realmente transmitidas.

```
curl -v -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

Base64 es una codificación, no cifrado. HTTP Basic vuelve a enviar la credencial en cada petición; por eso fuera del experimento local debe usarse sobre HTTPS. Ver la cabecera codificada sirve precisamente para entender por qué TLS no es opcional en un mecanismo de este tipo.

Con `curl`, el flag `-u` construye la cabecera Basic por nosotros:

```
curl -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

```
curl -v -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

Las tres variantes de `curl` permiten observar el mismo mecanismo desde distintos niveles:

```
curl -u ana:contraseña http://localhost:8080/api/v1/alumnos
```

```
curl -H "Authorization: Basic YW5hOmNvb3RyYXNl7Fh" \
  http://localhost:8080/api/v1/alumnos
```

```
curl -v -u ana:contraseña http://localhost:8080/api/v1/alumnos
```

T5.2 - Buenas prácticas con usuarios en memoria

Los usuarios en memoria deben considerarse configuración temporal o de test. No conviene poner contraseñas reales en el repositorio ni conservar este mecanismo como solución de producción si la aplicación necesita gestión de identidades.

Al terminar 6.2 habrá una restauración explícita: la configuración en memoria se elimina cuando 6.3 introduce `UsuarioDetailsService` respaldado por JPA.

Las cuentas in-memory nunca deben colarse en el estado final por comodidad. Tampoco deben usar contraseñas en texto claro o roles distintos a los documentados. El propósito del ejemplo es pedagógico y su vida útil está acotada.

Una buena práctica es declarar explícitamente el paso que las crea y el que las retira. Dejar claro cuándo se introduce y cuándo se retira cada simplificación evita que una configuración de demostración termine accidentalmente en producción.

Las reglas del ejercicio son deliberadamente estrictas: `InMemoryUserDetailsManager` queda limitado a desarrollo y tests; las contraseñas se codifican con un `PasswordEncoder`; los usuarios se concentran en un bean y no se dispersan por propiedades o clases arbitrarias; y la configuración de identidades se mantiene separada de la cadena de filtros.

También conviene documentar qué usuarios de desarrollo existen y qué roles poseen, evitar contraseñas compartidas como costumbre de trabajo y mantener los nombres de roles consistentes en configuración, código y tests. Estas prácticas no convierten a los usuarios en memoria en una solución productiva; hacen que el estado temporal sea comprensible y que su eliminación en 6.3 pueda hacerse sin dejar credenciales ocultas.

T5.3 - Testing de usuarios en memoria

Un test de seguridad puede declarar usuarios simulados sin depender de la configuración en memoria. `@WithMockUser` resulta útil para probar autorización; si necesitamos verificar la carga real de usuarios, conviene usar tests que involucren el `UserService` efectivo.

La idea clave es distinguir test de política de acceso y test de autenticación real. Ambos son necesarios, pero responden a preguntas distintas.

Ejemplo.

```
@Bean
UserService userDetailsService>PasswordEncoder encoder) {
    UserDetails ana = User.withUsername("ana")
        .password(encoder.encode("ana123"))
        .roles("USER")
        .build();
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN")
        .build();
    return new InMemoryUserDetailsManager(ana, admin);
}
```

Este bean existe sólo durante la etapa en memoria y desaparece al pasar a usuarios persistentes.

Los tests de esta fase deben demostrar que el encoder, los usuarios y las reglas se conectan correctamente. Un caso mínimo incluye usuario válido, contraseña errónea, acceso con rol adecuado y acceso con rol insuficiente.

No hay que sobreajustar tests a detalles internos del framework. Es más valioso afirmar `status().isForbidden()` para un USER en una ruta ADMIN que comprobar qué clase privada de filtro lanzó la excepción. Los tests deben sobrevivir a refactorizaciones que conservan el contrato.

Ejemplo.

```
curl -i -u ana:clave-de-ejemplo http://localhost:8080/api/v1/perfil
```

Para aislar autorización puede simularse una identidad:

```
@Test
@WithMockUser(username = "ana", roles = "USER")
void consultar_debeDevolver200_cuandoAutenticado() throws Exception {
    // ...
}
```

Si el objetivo es atravesar HTTP Basic y validar usuario/contraseña, MockMvc ofrece el post-processor correspondiente:

```
mockMvc.perform(get("/api/v1/alumnos")
    .with(httpBasic("ana", "contraseña")))
    .andExpect(status().isOk());
```

Pregunta de reflexión: ¿Qué diferencia hay entre `@WithMockUser` y `httpBasic` en un test?

Respuesta razonada: `@WithMockUser` coloca directamente una identidad simulada en el contexto y sirve para probar autorización sin recorrer la autenticación real. `httpBasic(...)` envía credenciales en la petición y permite probar también el mecanismo HTTP Basic y la validación de usuario/contraseña.

Resumen de la teoría

- `UserDetails` define el contrato que Spring Security necesita para representar una identidad autenticable.
- `UserDetailsService` localiza usuarios; `PasswordEncoder` verifica contraseñas; son responsabilidades distintas.
- `InMemoryUserDetailsManager` permite aprender el flujo completo sin persistencia, pero es una solución temporal.
- Los roles son authorities con la convención `ROLE_`; `hasRole("ADMIN")` busca `ROLE_ADMIN`.
- HTTP Basic envía credenciales en cada petición y permite observar con claridad 401 frente a 403.

Punto 6.3 - Usuarios en base de datos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué los usuarios en memoria no sirven para producción.
2. Modelar entidades Usuario y Rol con una relación @ManyToMany.
3. Implementar un `UserDetailsService` personalizado que carga usuarios desde la base de datos.
4. Integrar el `UserDetailsService` personalizado con la cadena de filtros de Spring Security.
5. Cifrar contraseñas al registrar usuarios.
6. Diagnosticar y resolver los errores más comunes al usar usuarios en base de datos.

Bloque 1 - Por qué usuarios en base de datos

T1.1 - Las limitaciones de los usuarios en memoria

Una colección en memoria no permite administrar identidades de forma persistente. No hay altas duraderas, cambios de contraseña fiables, asignación de roles desde datos ni continuidad entre reinicios. Tampoco podemos consultar usuarios desde otras partes del dominio.

Por eso 6.3 traslada el modelo a JPA. La autenticación seguirá consumiendo `UserDetails`, pero esos datos se construirán a partir de entidades `Usuario` y `Rol`.

Las identidades en memoria no permiten administración persistente, auditoría de altas/bajas ni cambios de contraseña duraderos. Al pasar a base de datos la seguridad entra en el ciclo de vida real del dominio: los usuarios existen entre reinicios y sus roles pueden evolucionar.

Esta transición añade riesgos: unicidad, relaciones JPA, carga perezosa y transacciones. Por eso no se cambia todo a la vez sin pruebas. Primero se modela, luego se adapta a `UserDetailsService` y finalmente se elimina la fuente temporal.

La memoria deja de ser adecuada en cuanto la aplicación necesita gestionar identidades reales. Los usuarios deben sobrevivir a reinicios, poder registrarse, cambiar contraseña o estado, recibir y perder roles, y ser consultados de forma transaccional. Además, una aplicación con varios nodos no puede depender de una colección distinta dentro de cada proceso.

El objetivo de 6.3 no es cambiar el significado de autenticación, sino cambiar su fuente de verdad. `UserDetailsService` seguirá ofreciendo la misma interfaz, pero cargará la identidad desde JPA. Así se conserva el flujo aprendido en 6.2 mientras se añade persistencia. Al final del punto no debe quedar activo simultáneamente un conjunto de usuarios in-memory, porque existirían dos orígenes capaces de autenticar nombres distintos.

T1.2 - Modelo de datos: Usuario y Rol

`Usuario` representa una cuenta autenticable: nombre, email, password codificado, estado y roles. `Rol` representa una autoridad reutilizable. Separarlos evita duplicar cadenas de permisos en cada fila de usuario y permite asignar varias autoridades a una misma identidad.

Los nombres de rol se almacenan con la convención `ROLE_*`, de manera que la conversión a `GrantedAuthority` sea directa y consistente.

Separar `Usuario` y `Rol` evita repetir cadenas de rol en cada usuario y permite relaciones consistentes. `Usuario` almacena identidad, hash, email, estado y fecha; `Rol` representa una autoridad asignable. La API nunca debe devolver el hash de contraseña.

El modelo debe imponer invariantes útiles —username y email únicos, campos obligatorios— pero la base de datos no sustituye la validación de servicio. Una violación de unicidad debe convertirse en un error de negocio comprensible, no filtrarse como stack trace.

El modelo separa la identidad (`Usuario`) de la clasificación de permisos (`Rol`). `Usuario` contiene, entre otros datos, username único, contraseña codificada, email, estado activo y la colección de roles. `Rol` tiene un nombre único y puede incluir una descripción.

La separación evita almacenar una cadena de roles incrustada en el usuario y permite reutilizar el mismo rol para muchas identidades. También facilita operaciones administrativas posteriores, como cambiar los roles sin tocar las credenciales. El diseño debe mantener la contraseña fuera de los DTO de salida y aplicar restricciones de unicidad al username y al email, porque la autenticación necesita identificar inequívocamente a la cuenta.

T1.3 - @ManyToMany: la relación entre Usuario y Rol

Una cuenta puede tener varios roles y un mismo rol puede pertenecer a muchas cuentas. JPA modela esta situación con `@ManyToMany` y una tabla intermedia. La colección de roles debe inicializarse y mantenerse de forma predecible para evitar nulos o duplicados.

No necesitamos serializar la relación como JSON para autenticar. La entidad se usa internamente; hacia el exterior expondremos DTOs que contienen sólo los datos necesarios.

La relación muchos-a-muchos encaja porque un usuario puede tener varios roles y un rol pertenecer a muchos usuarios. La tabla intermedia materializa esa relación. Conviene definir claramente el propietario de la asociación y evitar cascadas que puedan borrar roles compartidos al eliminar un usuario.

También importa cuándo se cargan los roles. La autenticación necesita authorities mientras construye el principal; por eso la consulta o la transacción debe garantizar que esa colección está disponible sin provocar `LazyInitializationException`.

En JPA, la tabla de unión necesita claves foráneas coherentes y una colección que no se sustituya accidentalmente con una lista inmutable. También hay que decidir fetch y cascadas: cargar todos los roles de toda la aplicación de forma eager puede ser innecesario, pero la autenticación sí necesita acceder a los roles del usuario actual dentro de una frontera transaccional válida. La

solución del curso busca claridad y seguridad, no convertir la relación muchos-a-muchos en una excusa para cascadas destructivas.

Ejemplo.

```
usuarios(id, username, password, activo)
roles(id, nombre)
usuarios_rols(usuario_id, rol_id)
```

El lado propietario puede modelarse así:

```
@Entity
@Table(name = "usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "usuarios_rols",
        joinColumns = @JoinColumn(name = "usuario_id"),
        inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();
}
```

El lado inverso referencia la colección propietaria mediante mappedBy:

```
@Entity
@Table(name = "roles")
public class Rol {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();
}
```

Pregunta de reflexión: ¿Por qué en @ManyToMany hay que decidir un lado propietario? ¿Qué pasaría si ambos tuvieran @JoinTable?

Respuesta razonada: JPA necesita un lado propietario que gestione la tabla de unión y un lado inverso que la referencie con mappedBy. Si ambos definieran su propio @JoinTable, se modelarían dos relaciones distintas o un esquema incoherente, con escrituras duplicadas o inesperadas.

Bloque 2 - UserDetailsService personalizado

T2.1 - Por qué un UserDetailsService personalizado

La implementación personalizada conecta Spring Security con nuestro repositorio. Dado un username, busca Usuario; si no existe, informa de identidad desconocida; si existe, lo transforma a un principal con password codificado y authorities.

La clase no debe implementar lógica de login ni generar tokens. Su responsabilidad es pequeña y reutilizable: cargar una identidad de seguridad desde la fuente de datos.

El adaptador personalizado permite que Spring autentique contra nuestro modelo sin hacer que la entidad implemente toda la interfaz de seguridad. Esta capa traduce de Usuario/Rol a UserDetails/authorities y centraliza reglas como usuario activo.

La separación mejora pruebas y mantenimiento. El repositorio sabe buscar; el adaptador sabe traducir al contrato de Spring; el servicio de negocio sabe registrar o modificar cuentas. Mezclar esas responsabilidades en el controlador produce código difícil de asegurar.

La implementación personalizada existe porque nuestro esquema de dominio no tiene por qué coincidir con el esquema JDBC predefinido por Spring Security. Queremos buscar en `UsuarioRepository`, respetar el campo activo y transformar la relación de roles al formato de authorities que espera Spring.

Al implementar `UserDetailsService` mantenemos estable el contrato del framework y aislamos la traducción en una sola clase. Si mañana cambian las tablas o aparecen nuevos datos de identidad, el resto de la autenticación no necesita conocer esos detalles. El método debe buscar por username y lanzar `UsernameNotFoundException` cuando no exista; devolver null o una excepción de persistencia sin traducir rompe el contrato que espera el proveedor de autenticación.

El contrato que implementaremos sigue siendo exactamente el mismo:

```
UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException;
```

T2.2 - Estructura del `UserDetailsService` personalizado

Una implementación típica depende de `UsuarioRepository`, ejecuta una consulta como `findByUsername` y devuelve `UsuarioPrincipal`. Si la relación de roles es lazy, la consulta o el contexto transaccional deben garantizar que las authorities estén disponibles cuando se construye el principal.

Mantener esta transformación en un único lugar evita que controladores y servicios repitan cómo se interpreta un usuario JPA como identidad de Spring Security.

Una implementación típica busca por username, falla explícitamente si no existe y construye un `UsuarioPrincipal` con id, username, password codificada y authorities. No debe hacer comparaciones de contraseña ni emitir tokens: esas tareas pertenecen a otras capas.

El resultado debe ser determinista. Dos consultas del mismo usuario deben producir el mismo conjunto de authorities mientras la base de datos no cambie. Ordenar o usar conjuntos ayuda a evitar tests frágiles por orden accidental.

La estructura del servicio es sencilla pero sus detalles importan: se inyecta `UsuarioRepository`, se localiza el usuario y se construye un `UserDetails` con username, password ya codificado, authorities y estado de la cuenta. La carga suele ejecutarse en una transacción de sólo lectura, especialmente si la colección de roles necesita estar disponible durante la transformación.

```
@Override
@Transactional(readOnly = true)
public UserDetails loadUserByUsername(String username) {
    Usuario u = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(username));
    // transformar roles y construir UserDetails
}
```

El servicio no codifica de nuevo la contraseña al leerla ni decide si es correcta. Esa comprobación sigue perteneciendo al `PasswordEncoder` utilizado por el proveedor de autenticación.

```
@Override
@Transactional(readOnly = true)
public UserDetails loadUserByUsername(String username) {
    Usuario u = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(username));
    // transformar roles y construir UserDetails
}
```

Una implementación completa puede cargar el usuario, transformar los roles y construir el `UserDetails` sin comparar contraseñas manualmente:

```
@Service
public class UsuarioDetailsService implements UserDetailsService {
    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
```

```

    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(
            "Usuario no encontrado: " + username));

    List<GrantedAuthority> authorities = usuario.getRoles().stream()
        .map(rol -> (GrantedAuthority)
            new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
        .toList();

    return User.builder()
        .username(usuario.getUsername())
        .password(usuario.getPassword())
        .authorities(authorities)
        .accountExpired(false)
        .accountLocked(!usuario.isActivo())
        .credentialsExpired(false)
        .disabled(!usuario.isActivo())
        .build();
}
}

```

T2.3 - Transformación de roles a authorities

Cada `Rol.nombre` se convierte en `SimpleGrantedAuthority`. Si el rol se almacena como `ROLE_ADMIN`, esa misma cadena debe llegar al `Authentication`.

La coherencia de nombres es crítica: `hasRole("ADMIN")` y `hasAuthority("ROLE_ADMIN")` deben producir el mismo resultado si la authority está bien formada. Los tests de autorización comprobarán esta equivalencia práctica.

La transformación de roles debe respetar la convención elegida. Si la base almacena `ADMIN`, el adaptador puede producir `SimpleGrantedAuthority("ROLE_ADMIN")`. Si ya almacena `ROLE_ADMIN`, añadir otro prefijo daría `ROLE_ROLE_ADMIN` y todas las reglas fallarían.

Esta traducción merece un test unitario porque un error de una sola cadena bloquea toda la autorización. El test debe verificar tanto el rol esperado como la ausencia de duplicaciones o prefijos incorrectos.

Ejemplo.

```

Usuario u = repo.findByUsername(username)
    .orElseThrow(() -> new UsernameNotFoundException(username));

```

Los roles persistidos se transforman en `GrantedAuthority`. Si la tabla almacena `ADMIN`, el adaptador puede construir `ROLE_ADMIN`:

```

List<GrantedAuthority> authorities = usuario.getRoles().stream()
    .map(rol -> new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
    .toList();

```

Este es el punto exacto en el que se fija la convención. Añadir el prefijo dos veces produciría `ROLE_ROLE_ADMIN`; no añadirlo y usar después `hasRole("ADMIN")` haría que la autorización fallara. La transformación debe ser determinista y estar cubierta por tests. También es importante que los roles estén disponibles durante la lectura: una colección lazy fuera de la transacción puede provocar errores antes incluso de que Spring pueda autenticar.

Los roles persistidos se transforman en `GrantedAuthority`. Si la tabla almacena `ADMIN`, el adaptador puede construir `ROLE_ADMIN`:

```

List<GrantedAuthority> authorities = usuario.getRoles().stream()
    .map(rol -> new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
    .toList();

```

Este es el punto exacto en el que se fija la convención. Añadir el prefijo dos veces produciría `ROLE_ROLE_ADMIN`; no añadirlo y usar después `hasRole("ADMIN")` haría que la autorización fallara. La transformación debe ser determinista y estar cubierta por tests. También es importante que los roles estén disponibles durante la lectura: una colección lazy fuera de la transacción puede provocar errores antes incluso de que Spring pueda autenticar.

La alternativa **sin** prefijo también existe, pero cambia el contrato de autorización:

```

.map(rol -> new SimpleGrantedAuthority(rol.getNombre()))

```

Por eso no puede mezclarse silenciosamente con `hasRole("ADMIN")`, que espera `ROLE_ADMIN`.

Pregunta de reflexión: ¿Qué pasaría si guardaras los roles en la base de datos con prefijo ROLE_ADMIN y luego los usaras con hasRole("ADMIN")?

Respuesta razonada: hasRole("ADMIN") busca la authority ROLE_ADMIN. Si la transformación desde la base de datos añadiera además el prefijo a un valor ya guardado como ROLE_ADMIN, acabaríamos con ROLE_ROLE_ADMIN. La convención debe elegirse una sola vez y aplicarse de forma coherente.

Bloque 3 - Integración con la configuración

T3.1 - Cómo detecta Spring Security el UserDetailsService

Cuando el contexto contiene un único bean UserDetailsService y un PasswordEncoder compatible, Spring Boot puede construir el proveedor de autenticación necesario. Aun así, el login explícito obtendrá AuthenticationManager desde AuthenticationConfiguration, evitando crear gestores manualmente.

La configuración final no declara usuarios estáticos. La presencia del servicio JPA es suficiente para que la autenticación usuario/contraseña use la base de datos.

Spring puede utilizar un UserDetailsService bean junto con un PasswordEncoder para construir el proveedor de autenticación adecuado. La magia aparente se vuelve comprensible si vemos cada bean como un contrato que la auto-configuración conecta.

Si aparecen varios UserDetailsService sin una intención clara, la configuración deja de ser obvia. Por eso al terminar la transición debe existir una única fuente de usuarios para el flujo normal: la persistente.

Spring Security no necesita que registremos a mano un provider distinto si existe un único UserDetailsService adecuado y un PasswordEncoder. Boot puede usar esos beans para configurar la autenticación basada en usuario y contraseña. Por eso, al pasar a base de datos, la operación crítica es retirar la configuración in-memory y dejar una única fuente de usuarios.

Mantener dos UserDetailsService sin una configuración explícita genera ambigüedad y hace difícil saber cuál autentica. La transición del curso es intencionadamente limpia: 6.2 demuestra memoria; 6.3 elimina ese bean y registra el servicio respaldado por UsuarioRepository. El SecurityFilterChain sigue ocupándose de HTTP y autorización, mientras la fuente de identidad cambia por debajo.

T3.2 - Configuración de la cadena de filtros

La cadena HTTP no necesita saber cómo se persisten usuarios. Su trabajo es definir rutas, sesión, filtros y manejadores. Esta separación permite evolucionar el almacenamiento de identidades sin reescribir la política HTTP.

En 6.3 todavía podemos conservar HTTP Basic para comprobar la nueva fuente de usuarios. Más adelante será sustituido por JWT, pero UserDetailsService seguirá siendo útil para login y reconstrucción del principal.

La cadena de filtros sigue protegiendo rutas, pero ahora la autenticación delega en usuarios persistentes. Es importante no abrir globalmente /api/v1/auth/**: registro, login y refresh tienen riesgos diferentes y deben declararse explícitamente.

Una allowlist concreta funciona además como documentación de superficie pública. Si mañana se añade /api/v1/auth/admin-reset, no debería quedar público sólo por compartir prefijo con login.

La cadena de filtros no necesita conocer detalles JPA. Continúa expresando qué rutas son públicas y cuáles requieren autenticación, y mantiene HTTP Basic durante esta fase para poder probar las cuentas persistidas. El registro se abre explícitamente; el resto se protege según las reglas del punto.

La clave arquitectónica es que SecurityFilterChain define política HTTP, mientras UserDetailsService define cómo cargar credenciales. Mezclar ambas responsabilidades llevaría a consultas de repositorio dentro de la configuración web. En esta etapa también se conserva el tratamiento de CSRF previsto para la API y la configuración CORS heredada: introducir usuarios de base de datos no debe romper contratos que ya funcionaban en M5.

El paso de memoria a base de datos no obliga a rehacer la cadena HTTP Basic; la configuración web puede mantenerse mientras cambia el UserDetailsService:

@Bean

```

public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}

```

T3.3 - Inicialización de usuarios

Para poder probar el módulo necesitamos roles y usuarios conocidos en el perfil de desarrollo. El inicializador debe ser **idempotente**: si un rol o usuario ya existe, no debe crear duplicados ni cambiar contraseñas de forma inesperada en cada arranque.

La idempotencia convierte los datos de desarrollo en una herramienta reproducible. Además, las contraseñas se codifican con el mismo PasswordEncoder usado por la autenticación.

Un inicializador de desarrollo puede crear roles y cuentas conocidas para probar el curso. Debe ser idempotente: arrancar dos veces no debe duplicar usuarios ni roles. Las contraseñas se codifican con el mismo bean que usa el resto de la aplicación.

Los datos de demostración pertenecen a perfiles controlados; no deben convertirse en credenciales de producción. En un sistema real, el bootstrap de administradores tendría un procedimiento más seguro y auditable.

Ejemplo.

base de datos -> UsuarioDetailsService -> AuthenticationManager -> identidad autenticada

La fuente parte de un CommandLineRunner sencillo. En el proyecto acumulativo lo haremos idempotente por identidad, pero conviene conservar el ejemplo que explica el mecanismo:

```

@Configuration
public class UsuariosInicialesConfig {
    @Bean
    public CommandLineRunner cargarUsuarios(
        UsuarioRepository usuarioRepository,
        RolRepository rolRepository,
        PasswordEncoder passwordEncoder) {
        return args -> {
            if (usuarioRepository.count() > 0) {
                return;
            }

            Rol rolAdmin = rolRepository.save(
                new Rol("ADMIN", "Administrador"));
            Rol rolUser = rolRepository.save(
                new Rol("USER", "Usuario"));

            Usuario admin = new Usuario();
            admin.setUsername("admin");
            admin.setPassword(passwordEncoder.encode("admin123"));
            admin.setEmail("admin@educacion.gob.es");
            admin.setActivo(true);
            admin.getRoles().add(rolAdmin);
            admin.getRoles().add(rolUser);
            usuarioRepository.save(admin);

            Usuario ana = new Usuario();
            ana.setUsername("ana");
            ana.setPassword(passwordEncoder.encode("ana123"));
            ana.setEmail("ana@educacion.gob.es");
            ana.setActivo(true);
            ana.getRoles().add(rolUser);
            usuarioRepository.save(ana);
        };
    }
}

```

Pregunta de reflexión: ¿Por qué se comprueba `count() > 0` antes de insertar los usuarios iniciales?

Respuesta razonada: Para que el inicializador sea idempotente. Si la aplicación arranca varias veces sobre una base persistente, no debe intentar recrear los mismos usuarios y roles ni provocar violaciones de unicidad.

Bloque 4 - Registro de usuarios

T4.1 - Por qué un endpoint de registro

Un sistema con usuarios persistidos necesita un camino controlado para crear cuentas. El endpoint de registro recibe datos públicos, valida formato y unicidad, codifica la contraseña y asigna únicamente los roles permitidos por la política de la aplicación.

Nunca debe aceptar directamente una entidad `Usuario` desde JSON, porque eso permitiría al cliente manipular campos internos como roles, estado o hash de contraseña.

El registro público crea una identidad, por lo que es una operación sensible aunque esté permitida sin autenticación. Debe validar entrada, impedir duplicados, codificar la contraseña y asignar sólo el rol permitido por la política. Nunca debe aceptar del cliente “quiero ser ADMIN”.

El endpoint demuestra una diferencia importante: público no significa sin reglas. La ruta no exige una identidad previa, pero el servicio aplica más invariantes que muchas rutas autenticadas.

Una base de datos permite que las cuentas dejen de ser únicamente datos de arranque. El endpoint de registro recibe datos públicos de alta, valida duplicados, codifica la contraseña y asigna un rol inicial controlado por el servidor. El cliente no debe poder elegir libremente ADMIN en el JSON de registro.

La respuesta tampoco debe devolver la contraseña ni su hash. Por eso el flujo usa DTOs diferentes del modelo JPA: el DTO de entrada contiene los campos que el cliente puede proponer; el DTO de salida sólo expone información segura. El endpoint puede ser público porque precisamente permite obtener una identidad, pero su lógica de negocio debe impedir usernames/emails duplicados y aplicar siempre el `PasswordEncoder` antes de persistir.

T4.2 - DTO de registro

El DTO de registro define la frontera HTTP: username, email y contraseña con restricciones de validación. Su propósito es separar el contrato de entrada del modelo persistente. Incluso si ambos comparten campos, no son la misma responsabilidad.

Las validaciones sintácticas pertenecen al DTO; las reglas que requieren consultar la base de datos, como “username único”, pertenecen al servicio.

Un DTO de registro limita el contrato de entrada a los datos necesarios. Evita exponer campos internos como activo, roles o fechaCreacion y permite aplicar validación sin contaminar la entidad JPA.

La salida utiliza otro DTO que no contiene password. Esta separación reduce la posibilidad de que una refactorización serialice accidentalmente el hash. Los tests deberían inspeccionar el JSON y comprobar explícitamente que el campo sensible no existe.

El DTO de registro es una frontera de entrada. Incluye username, password y email con las validaciones apropiadas, pero no campos internos como activo, fecha de creación o lista arbitraria de roles. Esa selección evita que el contrato HTTP quede acoplado a todas las columnas de `Usuario`.

Bean Validation permite rechazar datos incompletos antes de ejecutar el servicio. La contraseña recibida sigue estando en texto plano sólo durante el tiempo necesario para procesar la petición y nunca se serializa de vuelta. Tras validar, el servicio construye la entidad, codifica el secreto y asigna el rol USER que determine la aplicación. Separar DTO y entidad también permite cambiar el esquema de persistencia sin romper automáticamente a los clientes.

El DTO de entrada limita y valida los datos que el cliente puede decidir:

```
public class RegistroRequestDTO {
    @NotBlank(message = "El nombre de usuario es obligatorio")
    @Size(min = 3, max = 50,
        message = "El nombre de usuario debe tener entre 3 y 50 caracteres")
    private String username;

    @NotBlank(message = "La contraseña es obligatoria")
```

```
@Size(min = 8,
      message = "La contraseña debe tener al menos 8 caracteres")
private String password;

@NotBlank(message = "El email es obligatorio")
@email(message = "El email no tiene un formato válido")
private String email;
}
```

La respuesta excluye completamente la contraseña:

```
public class UsuarioResponseDTO {
    @JsonProperty("id")
    private String identificador;
    private String username;
    private String email;
    private List<String> roles;
}
```

T4.3 - Servicio de registro

AuthService coordina la operación: comprueba duplicados, obtiene el rol inicial, codifica la contraseña y persiste la cuenta. El rol por defecto debe ser una decisión de servidor, no un campo de libre elección del cliente.

Los errores de negocio se traducen a respuestas consistentes mediante el manejo de excepciones del proyecto. Así el registro se integra con el contrato de errores heredado de M5.

El servicio de registro coordina comprobaciones de unicidad, cifrado y asignación de rol dentro de una transacción. El controlador no debe replicar estas reglas. Si username o email ya existen, el servicio produce un error de negocio consistente con el contrato común de errores.

La transacción importa porque la creación del usuario y sus relaciones debe quedar completa o no quedar. Un fallo a mitad no debe producir una cuenta parcial sin roles o con datos inconsistentes.

Ejemplo.

```
{"username": "pedro", "password": "...", "email": "pedro@example.org"}
```

Pregunta de reflexión: ¿Qué controles debe aplicar el registro antes de persistir una cuenta para evitar duplicados, contraseñas en claro o asignación arbitraria de privilegios?

Respuesta razonada: El registro debe rechazar username/email duplicados, cifrar la contraseña antes de guardar y asignar únicamente roles permitidos por el servidor. La respuesta no debe incluir ni la contraseña ni su hash.

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas con usuarios en base de datos

Las entidades de autenticación contienen información sensible. No deben serializar el hash de contraseña ni exponerse de forma accidental. Los endpoints devuelven DTOs y las consultas se diseñan para cargar sólo lo necesario.

También conviene mantener separadas las operaciones administrativas de las operaciones de perfil. Un usuario puede consultar su propia identidad sin adquirir privilegios para listar o modificar cuentas ajenas.

Las cuentas persistentes requieren políticas que el ejemplo mínimo sólo introduce: contraseñas robustas, desactivación de cuentas, cambios auditables y mínimos privilegios. El módulo no pretende resolver gestión de identidad empresarial, pero debe dejar claras esas fronteras.

La contraseña nunca sale de la capa de autenticación, los roles no se confían al cliente y las rutas administrativas exigen autorización. Estas tres reglas sencillas previenen errores frecuentes en APIs educativas.

Las buenas prácticas de esta fase incluyen contraseña siempre codificada; username y email únicos; roles modelados en su propia tabla; DTOs de entrada y salida en lugar de exponer entidades; y transacciones bien delimitadas al cargar o modificar la identidad. El inicializador puede crear cuentas de desarrollo, pero debe comprobar si ya existen para no duplicarlas en cada arranque.

También conviene mantener una sola fuente de autenticación, no filtrar hashes en logs o respuestas

y tratar los cambios de contraseña con el mismo encoder que el registro. Las relaciones JPA deben cargarse de forma compatible con la autenticación, porque las authorities se necesitan durante el login. Estas reglas preparan el terreno para JWT: el token sólo será fiable si la identidad y sus roles proceden de un modelo persistente coherente.

T5.2 - Errores comunes

Los fallos habituales incluyen guardar la contraseña sin codificar, mezclar `ROLE_ADMIN` con `ADMIN`, crear usuarios iniciales duplicados, devolver la entidad `Usuario` completa o confiar en un rol enviado por el cliente durante el registro.

Otro error sutil es dejar activo el `InMemoryUserDetailsManager` al introducir JPA: dos fuentes de identidad pueden generar comportamientos difíciles de explicar. La transición debe ser explícita.

Los fallos típicos de esta transición son duplicados, hashes incompatibles, roles sin prefijo correcto, colecciones lazy fuera de sesión y coexistencia accidental del usuario in-memory. Cada uno tiene un síntoma distinto: conflicto al registrar, 401 con clave correcta, 403 inesperado, excepción JPA o autenticación “fantasma”.

Diagnosticar por síntomas obliga a inspeccionar la capa adecuada en vez de abrir temporalmente la seguridad con `permitAll()` para “hacer que funcione”.

Otro error frecuente es tratar la desactivación de una cuenta como si fuese sólo una propiedad visual. El `UserDetails` debe reflejar el estado de la cuenta para que la autenticación pueda rechazarla. Del mismo modo, cambiar roles en base de datos debe repercutir en las authorities que Spring construye según la estrategia documentada. Estos casos muestran que la seguridad persistente no consiste únicamente en “guardar usuarios”, sino en traducir correctamente su estado operativo.

Los fallos típicos de esta transición son muy concretos: conservar al mismo tiempo el `InMemoryUserDetailsManager`; guardar una contraseña sin codificar y obtener siempre `BadCredentials`; olvidar el prefijo `ROLE_`; intentar leer roles lazy después de cerrar la transacción; permitir usernames duplicados; devolver la entidad con su password; o confiar en valores de rol enviados por el cliente.

También puede fallar el arranque si el inicializador presupone roles inexistentes o si crea registros duplicados en cada ejecución. Un `UserDetailsService` debe traducir correctamente “usuario no encontrado” a `UsernameNotFoundException`, no dejar escapar una excepción genérica. Cada error tiene una causa distinta, por eso la práctica los comprueba de forma aislada antes de pasar a autorización avanzada.

T5.3 - Testing de usuarios en base de datos

Los tests deben comprobar tanto persistencia como seguridad. Necesitamos demostrar que el registro crea una cuenta con password codificado, que el login usa esos datos y que las authorities recuperadas de la base de datos gobiernan el acceso.

También conviene verificar los casos negativos: usuario duplicado, email duplicado, credenciales erróneas y ausencia del rol requerido. Un test útil no sólo confirma el “camino feliz”.

Ejemplo.

```
@Service
public class UsuarioDetailsService implements UserDetailsService {
    private final UsuarioRepository repository;

    public UsuarioDetailsService(UsuarioRepository repository) {
        this.repository = repository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        Usuario usuario = repository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        return UsuarioPrincipal.from(usuario);
    }
}
```

Los tests deben comprobar registro, autenticación posterior, duplicados y carga de roles. Para persistencia se usa H2 en perfil de test, pero la propiedad que verificamos no es “H2 funciona”: es

que el modelo y el servicio cumplen el contrato.

Un test de integración valioso registra un usuario, autentica con la contraseña original, cambia la contraseña y demuestra que la antigua deja de servir. Eso conecta varias piezas sin depender de mocks excesivos.

La base de datos de test debe empezar en un estado controlado. Si un test depende de usuarios creados por otro, el orden de ejecución puede ocultar defectos. Conviene preparar fixtures explícitos o utilizar transacciones que aíslen casos. También se prueba que el hash persistido no sea igual a la contraseña original y que la respuesta HTTP nunca lo exponga; así el test verifica seguridad del dato además de autenticación.

Ejemplo.

```
registro -> 201
registro duplicado -> 409
credenciales erróneas -> 401
```

Pregunta de reflexión: ¿Por qué es importante verificar que se lanza UsernameNotFoundException y no otra excepción?

Respuesta razonada: Porque UsernameNotFoundException forma parte del contrato que Spring Security espera de UserDetailsService. Verificar esa excepción demuestra que la ausencia de usuario se traduce por la vía de autenticación prevista y no por un fallo accidental de repositorio, null o lógica de negocio.

Resumen de la teoría

- Los usuarios reales deben persistir y poder administrarse sin recompilar la aplicación.
- Usuario y Rol se modelan con una relación @ManyToMany y una tabla de unión.
- Un UserDetailsService personalizado transforma los roles persistidos en authorities de Spring Security.
- El registro cifra la contraseña, impide duplicados y asigna un rol inicial controlado por el servidor.
- Los DTO de salida nunca exponen la contraseña ni su hash.

Punto 6.4 - Autorización por roles

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Diferenciar autorización por URL de autorización por método.
2. Usar hasRole, hasAuthority y hasAnyRole en la configuración.
3. Proteger métodos con @PreAuthorize y @Secured.
4. Aplicar expresiones SpEL en @PreAuthorize para reglas complejas.
5. Acceder a la identidad del usuario autenticado desde el servicio.
6. Diagnosticar y resolver los errores más comunes al configurar autorización.

Bloque 1 - Autorización por URL vs por método

T1.1 - Autorización por URL

Las reglas de URL se expresan en SecurityFilterChain y se evalúan antes de que el controlador ejecute su lógica. Son adecuadas para políticas transversales fáciles de describir por método HTTP y patrón de ruta: por ejemplo, permitir un endpoint público o exigir ADMIN para una zona administrativa.

Su ventaja principal es la visibilidad: una revisión de SecurityConfig permite entender gran parte del perímetro de la API. La desventaja es que no siempre capturan reglas ligadas a argumentos concretos o a decisiones de negocio.

La autorización por URL protege la superficie HTTP antes de entrar al controlador. Es idónea para reglas generales como “todo /api/v1/admin/** exige ADMIN” o “esta ruta de información es pública”. Se ve de un vistazo en SecurityFilterChain y cubre cualquier método que coincida con el matcher.

La precisión importa: método HTTP y ruta pueden combinarse para abrir sólo un POST concreto sin exponer GET, PUT o DELETE vecinos. El orden de matchers debe ir de reglas específicas a la regla final general.

T1.2 - Autorización por método

La seguridad por método se aplica sobre métodos de controlador o, preferiblemente, de servicio mediante anotaciones como `@PreAuthorize`. Permite expresar condiciones más cercanas al caso de uso y utilizar información del usuario autenticado o de los parámetros.

Para que funcione hay que activar `@EnableMethodSecurity`. Una vez activada, Spring crea proxies que interceptan la llamada al método y evalúan la expresión antes de ejecutar la lógica protegida.

La autorización por método protege una operación Java mediante `@PreAuthorize`, incluso si se invoca desde otro controlador o desde una ruta que cambie. Es útil cuando la regla pertenece semánticamente al caso de uso, no sólo a la URL.

Para que funcione se activa seguridad de método. La anotación debe expresar una política comprensible; expresiones demasiado complejas son una señal de que quizá la decisión de negocio merece un componente dedicado y tests propios.

La autorización por método acerca la regla a la operación que protege. Se habilita para poder aplicar anotaciones como `@PreAuthorize` en controladores o servicios, incluso cuando la decisión depende de parámetros o del usuario autenticado. Por ejemplo, una URL puede exigir autenticación y el método decidir además que sólo el propietario o un ADMIN puede consultar un recurso.

Esta capa resulta útil cuando la ruta no expresa todo el contexto de negocio. Una regla de “el expediente pertenece al usuario actual” no se describe bien sólo con un patrón URL. La protección por método tampoco reemplaza necesariamente a la de URL: ambas pueden coexistir como defensa en profundidad, una para áreas generales y otra para operaciones concretas.

T1.3 - Cuándo usar cada una

URL y método no son enfoques rivales. La configuración HTTP puede establecer el perímetro grueso y la capa de servicio aplicar reglas de dominio más específicas. Esa defensa en profundidad evita que un refactor de rutas elimine accidentalmente una restricción importante.

La regla práctica es colocar la política donde mejor expresa su intención y evitar duplicaciones sin propósito. Si la condición depende del propietario de un recurso, suele pertenecer al servicio; si depende del tipo de endpoint, suele ser clara en la cadena HTTP.

URL y método pueden complementarse. La cadena HTTP establece el perímetro y el método refuerza operaciones críticas. No se trata de duplicar todas las reglas, sino de colocar cada garantía en la frontera adecuada.

Un test debe considerar ambas capas. Si la URL permite ADMIN pero el método exige otra condición, el resultado final debe reflejar la intersección. Documentar sólo una de las dos produce documentación incompleta.

Ejemplo.

```
auth.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated();
```

Pregunta de reflexión: ¿Por qué se combinan las dos estrategias en lugar de usar solo una?

Respuesta razonada: Las reglas por URL protegen zonas completas del API y actúan como primera barrera; la seguridad por método expresa reglas cercanas a la operación y puede usar parámetros o el principal. Combinarlas aporta defensa en profundidad y mantiene la intención de seguridad incluso si cambia una ruta o una capa de entrada.

Bloque 2 - hasRole, hasAuthority, hasAnyRole

T2.1 - hasRole vs hasAuthority

`hasRole("ADMIN")` aplica la convención `ROLE_` y busca `ROLE_ADMIN`. `hasAuthority("ROLE_ADMIN")` compara exactamente la cadena indicada. Ambas expresiones pueden representar la misma política, pero sólo si los nombres almacenados y las authorities construidas siguen la misma convención.

Elegir una convención estable reduce errores. En M6 los roles persistidos usan `ROLE_*`, por lo que las dos formas son equivalentes cuando se emplean correctamente.

`hasRole("ADMIN")` es una forma conveniente de preguntar por `ROLE_ADMIN`; `hasAuthority("ROLE_ADMIN")` expresa la cadena completa. Ambas pueden ser correctas si la convención se respeta. El error aparece cuando almacenamiento, principal y regla usan convenciones distintas.

Antes de depurar un 403, imprime o inspecciona en test las `authorities` efectivas del principal. Ver el valor real suele revelar inmediatamente un prefijo ausente o duplicado.

`hasRole` aplica la convención de prefijo `ROLE_`, mientras `hasAuthority` compara el texto exacto. Por tanto:

```
.hasRole("ADMIN")
```

espera una `authority` `ROLE_ADMIN`; en cambio:

```
.hasAuthority("ROLE_ADMIN")
```

busca literalmente ese valor. Es un error habitual escribir `hasRole("ROLE_ADMIN")` y terminar buscando conceptualmente un prefijo duplicado.

La elección depende del modelo. Si usamos roles `USER/GESTOR/ADMIN`, `hasRole` expresa mejor la intención. Si trabajamos con permisos finos como `EXPEDIENTE_APROBAR`, `hasAuthority` puede resultar más natural. Lo fundamental es que el valor producido por `UserDetailsService` y el que consulta la regla coincidan exactamente.

`hasRole` aplica la convención de prefijo `ROLE_`, mientras `hasAuthority` compara el texto exacto. Por tanto:

```
.hasRole("ADMIN")
```

espera una `authority` `ROLE_ADMIN`; en cambio:

```
.hasAuthority("ROLE_ADMIN")
```

busca literalmente ese valor. Es un error habitual escribir `hasRole("ROLE_ADMIN")` y terminar buscando conceptualmente un prefijo duplicado.

T2.2 - `hasAnyRole` y `hasAnyAuthority`

Cuando una operación admite varias autoridades, `hasAnyRole("GESTOR", "ADMIN")` es más expresivo que duplicar reglas. La versión `hasAnyAuthority` ofrece la misma idea sin aplicar prefijos.

Estas expresiones son especialmente útiles en endpoints de gestión donde varias categorías de usuario comparten una capacidad. Los tests deben cubrir al menos un rol permitido y uno rechazado.

`hasAnyRole` y `hasAnyAuthority` expresan alternativas: basta con poseer una de las autoridades indicadas. Son útiles para operaciones compartidas por `GESTOR` y `ADMIN` sin crear reglas duplicadas.

No conviene convertir una lista creciente de roles en política opaca. Si la regla empieza a representar capacidades de negocio, puede ser más claro introducir `authorities` específicas y asignarlas a roles. El curso mantiene roles por simplicidad, pero muestra la dirección de evolución.

Las variantes `hasAnyRole` y `hasAnyAuthority` permiten aceptar cualquiera de varios valores. Una zona de gestor, por ejemplo, puede admitir tanto `GESTOR` como `ADMIN`:

```
.requestMatchers("/api/v1/gestor/**")
    .hasAnyRole("GESTOR", "ADMIN")
```

De nuevo, la variante de roles añade la convención `ROLE_`; la variante de `authorities` no transforma los nombres. Estas expresiones evitan duplicar reglas para cada rol y hacen visible la política en un solo lugar. No deben usarse para ocultar una jerarquía confusa: si las combinaciones crecen demasiado, conviene revisar el modelo de permisos en lugar de encadenar listas cada vez mayores.

Las variantes `hasAnyRole` y `hasAnyAuthority` permiten aceptar cualquiera de varios valores. Una zona de gestor, por ejemplo, puede admitir tanto `GESTOR` como `ADMIN`:

```
.requestMatchers("/api/v1/gestor/**")
    .hasAnyRole("GESTOR", "ADMIN")
```

T2.3 - Expresiones complejas con SpEL

`@PreAuthorize` puede usar SpEL para combinar roles, parámetros y datos del principal. Es posible escribir condiciones como "ADMIN o el propio usuario". Esta potencia debe usarse con moderación: una expresión demasiado larga se vuelve difícil de leer y de testear.

Cuando la decisión exige cargar datos o evaluar reglas de negocio, suele ser mejor delegar en un componente de autorización o en el servicio y mantener la expresión corta.

SpEL permite combinar identidad, argumentos del método y beans de autorización. Puede expresar, por ejemplo, "ADMIN o propietario del recurso". Esa potencia también aumenta el riesgo de reglas difíciles de leer o de probar.

Una expresión con el id del principal sólo es fiable si `UsuarioPrincipal` expone un identificador auténtico obtenido de la base de datos. Nunca se debe confiar en un id enviado por el cliente como prueba de propiedad.

Ejemplo.

```
hasRole("ADMIN")      -> busca ROLE_ADMIN
hasAuthority("ADMIN") -> busca ADMIN
```

Las expresiones pueden combinar condiciones sobre rol, identidad y authorities:

```
.hasRole("ADMIN").or().authentication().getName().equals("ana")
@PreAuthorize("#id == authentication.name or hasRole('ADMIN')")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
    // ...
}
.hasAuthority("READ_ALUMNOS")
.isAuthenticated()
.hasRole("ADMIN").and().hasAuthority("WRITE_ALUMNOS")
```

Pregunta de reflexión: ¿Qué diferencia hay entre `hasRole("ADMIN")` y `hasAuthority("ADMIN")`? ¿Cuándo usarías cada uno?

Respuesta razonada: `hasRole("ADMIN")` aplica la convención de prefijo y busca `ROLE_ADMIN`; `hasAuthority("ADMIN")` busca exactamente `ADMIN`. Usaría roles para funciones amplias del usuario y authorities explícitas cuando el modelo necesita permisos con nombres exactos o más granulares.

Bloque 3 - @PreAuthorize y @Secured

T3.1 - Activar la seguridad por método

La anotación `@EnableMethodSecurity` habilita la interceptación de métodos protegidos. Sin ella, `@PreAuthorize` puede compilar y quedar visualmente presente sin producir ningún efecto, un fallo especialmente peligroso porque parece que la aplicación está protegida.

Por eso la activación y los tests forman una pareja inseparable: un test de acceso denegado confirma que el proxy de seguridad está realmente operativo.

La seguridad por método se activa una vez mediante `@EnableMethodSecurity`. A partir de entonces los beans proxificados pueden aplicar anotaciones como `@PreAuthorize`. Si se olvida la activación, las anotaciones pueden compilar y quedar visualmente presentes sin proteger nada.

Por eso no basta con que la anotación aparezca en el código: un test en el que un USER recibe 403 al invocar la ruta protegida demuestra que la infraestructura está realmente activa.

La seguridad por método se activa explícitamente con `@EnableMethodSecurity`. Esta es la API moderna que sustituye al enfoque histórico `@EnableGlobalMethodSecurity` de versiones anteriores de Spring Security. Sin la activación, una anotación `@PreAuthorize` puede estar escrita correctamente y, aun así, no participar en la decisión de acceso.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

La interceptación se realiza mediante proxies de Spring, por lo que la ubicación del método importa: se aplica sobre beans gestionados por el contenedor y sobre invocaciones que atraviesan el proxy. Esta idea explica problemas como las llamadas internas dentro del mismo bean, que pueden saltarse la interceptación esperada.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

T3.2 - @PreAuthorize en detalle

@PreAuthorize evalúa la expresión antes de entrar en el método. Puede consultar authentication, authorities, parámetros por nombre y propiedades del principal. Si la expresión es falsa, Spring corta la ejecución y produce un error de acceso.

En el proyecto lo utilizaremos para proteger operaciones administrativas y para demostrar condiciones que no se expresan cómodamente sólo por URL.

@PreAuthorize evalúa la expresión antes de ejecutar el método. Puede consultar roles, authorities, parámetros y principal. Es adecuada para impedir que una operación sensible llegue a su lógica de negocio cuando la política no se cumple.

La expresión debe mantenerse pequeña y acompañada por tests positivos y negativos. No basta con probar ADMIN=200; también hay que probar USER/GESTOR=403 según el contrato.

@PreAuthorize se evalúa antes de ejecutar el método y acepta expresiones SpEL. Puede colocarse en métodos de controlador, métodos de servicio o incluso en una clase completa. Los casos sencillos expresan roles:

```
@PreAuthorize("hasRole('ADMIN')")
public void eliminarAlumno(Long id) { ... }
```

pero también puede combinar el rol con parámetros y datos del principal:

```
@PreAuthorize("hasRole('ADMIN') or #usuarioId == authentication.principal.id")
```

Aquí #usuarioId se refiere al parámetro y authentication.principal.id exige que el principal exponga esa propiedad. Por eso en el módulo se prepara UsuarioPrincipal: una expresión correcta puede fallar si el objeto guardado como principal no tiene el dato que la expresión intenta leer.

```
@PreAuthorize("hasRole('ADMIN')")
public void eliminarAlumno(Long id) { ... }
```

pero también puede combinar el rol con parámetros y datos del principal:

```
@PreAuthorize("hasRole('ADMIN') or #usuarioId == authentication.principal.id")
```

T3.3 - @Secured y @PostAuthorize

@Secured ofrece una forma más simple basada principalmente en roles. @PostAuthorize evalúa después de ejecutar el método y puede inspeccionar el resultado, aunque implica que la lógica ya se ha ejecutado.

M6 se centra en @PreAuthorize porque es flexible y evita realizar trabajo que luego será rechazado. Conocer las alternativas ayuda a reconocer código heredado y a elegir la herramienta adecuada.

@Secured ofrece una sintaxis más limitada basada en roles; @PostAuthorize evalúa después de ejecutar y puede consultar el resultado. Son herramientas diferentes, no sinónimos. En una API REST habitual @PreAuthorize cubre la mayoría de necesidades del curso.

@PostAuthorize requiere especial cuidado porque la operación ya se ejecutó. No debe usarse para bloquear efectos secundarios que deberían haberse impedido antes. Su valor está más cerca de filtrar acceso al resultado que de impedir una mutación.

Ejemplo.

```
@PreAuthorize("hasRole('ADMIN')")
public List<UsuarioResponseDTO> listarUsuarios() { ... }
```

Pregunta de reflexión: ¿Cuándo usarías @PostAuthorize en lugar de @PreAuthorize?

Respuesta razonada: @PreAuthorize comprueba la regla antes de ejecutar el método y es la opción habitual cuando basta con identidad, roles y parámetros de entrada. @PostAuthorize se usa cuando la decisión necesita inspeccionar el resultado devuelto, por ejemplo para autorizar sólo si el recurso obtenido pertenece al usuario actual.

Bloque 4 - Acceder al usuario autenticado

T4.1 - Desde el controlador

Un controlador puede recibir Authentication, Principal o @AuthenticationPrincipal. Esto resulta útil para endpoints como “mi perfil”, donde la identidad forma parte directa del contrato HTTP.

Aun así, el controlador no debería contener decisiones de negocio complejas. Lo normal es extraer la identidad y delegar en el servicio, manteniendo el controlador como adaptador web.

En un controlador puede recibirse Authentication para conocer principal y authorities. Es útil cuando la respuesta depende de la identidad actual, por ejemplo /perfil. Sin embargo, el controlador no debería implementar matrices de permisos manuales que ya pertenecen a Spring Security. Al devolver información del usuario autenticado, se construye un DTO seguro. Nunca se serializa el principal completo si contiene hash de contraseña u otros datos internos.

En un controlador hay varias formas de obtener la identidad. Se puede recibir Authentication, que da nombre, principal y authorities; se puede inyectar directamente el principal con @AuthenticationPrincipal; o utilizar la interfaz Java Principal, que ofrece básicamente getName().

```
@GetMapping("/perfil")
Map<String,Object> perfil(Authentication authentication) { ... }

@GetMapping("/perfil")
Map<String,Object> perfil(@AuthenticationPrincipal UserDetails user) { ... }
```

@AuthenticationPrincipal es cómodo cuando sólo interesa el objeto de usuario. Authentication resulta útil cuando además necesitamos las authorities u otros detalles del contexto. Principal es la opción más limitada. Las tres leen la identidad que Spring ya colocó en el contexto; no vuelven a autenticar al cliente.

```
@GetMapping("/perfil")
Map<String,Object> perfil(Authentication authentication) { ... }

@GetMapping("/perfil")
Map<String,Object> perfil(@AuthenticationPrincipal UserDetails user) { ... }
```

T4.2 - Desde el servicio

En la capa de servicio puede consultarse el SecurityContextHolder, aunque conviene encapsular ese acceso para no propagar dependencias de seguridad por todo el dominio. Otra opción es pasar al servicio el identificador del usuario obtenido en la capa web.

La decisión depende del diseño, pero la regla sigue siendo la misma: no confiar en un userId enviado por el cliente para representar la identidad autenticada.

En servicios, la identidad puede consultarse cuando una regla de negocio depende del actor actual, pero conviene hacer esa dependencia explícita. Una alternativa limpia es pasar el id del usuario autenticado desde una frontera controlada o encapsular el acceso al contexto.

El peligro es llenar el dominio de llamadas estáticas a SecurityContextHolder, lo que dificulta tests. La seguridad debe ayudar a expresar reglas, no convertirse en dependencia oculta por todas partes.

En una capa de servicio no existe la misma inyección de parámetros HTTP. Una posibilidad es consultar SecurityContextHolder:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
```

Otra opción es que el controlador extraiga la identidad y la pase al servicio como argumento. Esta segunda forma hace más explícita la dependencia y simplifica muchos tests, porque el servicio no necesita un SecurityContext preparado.

SecurityContextHolder usa por defecto un contexto asociado al hilo de la petición, y Spring Security se encarga de inicializarlo y limpiarlo. Aun así, no conviene convertirlo en una dependencia oculta en todos los métodos de negocio; se reserva para los lugares donde pasar la identidad explícitamente resulta poco práctico.

En una capa de servicio no existe la misma inyección de parámetros HTTP. Una posibilidad es consultar SecurityContextHolder:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
```

T4.3 - Diferencia entre principal y authorities

El principal describe **quién** está autenticado; las authorities describen **qué permisos** posee. Mezclar ambos conceptos conduce a expresiones frágiles, por ejemplo asumir que el username implica un rol.

UsuarioPrincipal permite reunir identificador, username, email y authorities en una representación explícita. Las reglas de autorización leen los permisos; los casos de uso de perfil pueden leer la identidad.

El principal identifica al sujeto; las authorities describen permisos concedidos. Un username por sí solo no demuestra que alguien sea administrador. Del mismo modo, una authority no sustituye la identidad cuando necesitamos saber quién modificó un recurso.

Los logs de auditoría suelen necesitar ambos: actor y operación autorizada. Separar los conceptos ayuda a no confundir “quién” con “qué puede hacer”.

Ejemplo.

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
```

Un Authentication contiene conceptos distintos: principal representa la identidad; credentials son las credenciales utilizadas durante la autenticación y pueden limpiarse después; authorities son los roles/permisos empleados para autorizar; y getName() ofrece un nombre lógico, normalmente derivado del principal.

Esta distinción importa con JWT. El principal puede ser un UsuarioPrincipal con id, username y email, mientras las authorities siguen siendo una colección separada. No debemos meter toda la política dentro del principal ni interpretar el token bruto como si fuera el usuario. Separar identidad y permisos permite que una expresión consulte authentication.principal.id y otra use hasRole('ADMIN') sin mezclar responsabilidades.

Esta distinción importa con JWT. El principal puede ser un UsuarioPrincipal con id, username y email, mientras las authorities siguen siendo una colección separada. No debemos meter toda la política dentro del principal ni interpretar el token bruto como si fuera el usuario. Separar identidad y permisos permite que una expresión consulte authentication.principal.id y otra use hasRole('ADMIN') sin mezclar responsabilidades. > **Pregunta de reflexión:** ¿Qué diferencia hay entre Authentication, Principal y @AuthenticationPrincipal?

Respuesta razonada: Principal expone esencialmente la identidad nominal; Authentication añade authorities, credenciales y estado de autenticación; @AuthenticationPrincipal inyecta directamente el objeto principal almacenado dentro del Authentication, pudiendo ser un UserDetails o nuestro UsuarioPrincipal tipado.

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas con autorización

Las reglas deben ser lo más pequeñas y declarativas posible, con denegación por defecto. Las rutas públicas se enumeran de forma explícita y el resto exige autenticación. Los roles administrativos no se aceptan desde el cliente ni se infieren a partir de parámetros.

También es importante revisar el orden de los matchers: una regla amplia colocada antes puede eclipsar otra más específica.

Una política de autorización debe seguir mínimo privilegio y denegar por defecto. Las excepciones públicas son concretas; la regla final protege el resto. Las operaciones administrativas añaden restricciones explícitas y, cuando tiene sentido, defensa en profundidad por método.

También se evita basar permisos en datos que el cliente controla. El rol procede del principal autenticado, no de una cabecera X-Role ni de un campo JSON enviado en cada petición.

La autorización debe expresarse cerca de la frontera que mejor conoce la regla. Los patrones URL son adecuados para áreas generales; @PreAuthorize para condiciones de negocio o dependientes de parámetros. Combinar ambas capas evita que un refactor de rutas deje una operación sensible completamente desprotegida.

También conviene mantener nombres de roles consistentes, evitar strings dispersos cuando el dominio crece y probar siempre 401, 403 y acceso permitido. Las reglas no deben depender de datos que el principal actual no expone. Si una expresión SpEL necesita id, primero hay que garantizar un principal con getId(). Y como la seguridad por método depende de proxies, hay que vigilar métodos privados y llamadas internas que no atraviesan la interceptación de Spring.

T5.2 - Errores comunes

Los errores típicos son confundir ROLE_ADMIN con ADMIN, olvidar @EnableMethodSecurity, usar patrones de URL demasiado amplios, confiar en datos del request para decidir identidad o dejar una ruta sensible bajo permitAll.

Otro fallo frecuente es probar sólo con el usuario autorizado. Un test negativo revela rápidamente si una regla realmente protege el recurso.

Entre los errores comunes están confundir 401/403, usar `hasAuthority("ADMIN")` cuando existe `ROLE_ADMIN`, olvidar `@EnableMethodSecurity`, ordenar mal matchers o abrir un wildcard demasiado amplio. Cada error debe reproducirse con un test o una petición concreta.

La corrección no es añadir `permitAll()` hasta que desaparezca el 403. La corrección es observar principal, authorities y regla aplicada, y modificar sólo la causa identificada.

También deben revisarse reglas aparentemente redundantes. Dos matchers que protegen la misma ruta con condiciones diferentes pueden depender del orden y crear una política que nadie entiende. La solución no es acumular otra excepción, sino simplificar la matriz y dejar una única intención verificable por ruta/método.

Estos fallos se diagnostican mejor por su causa concreta. Si `@PreAuthorize` parece ignorarse, hay que comprobar primero que `@EnableMethodSecurity` está activo. Si `hasRole("ROLE_ADMIN")` no funciona, el problema es el doble prefijo: `hasRole` ya busca `ROLE_ADMIN`, por lo que se escribe `hasRole("ADMIN")`. A la inversa, `hasAuthority("ADMIN")` no coincide con una authority llamada `ROLE_ADMIN`; en ese caso se usa el nombre completo o se expresa la regla como rol.

También hay errores ligados al lugar donde se aplica la seguridad. Una expresión SpEL que accede a `authentication.principal.id` sólo puede funcionar si el principal expone realmente esa propiedad. Una anotación colocada en un método privado o una llamada interna que no atraviesa el proxy de Spring puede no ser interceptada como se espera. Por eso no basta con que el código compile: hay que probar de forma separada el acceso anónimo, el usuario autenticado sin permiso y el usuario autorizado, y revisar que una ruta nueva no quede accidentalmente fuera de la política.

T5.3 - Testing de autorización

Una matriz de tests pequeña pero sistemática funciona mejor que casos aislados: anónimo, USER, GESTOR y ADMIN frente a operaciones públicas, autenticadas y administrativas. `@WithMockUser` permite expresar esta matriz con claridad.

Para reglas que dependen de un principal real, los tests de integración complementan los mocks y verifican que los roles persistidos llegan correctamente al `SecurityContext`.

Ejemplo.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/v1/public/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public void cambiarRoles(Long usuarioId, Set<String> roles) { ... }
```

Los tests de autorización forman una matriz de sujetos y resultados: anónimo, USER, GESTOR, ADMIN frente a cada operación sensible. Una matriz hace visibles huecos que un único “happy path” no detecta.

Con `@WithMockUser` se prueban reglas aisladas; con integración JWT se comprueba además que los roles reales llegan al contexto. Ambos niveles son complementarios.

Ejemplo.

ANÓNIMO -> 401

USER -> 403

ADMIN -> 200

Los tres resultados básicos deben quedar separados en tests, no inferirse a partir de un único caso de éxito:

```
@Test
```

```
@WithMockUser(roles = "USER")
```

```
void listarUsuarios_debeDevolver403_cuandoNoEsAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isForbidden());
}
```

```
@Test
```

```
@WithMockUser(roles = "ADMIN")
```

```
void listarUsuarios_debeDevolver200_cuandoEsAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk());
}
```

```
@Test
void listarUsuarios_debeDevolver401_cuandoNoAutenticado() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isUnauthorized());
}
```

Pregunta de reflexión: ¿Por qué es importante testear los tres escenarios y no solo el de éxito?

Respuesta razonada: Porque 200, 401 y 403 validan contratos distintos. Probar sólo el éxito no detectaría una ruta accidentalmente pública ni una autorización demasiado permisiva; los casos negativos son precisamente los que demuestran que la barrera de seguridad existe.

Resumen de la teoría

- La autorización por URL protege áreas completas antes de llegar al controlador.
- La autorización por método expresa reglas próximas a la operación y puede usar SpEL.
- `hasRole` aplica la convención `ROLE_`; `hasAuthority` compara la `authority` exacta.
- `@AuthenticationPrincipal` permite acceder al principal autenticado sin consultar manualmente el contexto.
- Combinar URL y método aporta defensa en profundidad y debe verificarse con tests 401/403/200.

Punto 6.5 - Introducción a JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar qué es un JSON Web Token (JWT) y qué problema resuelve.
2. Diferenciar autenticación con sesión (stateful) de autenticación con token (stateless).
3. Describir la estructura de un JWT: header, payload y signature.
4. Identificar los claims registrados, públicos y privados de un JWT.
5. Explicar cómo funciona la firma y por qué garantiza la integridad del token.
6. Reconocer los casos de uso de JWT y cuándo no usarlo.
7. Diagnosticar y resolver los errores más comunes al trabajar con JWT.

Bloque 1 - Qué es un JWT

T1.1 - Definición y origen

JWT significa JSON Web Token y define una representación compacta de claims que puede firmarse digitalmente. Su forma habitual es una cadena de tres segmentos codificados en Base64URL y separados por puntos.

El token no es una sesión escondida ni un formato de cifrado por defecto. Es un contenedor firmado cuya utilidad depende de qué claims contiene, cómo se protege la clave y cómo se valida en cada petición.

JWT significa JSON Web Token y está estandarizado para representar claims en una estructura compacta que puede firmarse. El token no es “una sesión comprimida”: es un mensaje que el receptor puede verificar criptográficamente y del que extrae afirmaciones.

La firma proporciona integridad y autenticidad respecto de la clave utilizada, no confidencialidad. Cualquiera que posea el token puede decodificar header y payload si no existe cifrado adicional.

El estándar RFC 7519 define el JWT como una forma compacta de transportar **claims**, es decir, afirmaciones sobre un sujeto. El token puede viajar en una cabecera HTTP y está pensado para intercambios donde interesa que el receptor pueda comprobar que esas afirmaciones no se han modificado. Un JWT puede estar firmado y, en otras variantes del ecosistema JOSE, incluso cifrado, pero el caso que estudiamos aquí es el **JWS firmado** que se usa habitualmente para autenticación de APIs.

La palabra *autocontenido* significa que el token puede incluir los datos mínimos que el receptor necesita para tomar una decisión inmediata: sujeto, instante de emisión, expiración y, si el diseño lo justifica, roles. No significa que deba copiarse dentro del JWT todo el registro del usuario. Cuantos

más datos se duplican, más difícil es mantenerlos sincronizados y mayor es la información expuesta si el token se inspecciona.

T1.2 - El problema que resuelve

En una API distribuida resulta útil transportar una prueba verificable de autenticación sin consultar una sesión de servidor en cada llamada. El emisor genera un token y los componentes que conocen la clave o la clave pública pueden verificarlo.

Esto reduce dependencia de estado de sesión, pero traslada responsabilidades al diseño del token: expiración corta, gestión de claves, refresh y revocación cuando sea necesaria.

En una API distribuida, un token firmado permite que cada petición aporte evidencia de autenticación sin depender de una sesión de servidor. El servidor verifica firma y claims y reconstruye el contexto. Esto facilita escalado horizontal, pero desplaza responsabilidades al diseño del token.

La contrapartida es que un token emitido no desaparece por arte de magia. Expiración, rotación y revocación deben diseñarse explícitamente, sobre todo para logout y credenciales comprometidas.

El beneficio stateless aparece especialmente cuando varias instancias atienden peticiones: ninguna necesita localizar una sesión concreta para reconocer al cliente. Pero esa ventaja no elimina coste; la validación criptográfica se repite y la revocación inmediata se vuelve menos trivial. Por eso JWT resuelve un problema de distribución a cambio de trasladar complejidad a expiración, refresh y gestión de claves. El diseño debe valorar ambos lados y no presentar el token como una simplificación gratuita.

El contraste con una sesión tradicional permite ver el problema con precisión. En un esquema *stateful*, el usuario envía sus credenciales, el servidor las valida, crea una sesión y conserva en memoria o en un almacén compartido el estado asociado. El cliente recibe un identificador de sesión y lo reenvía en las peticiones posteriores; el servidor debe localizar ese identificador y recuperar el estado antes de saber quién está llamando.

Cuando una aplicación crece a varias instancias, ese estado obliga a decidir dónde viven las sesiones: afinidad de sesión, replicación o un almacén central. JWT propone otra opción: tras autenticar al usuario, el servidor emite una credencial firmada y el cliente la presenta en cada petición. Cualquier instancia que pueda verificar la firma puede reconstruir la identidad y los claims necesarios sin recuperar una sesión HTTP previa. Esa es la necesidad que JWT resuelve; no elimina la autenticación inicial ni convierte el sistema en “sin estado” en todos los sentidos, porque refresh, revocación o listas negras pueden introducir estado deliberadamente.

T1.3 - Stateless vs stateful

En un sistema *stateful* el servidor asocia al cliente con una sesión almacenada. En el enfoque *stateless* de M6 cada petición aporta la credencial necesaria y `SessionCreationPolicy.STATELESS` impide depender de `HttpSession` para conservar autenticación.

“Stateless” no significa “sin ningún estado en todo el sistema”. Por ejemplo, una lista de refresh tokens usados o tokens revocados introduce un estado controlado para resolver logout y rotación.

Stateful significa que el servidor conserva estado de sesión asociado a un identificador; stateless significa que cada petición aporta lo necesario para autenticarse. `SessionCreationPolicy.STATELESS` evita usar la sesión HTTP como mecanismo de persistencia del `SecurityContext`.

Stateless no significa “sin estado en todo el sistema”. La base de datos, la lista educativa de revocación y el estado de refresh siguen existiendo. Significa que la autenticación de cada petición no depende de una sesión HTTP previa.

Ejemplo.

`header.payload.signature`

Pregunta de reflexión: ¿Qué propiedad de una sesión estatal deja de necesitar el servidor cuando usa JWT stateless, y qué responsabilidad nueva aparece en el token?

Respuesta razonada: En una sesión *stateful* el servidor conserva estado de sesión entre peticiones. Con JWT *stateless* la información necesaria viaja firmada en cada petición, lo que reduce estado central pero obliga a gestionar con cuidado expiración, revocación y protección del token.

Bloque 2 - Estructura de un JWT

T2.1 - Tres partes separadas por puntos

Un JWT firmado suele tener `header.payload.signature`. Header y payload son JSON codificados en Base64URL; la firma se calcula sobre esos dos segmentos. Separar visualmente las partes ayuda a entender que sólo la tercera aporta autenticidad.

Decodificar los dos primeros segmentos es trivial y no demuestra que el token sea válido. Cualquier cliente puede leerlos o construir otros distintos.

Un JWT compacto tiene tres segmentos Base64URL separados por puntos: header, payload y firma. La separación permite inspeccionar metadatos y claims sin tener la clave, pero sólo la verificación de la firma permite confiar en que no fueron alterados.

Contar los tres segmentos es un diagnóstico básico, no una validación. Un atacante puede construir tres segmentos perfectamente formados. El receptor debe verificar criptografía y claims.

La forma compacta se obtiene concatenando tres cadenas:

`BASE64URL(header).BASE64URL(payload).BASE64URL(signature)`

Los dos puntos que separan los segmentos son parte esencial del formato. Header y payload se serializan como JSON antes de codificarse. La firma no es JSON: son bytes producidos por el algoritmo criptográfico y después representados también en Base64URL. Base64URL usa un alfabeto seguro para URL y normalmente elimina el padding =; por eso un JWT puede viajar cómodamente en HTTP sin escapar caracteres especiales.

Separar las tres partes también ayuda a diagnosticar errores: un token con uno o dos segmentos está mal formado antes incluso de verificar la firma; uno con tres segmentos puede seguir siendo completamente falso. La estructura sólo demuestra que el mensaje *parece* un JWT, no que sea confiable.

Un token real tiene este aspecto; los saltos son sólo para poder leerlo en el material:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkJFuYSBHYXJw61hIiwiaW

↪ F0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMe

KKF2QT4fwpMeJf36P0k6yJV adQssw5c

T2.2 - El header

El header describe metadatos del token, principalmente el tipo y el algoritmo de firma. En un token HMAC puede indicar HS256. El validador no debe aceptar algoritmos arbitrarios enviados por un atacante; la política criptográfica pertenece al servidor.

El header es pequeño, pero forma parte exacta de los datos firmados. Cambiarlo invalida la firma.

El header suele declarar el tipo y el algoritmo, por ejemplo `typ=JWT` y `alg=HS256`. No se debe confiar ciegamente en cualquier algoritmo indicado por el propio token; la librería y la configuración del servidor deben imponer los algoritmos esperados.

El header es visible. Nunca se colocan secretos allí. Su función es describir cómo interpretar/verificar el mensaje, no transportar credenciales privadas.

Aunque el header declare HS256, la confianza no puede depender de esa declaración no verificada. El servidor ya sabe qué tipo de claves y algoritmo admite. Esta idea evita la clase de errores donde el atacante intenta seleccionar un algoritmo más débil mediante datos controlados por él mismo.

Un header típico puede representarse así:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

alg indica el algoritmo empleado para la firma y typ describe el tipo de objeto. En sistemas con varias claves puede aparecer además kid (*key id*) para identificar qué clave debe usar el verificador. Ninguno de estos campos es secreto y todos quedan cubiertos por la firma junto con el payload.

La aplicación no debe permitir que el atacante elija libremente la política criptográfica. Aunque el token incluya alg, el servidor debe validar con las claves y algoritmos que su configuración considera aceptables. El header ayuda a interpretar el mensaje, pero la confianza siempre procede de la verificación con una política controlada por el receptor.

T2.3 - El payload

El payload contiene claims. Algunos son registrados, como sub, iat y exp; otros son propios de la aplicación, por ejemplo roles. Los claims no deben incluir secretos, porque el payload se puede decodificar sin conocer la clave.

En M6 sub identifica al usuario y la expiración limita la vida del access token. Las authorities se reconstruyen de forma controlada a partir de la identidad del sistema.

El payload contiene claims como sub, iat y exp, además de claims propios como roles. Base64URL sólo representa bytes en texto seguro para URL; no cifra el contenido. Por eso no debemos guardar contraseñas, secretos ni datos personales innecesarios.

La selección de claims sigue el principio de minimización. Si una información puede cambiar frecuentemente o no es necesaria para autorizar la petición, quizá sea mejor consultarla en servidor.

Ejemplo.

```
{"sub":"ana","roles":["USER"],"iat":...,"exp":...}
```

Pregunta de reflexión: ¿Qué partes de un JWT puede leer cualquiera y qué parte aporta la evidencia criptográfica de integridad?

Respuesta razonada: Header y payload pueden decodificarse sin secreto; la firma es la parte que permite comprobar que el contenido no fue alterado por alguien que desconoce la clave privada o secreta correspondiente.

Bloque 3 - La firma

T3.1 - Cómo se calcula la firma

Con HMAC, el emisor combina el header y payload codificados, calcula un MAC con una clave secreta y codifica el resultado en Base64URL. El receptor repite el cálculo con la misma clave y compara el resultado de manera segura.

Si cambia un solo byte del contenido, el cálculo ya no coincide. Por eso modificar manualmente el payload produce un token que se puede leer pero no aceptar.

La firma se calcula sobre la representación codificada de header y payload usando una clave y un algoritmo. Si se cambia un solo byte del payload sin recalcular con la clave correcta, la verificación falla.

El experimento del módulo —modificar el payload— es importante porque separa “puedo leer el token” de “puedo producir un token aceptado”. Decodificar es una operación de formato; verificar es una operación de seguridad.

Para HS256, la entrada de la firma es exactamente la concatenación de las dos primeras partes:

```
base64Url(header) + "." + base64Url(payload)
```

Sobre esa cadena se calcula HMAC-SHA256 con el secreto compartido. El resultado binario se codifica de nuevo en Base64URL y forma el tercer segmento. El receptor repite el cálculo usando **la misma clave** y compara el resultado con la firma recibida. Si no coinciden, el token debe rechazarse antes de confiar en cualquier claim.

Este detalle explica por qué no se debe parsear el payload, leer roles=ADMIN y decidir permisos antes de verificar. El payload está controlado por quien presente el token hasta que la firma ha sido validada. Primero se verifica la autenticidad del conjunto; sólo después se interpretan subject, expiración y permisos.

T3.2 - Firma simétrica vs asimétrica

HS256 usa una clave simétrica: quien firma y quien verifica conocen el mismo secreto. Algoritmos como RS256 usan una clave privada para firmar y una pública para verificar.

La elección depende de la arquitectura. Para una aplicación docente monolítica, HMAC simplifica el ejemplo. En sistemas con muchos verificadores independientes, la criptografía asimétrica puede facilitar la distribución de claves.

Con algoritmos simétricos como HS256, emisor y verificador comparten la misma clave secreta. Con algoritmos asimétricos, la clave privada firma y la pública verifica. La elección afecta distribución de claves y arquitectura.

El proyecto utiliza HS256 por claridad y porque emisor/verificador viven en la misma aplicación. En

un ecosistema con múltiples servicios verificadores, una firma asimétrica puede reducir la necesidad de distribuir el secreto de firma.

En una firma simétrica como HS256, la misma clave sirve para emitir y validar. Esto simplifica una aplicación única, pero cualquier sistema que necesite verificar también recibe capacidad potencial de firmar, porque conoce el secreto. La protección y distribución de esa clave son por tanto críticas.

En un esquema asimétrico como RSA o EC, el emisor conserva una **clave privada** y los consumidores verifican con una **clave pública**. Los verificadores pueden comprobar tokens sin adquirir capacidad de emitirlos. Esta separación resulta útil cuando un servicio central de identidad firma y muchos servicios independientes validan. A cambio, la gestión de pares de claves y rotación es más compleja. El curso usa HS256 para centrarse en el flujo JWT, no porque sea la única opción correcta.

T3.3 - Por qué la firma garantiza la integridad

La firma vincula criptográficamente el contenido con una clave. Si un atacante cambia roles, subject o expiración sin poder recalcular una firma válida, la verificación falla.

La firma no oculta la información. Integridad y confidencialidad son propiedades distintas. Si un claim no debe ser visible al cliente, no debe colocarse en un JWT firmado sin cifrado adicional.

La firma garantiza que header y payload no cambiaron desde que una parte con la clave válida produjo el token. No demuestra que todos los claims sean “verdad eterna”: el servidor aún debe comprobar expiración, tipo de token, sujeto y cualquier condición de negocio.

Un token con firma válida pero expirado debe rechazarse. Un refresh token firmado no debe aceptarse como access token si el diseño distingue ambos tipos.

Ejemplo.

```
firma = HMACSHA256(base64url(header) + "." + base64url(payload), clave)
```

Pregunta de reflexión: ¿Qué diferencia hay entre firmar y cifrar un JWT? ¿Qué garantiza cada uno?

Respuesta razonada: Firmar aporta integridad y autenticidad: permite detectar modificaciones y verificar quién emitió el token según el algoritmo y la clave. Cifrar aporta confidencialidad: impide leer el contenido sin la clave de descifrado. Un JWT firmado normal no oculta su payload.

Bloque 4 - Cuándo usar JWT

T4.1 - Casos de uso ideales

JWT encaja bien cuando una API necesita credenciales portables, vida limitada y validación local. Es habitual en APIs REST, arquitecturas distribuidas y comunicación entre componentes que comparten una política de confianza.

La ganancia principal es desacoplar la validación de una sesión central. A cambio hay que gestionar expiraciones y renovación con cuidado.

JWT encaja bien en APIs donde clientes envían Bearer tokens, se desea autenticación stateless y varios nodos deben verificar las mismas credenciales sin compartir sesión HTTP. También puede servir como formato de claims entre servicios con una política de claves sólida.

La ventaja operativa no elimina la necesidad de HTTPS. Un Bearer robado puede usarse mientras sea válido; TLS protege el token durante el transporte.

JWT encaja especialmente bien cuando el cliente llama a una API REST y cada petición debe llevar su propia credencial, cuando varias instancias de servidor atienden tráfico sin compartir una sesión HTTP concreta o cuando distintos servicios necesitan comprobar una identidad emitida por un componente de autenticación. También es habitual en aplicaciones móviles y SPA, siempre acompañado de HTTPS y una estrategia segura de almacenamiento del token.

La ventaja importante no es “ser moderno”, sino que el receptor puede validar el mensaje de forma local con la clave adecuada. En arquitecturas distribuidas esto reduce consultas a un almacén de sesiones central. Aun así, los datos de negocio siguen viviendo en base de datos y las decisiones que dependan de información mutable pueden necesitar una consulta adicional.

T4.2 - Cuándo no usar JWT

No todo sistema necesita JWT. Una aplicación web monolítica con sesión de servidor puede ser más simple y más fácil de revocar. Tampoco conviene usar JWT como almacén de datos de usuario ni crear tokens de vida muy larga para evitar implementar refresh.

La tecnología debe resolver un problema real. En M6 se usa porque queremos aprender una API REST stateless, no porque sea obligatoria en Spring Boot.

No todo necesita JWT. Una aplicación web monolítica con sesión segura y cookies HttpOnly puede ser más simple y ofrecer revocación inmediata. Elegir JWT sólo por moda añade expiración, refresh, almacenamiento de cliente y revocación que hay que implementar bien.

La pregunta correcta es qué propiedades necesita el sistema: distribución, tipos de cliente, control de sesión, SSO y modelo de amenazas. La tecnología viene después.

JWT no es automáticamente mejor que una sesión tradicional. En una aplicación web monolítica donde servidor y navegador comparten una sesión, invalidar una sesión central puede ser más sencillo que revocar tokens ya emitidos. Tampoco es una buena elección para transportar grandes cantidades de datos, información secreta o atributos que cambian constantemente.

Un token largo viaja en cada petición y aumenta el coste de red. Además, si los permisos cambian mientras un access token sigue vivo, el contenido firmado puede quedar desactualizado hasta la expiración. Por eso el diseño debe decidir qué claims merece la pena incluir y cuánto debe durar el token. La regla es elegir JWT cuando sus propiedades resuelven una necesidad real, no sólo para evitar usar sesiones.

T4.3 - El ciclo de vida de un token

El access token debe tener vida relativamente corta. Cuando expira, el cliente utiliza un refresh token válido para obtener un nuevo par. El refresh token tiene una vida mayor y requiere controles adicionales porque su compromiso permite renovar credenciales.

Nuestro diseño rota el refresh token y considera de un solo uso los tokens registrados. El logout revoca los tokens que el servidor decide recordar.

El ciclo típico es: autenticar credenciales, emitir access token corto y refresh token más duradero, usar access en recursos, renovar mediante refresh y revocar/rotar cuando corresponde. Cada transición tiene un endpoint y un resultado comprobable.

El access token debe caducar aunque el usuario no cierre sesión. El refresh no se reenvía a todos los recursos; se reserva para obtener nuevos tokens, reduciendo su exposición.

Ejemplo.

login -> access corto + refresh largo -> API -> refresh -> nuevo par

Pregunta de reflexión: ¿Por qué el token de acceso tiene una duración corta? ¿Qué pasa si el token se filtra?

Respuesta razonada: Porque un access token robado puede reutilizarse mientras siga siendo válido. Una duración corta limita esa ventana de abuso; el refresh token permite renovar la sesión sin obligar al usuario a introducir credenciales continuamente, con controles de rotación y revocación.

Bloque 5 - Errores y buenas prácticas

T5.1 - Errores comunes con JWT

Los fallos más graves son aceptar un token sólo porque se puede decodificar, utilizar una clave demasiado corta, no verificar expiración, incluir datos sensibles, reutilizar refresh tokens indefinidamente o confundir un refresh token con un access token.

También es peligroso confiar ciegamente en claims de roles si el sistema no define cómo se sincronizan con los permisos actuales.

Errores frecuentes: tratar Base64 como cifrado, guardar secretos en el payload, usar claves débiles, aceptar tokens expirados, no distinguir access/refresh o registrar tokens completos. También es peligroso confiar en claims sin verificar primero la firma.

Cada error tiene una prueba negativa asociada. La seguridad mejora cuando los caminos de rechazo son tan explícitos como el camino feliz.

Entre los errores más frecuentes están confundir **codificación** con cifrado, aceptar tokens sin verificar la firma, usar una clave demasiado corta, no validar exp, confiar en un algoritmo no permitido, incluir contraseñas o secretos en el payload y crear tokens con una duración excesiva. Otro fallo común es interpretar cualquier cadena que tenga tres segmentos como si ya fuera una credencial válida.

También hay errores operativos: registrar tokens completos en logs, enviarlos por HTTP sin TLS, copiar tokens reales a herramientas externas para depurar o almacenar access tokens en lugares accesibles a scripts no confiables. El problema no termina en la criptografía; un JWT válido robado puede reutilizarse mientras siga siendo aceptado. El diseño seguro combina firma, transporte HTTPS, expiración corta y controles de ciclo de vida.

T5.2 - Buenas prácticas con JWT

Las claves deben estar externalizadas, ser suficientemente largas y rotarse con un procedimiento definido. Los access tokens deben caducar, la verificación debe fijar el algoritmo esperado y los errores no deben revelar secretos.

En producción la clave no se incluye con fallback inseguro en properties. M6 exige `jwt.secret=${JWT_SECRET}` para el perfil de producción.

Buenas prácticas: clave suficientemente aleatoria, configuración fuera del código, access tokens de vida limitada, validación de exp, algoritmo fijado, mínimo de claims y tratamiento uniforme de errores. Los tokens no deben aparecer en repositorios ni logs.

En producción el secreto llega mediante `JWT_SECRET`. Una variable de entorno contiene configuración/secreto del servidor; no es el lugar donde “se guarda el access token del usuario”. Esa distinción corrige una confusión habitual.

Las buenas prácticas básicas son: access tokens de vida corta; refresh tokens con mayor protección y rotación; secreto o clave fuera del repositorio; validación de firma, algoritmo, tipo y expiración; claims mínimos; HTTPS obligatorio; y mensajes de error que no revelen detalles internos. Cuando se necesita logout inmediato, hay que diseñar revocación o reducir el tiempo de exposición del access token.

La clave debe poder rotarse. En producción no se incrusta como literal Java ni se publica en `application.properties`; se inyecta desde un sistema de configuración seguro. También conviene usar identificadores como `jti` cuando el ciclo de vida necesita distinguir tokens individuales. El objetivo es que la seguridad no dependa de una sola barrera, sino de varias decisiones coherentes.

T5.3 - Alternativas a JWT

Las sesiones de servidor siguen siendo una opción válida. También existen tokens opacos, donde el servidor de recursos consulta o introspecciona el token, y soluciones estándar OAuth2/OpenID Connect que delegan identidad y autorización en componentes especializados.

Conocer estas alternativas ayuda a no convertir JWT en una solución universal. Nuestro objetivo es comprender el mecanismo y sus límites.

Ejemplo.

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbmEiLCJleHAiOjE3MDAwMDAwMDB9.<firma>
```

Los dos primeros segmentos se pueden decodificar. Sólo una verificación criptográfica de la firma y de claims como exp permite confiar en el token.

Alternativas incluyen sesiones de servidor, tokens opacos y protocolos de identidad como OAuth2/OIDC. Un token opaco obliga a consultar al emisor o a una store, pero facilita revocación central. Una sesión tradicional puede simplificar clientes web del mismo dominio.

Conocer alternativas permite evaluar trade-offs. El módulo enseña JWT en profundidad, no afirma que sea universalmente superior.

OAuth2/OIDC merece una distinción adicional: no es simplemente “otra clase de token”, sino un conjunto de protocolos para delegar autorización e identidad entre actores. Un proveedor corporativo puede emitir JWT, pero la aplicación ya no gestiona directamente contraseñas de usuario. En muchos sistemas empresariales esa delegación es preferible. El módulo implementa JWT propio porque el objetivo didáctico es comprender firma, claims, filtro y ciclo de vida; no porque sea la única arquitectura válida.

Ejemplo.

Decodificar != verificar

Firma válida != token vigente
JWT != cifrado

Pregunta de reflexión: ¿Qué ventaja tienen los tokens opacos frente a JWT? ¿Qué inconveniente?

Respuesta razonada: Un token opaco no expone claims al cliente y obliga a resolver su estado en el servidor, lo que facilita revocación central e inmediata. A cambio, requiere una consulta o introspección para conocer la identidad y permisos, perdiendo parte de la autonomía y escalabilidad que puede aportar un JWT autocontenido.

Resumen de la teoría

- JWT es un formato de token autocontenido formado por header, payload y signature.
- Header y payload están codificados en Base64Url, no cifrados: cualquiera que obtenga el token puede leerlos.
- La firma protege la integridad; modificar el contenido sin recalcularla invalida el token.
- Los claims describen identidad, tiempos y datos de aplicación, pero no deben contener secretos.
- JWT encaja bien en APIs stateless, aunque no es la mejor opción cuando se necesita revocación inmediata o sesión centralizada.

Punto 6.6 - Generación de tokens JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Añadir la librería JJWT al proyecto y entender sus módulos.
2. Configurar la clave secreta de firma de forma segura.
3. Generar un token JWT con claims personalizados.
4. Configurar la expiración de un token.
5. Implementar un endpoint de login que devuelva un token.
6. Implementar refresh tokens para renovar el token de acceso.
7. Diagnosticar y resolver los errores más comunes al generar tokens.

Bloque 1 - La librería JJWT

T1.1 - Qué es JJWT y por qué usarla

JJWT es una biblioteca Java para crear, firmar, analizar y validar JWT. Evita implementar a mano detalles delicados de Base64URL, serialización de claims y verificación de firmas.

M6 utiliza JJWT 0.13.0. La versión importa porque la API ha evolucionado y muchos ejemplos antiguos de Internet usan métodos obsoletos. El código del curso adopta la API actual.

JJWT evita implementar a mano serialización, firma, parsing y validaciones criptográficas. La librería ofrece builders y parsers que expresan intención y reducen errores de manipulación de Base64 o MAC.

Usamos JJWT 0.13.0 porque el proyecto está en Java 17/Spring Boot 3.5.x y queremos una API moderna. Los ejemplos antiguos de internet pueden corresponder a versiones con métodos distintos; siempre se adapta a la versión declarada en el POM.

JJWT evita reimplementar operaciones delicadas de JOSE/JWT: serialización de claims, codificación Base64URL, selección de algoritmos, firma, parseo y validación de expiración. La práctica manual de 6.5 sirve para entender qué ocurre, pero una aplicación real debe apoyarse en una biblioteca mantenida y probada.

Con JJWT el código expresa intención: `Jwts.builder()` construye el token y `Jwts.parser()` lo verifica. Esto reduce la superficie de errores de implementación, pero no sustituye las decisiones de diseño. La librería no decide por nosotros cuánto dura un token, qué claims contiene, dónde se guarda el secreto ni cómo se revoca. Esas decisiones siguen perteneciendo a la aplicación.

T1.2 - Los módulos de JWT

JWT se distribuye en varios artefactos: una API pública y módulos de implementación/JSON que se usan en runtime. Maven debe declarar correctamente las dependencias para compilar contra la API y disponer de la implementación al ejecutar.

Separar los módulos evita acoplar el código de aplicación a clases internas y permite a la biblioteca elegir el soporte JSON adecuado.

JWT separa `jjwt-api` de implementaciones runtime (`jjwt-impl`) y soporte Jackson (`jjwt-jackson`). El código compila contra la API y las piezas runtime realizan el trabajo concreto de parsing/JSON.

Declarar scopes correctos evita acoplar el código a clases internas. Si sólo se añade `jjwt-api`, la compilación puede pasar y el runtime fallar por falta de implementación; por eso la práctica verifica ejecución además de Maven.

JWT se distribuye en varios artefactos porque separa la API pública de la implementación y de la integración JSON. En Maven usamos `jjwt-api` como dependencia de compilación y `jjwt-impl` más `jjwt-jackson` en runtime. Nuestro código compila contra interfaces y clases públicas, mientras la implementación concreta y el serializador se cargan al ejecutar.

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
```

Los módulos runtime deben permanecer presentes en el artefacto ejecutable. Si se añade sólo `jjwt-api`, el IDE puede dejar compilar y el fallo aparecer más tarde al construir o parsear un token. Por eso la práctica comprueba también el arranque, no únicamente compile.

T1.3 - La API de JWT

La creación se hace con `Jwts.builder()` y el análisis con un parser configurado con la clave de verificación. El estado final utiliza `Jwts.SIG.HS256` de forma explícita y `signWith(clave, Jwts.SIG.HS256)`.

Esta explicitud elimina ambigüedades y hace visible la política criptográfica del proyecto.

La API moderna construye tokens con `Jwts.builder()` y los firma con una `SecretKey`, por ejemplo `.signWith(clave, Jwts.SIG.HS256)`. Para leer, se construye un parser configurado con la clave de verificación y se analizan claims firmados.

No debemos desactivar validaciones para “hacer que pase” un token. Excepciones de firma o expiración son señales de seguridad y se traducen a una respuesta controlada.

Ejemplo.

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
```

Pregunta de reflexión: ¿Por qué `jjwt-api` no tiene `scope=runtime` y `jjwt-impl` sí?

Respuesta razonada: Nuestro código compila contra las clases públicas de `jjwt-api`, por eso esa dependencia debe estar disponible en compilación. `jjwt-impl` aporta la implementación interna que JWT necesita al ejecutar, pero la aplicación no debería importar directamente sus clases; por eso puede quedar en runtime.

Bloque 2 - La clave secreta

T2.1 - Cómo se representa una clave en JWT

Para HMAC JWT trabaja con un objeto `SecretKey`, no con un `String` utilizado directamente como contraseña. La configuración decodifica un secreto Base64 y construye una clave adecuada para HS256.

Representar la clave como bytes y luego como `SecretKey` evita errores de encoding y permite validar su tamaño antes de usarla.

La clave no es una `String` arbitraria usada directamente. Se decodifica/deriva a bytes con suficiente entropía y se convierte en `SecretKey` compatible con HMAC. JWT comprueba longitudes

mínimas para evitar claves criptográficamente débiles.

El formato de configuración debe ser inequívoco: si el secreto se guarda en Base64, el código lo decodifica como Base64; no se mezclan “texto literal” y “Base64” según el entorno.

Para HS256, una clave demasiado corta reduce seguridad y las bibliotecas modernas pueden rechazarla. Por eso ejemplos como `mi-secreto` no deben trasladarse a producción. El curso usa una representación suficientemente larga en perfiles controlados y exige `JWT_SECRET` fuera del repositorio para producción, manteniendo idéntica lógica de construcción.

JJWT trabaja con objetos de tipo `SecretKey` para HMAC. Esto evita tratar la clave como un `String` arbitrario en todas las capas. La representación textual se limita a la configuración; después se decodifica una sola vez y el resto del código recibe un objeto criptográfico preparado.

Para HS256 la clave debe tener al menos 256 bits efectivos. Una frase corta no se vuelve segura simplemente porque “parezca larga”. En configuración podemos almacenar material aleatorio codificado en Base64 y reconstruirlo mediante `Decoders.BASE64.decode(...)` y `Keys.hmacShaKeyFor(...)`. Esa secuencia convierte texto de configuración en bytes y finalmente en la clave que entiende JJWT.

T2.2 - Generar una clave segura

Una clave HMAC debe contener suficiente entropía. No sirven palabras legibles, nombres de proyecto ni secretos cortos. Para desarrollo puede fijarse un valor de prueba largo; en producción debe proceder de una variable de entorno o gestor de secretos.

La clave no se registra en logs ni se devuelve en respuestas. Su compromiso permitiría fabricar tokens válidos.

Una clave segura se genera con aleatoriedad criptográfica y longitud adecuada, no con frases memorables ni ejemplos del curso. Para desarrollo pueden existir valores controlados; producción debe inyectar un secreto independiente y no versionado.

La rotación de claves es un problema real que un sistema avanzado resolvería con identificadores `kid` y coexistencia temporal. El curso no implementa toda esa infraestructura, pero no debe presentar una clave fija en código como práctica válida.

Una clave segura se genera con un generador criptográfico o con las utilidades de la propia biblioteca, no escribiendo una frase memorable. Debe poseer entropía suficiente para el algoritmo y tratarse como un secreto de infraestructura. Para desarrollo puede existir un valor local controlado; para producción se suministra mediante variable de entorno o gestor de secretos.

La rotación debe contemplarse desde el diseño. Cambiar una clave invalida tokens firmados con la anterior salvo que se mantenga temporalmente una estrategia de verificación por versiones. Aunque el curso usa una clave única para mantener el ejemplo comprensible, la lección importante es que la clave tiene ciclo de vida propio: generación, almacenamiento, acceso restringido, rotación y retirada.

La clave puede generarse con `SecureRandom` y después externalizarse en Base64:

```
byte[] clave = new byte[32];
new SecureRandom().nextBytes(clave);
String claveBase64 = Base64.getEncoder().encodeToString(clave);
export JWT_SECRET=tuClaveGeneradaEnBase64
jwt.secret=${JWT_SECRET}
jwt.expiration=3600000
jwt.refresh-expiration=604800000
```

T2.3 - Construir la clave en el código

`JwtConfig` concentra la lectura de propiedades y la construcción de `SecretKey`. Así `JwtService` recibe una dependencia ya válida y no repite lógica de configuración.

Esta separación también facilita tests: podemos inyectar una clave controlada para verificar generación y expiración sin depender del entorno del desarrollador.

La configuración transforma `jwt.secret` en un bean/valor que `JwtService` utiliza para firmar y verificar. Centralizarlo evita que cada método construya claves de forma diferente. También simplifica tests: el perfil de test puede proporcionar un secreto conocido y suficientemente largo.

Una propiedad ausente en producción debe hacer fallar claramente el arranque o la configuración, no caer silenciosamente a un secreto por defecto inseguro.

Ejemplo.

```
jwt.secret=${JWT_SECRET}
```

La configuración puede materializar la clave una sola vez como bean:

```
@Bean
SecretKey jwtSecretKey(@Value("${jwt.secret}") String encoded) {
    byte[] bytes = Decoders.BASE64.decode(encoded);
    return Keys.hmacShaKeyFor(bytes);
}
```

De este modo JwtService no conoce cómo se obtuvo el secreto: recibe SecretKey por inyección de dependencias. Esta separación facilita tests y evita repetir decodificación. También permite que el valor de producción venga de \${JWT_SECRET} sin modificar el servicio.

Si la cadena no es Base64 válida o los bytes son insuficientes, el arranque debe fallar de forma visible. Es preferible descubrir una clave insegura al iniciar la aplicación que aceptar silenciosamente una configuración criptográfica débil. > **Pregunta de reflexión:** ¿Por qué Keys.hmacShaKeyFor lanza una excepción si la clave es demasiado corta?

Respuesta razonada: Porque un algoritmo HMAC necesita una clave con entropía suficiente para alcanzar la seguridad esperada. JWT rechaza claves débiles para evitar que una configuración aparentemente funcional degrade la resistencia criptográfica del token.

Bloque 3 - Generar un token**T3.1 - La estructura del servicio**

JwtService centraliza generar, parsear y validar tokens. No debe conocer HTTP ni decidir roles de endpoints. Sus entradas son identidades/claims y sus salidas son tokens o datos verificados.

Concentrar la lógica evita que login, refresh y filtro implementen variantes ligeramente distintas de la misma validación.

JwtService encapsula operaciones de token: generar access/refresh, extraer claims y validar. Esa frontera mantiene detalles de JWT fuera de controladores y filtros. El filtro pregunta al servicio si un token es válido; no reconstruye criptografía por su cuenta.

Separar métodos para access y refresh ayuda a expresar claims y duraciones distintas. El nombre y tipo del token deben poder comprobarse para impedir intercambiarlos.

La interfaz pública del servicio debe ser pequeña y semántica: generar access, generar refresh, extraer sujeto/claims y validar el propósito del token. Ocultar llamadas directas a JWT detrás de esa capa reduce duplicación y permite endurecer validaciones en un solo lugar. Si controlador y filtro parsearan por separado, una futura corrección podría aplicarse sólo a uno y dejar políticas inconsistentes.

JwtService concentra la política técnica de los tokens: generación de access, generación de refresh, parseo, extracción de claims y validación de tipo. Los controladores no deben conocer detalles de JWT ni construir firmas manualmente; delegan en este servicio.

El constructor recibe la SecretKey y las duraciones configuradas. Así el servicio es determinista respecto a su configuración y puede probarse aislado. También conviene centralizar nombres de claims propios como tipo, porque una discrepancia entre emisor y validador convertiría un token correcto criptográficamente en un token imposible de usar.

La interfaz del servicio refleja operaciones de negocio de seguridad —generar, extraer, validar— en lugar de exponer el parser de JWT a toda la aplicación. Esa encapsulación reduce acoplamiento y hace más sencillo migrar de biblioteca o algoritmo.

La fuente introduce primero el esqueleto completo del servicio para que sea visible qué responsabilidades crecerán después:

```
@Service
public class JwtService {
    private final SecretKey clave;
    private final long expirationMs;
    private final long refreshExpirationMs;

    public JwtService(
        SecretKey clave,
        @Value("${jwt.expiration}") long expirationMs,
        @Value("${jwt.refresh-expiration}") long refreshExpirationMs) {
```

```

        this.clave = clave;
        this.expirationMs = expirationMs;
        this.refreshExpirationMs = refreshExpirationMs;
    }

    public String generarToken(Usuario usuario) {
        // ...
    }

    public String generarRefreshToken(Usuario usuario) {
        // ...
    }

    public String extraerUsername(String token) {
        // ...
    }

    public Claims extraerClaims(String token) {
        // ...
    }

    public List<GrantedAuthority> extraerAuthorities(String token) {
        // se incorporará cuando el filtro necesite reconstruir authorities
    }

    public boolean esValido(String token) {
        // ...
    }
}

```

T3.2 - Generar el token de acceso

El access token incluye subject, fechas de emisión/expiración y los claims que el diseño necesite. Se firma con HS256 y la clave configurada. La duración proviene de `jwt.expiration`, no de un número mágico repetido.

Un token generado se considera válido sólo si más tarde el parser puede verificar firma y expiración. Los tests de `JwtService` deben cubrir ambas cosas.

El access token representa una autorización de corta duración. Incluye sujeto, roles necesarios y tiempos `iat/exp`; se firma con HS256. No necesita contener todos los campos de `Usuario`: cuanto más pequeño y estable, mejor.

Al construirlo, los tiempos se calculan desde un reloj controlable en tests cuando sea posible. Los tests no deberían depender de dormir segundos para demostrar expiración si pueden fijar tiempos.

El access token incluye identidad y permisos necesarios para autorizar peticiones durante un periodo corto. Un ejemplo conceptual del builder es:

```

return Jwts.builder()
    .subject(usuario.getUsername())
    .id(UUID.randomUUID().toString())
    .claim("tipo", "access")
    .claim("id", usuario.getId())
    .claim("email", usuario.getEmail())
    .claim("roles", roles)
    .issuedAt(ahora)
    .expiration(expiracion)
    .signWith(clave, Jwts.SIG.HS256)
    .compact();

```

`sub`, `jti`, `iat` y `exp` tienen significado estándar; `tipo`, `id`, `email` y `roles` son claims propios. La duración corta limita el impacto de un robo y obliga a usar `refresh` cuando la sesión lógica debe prolongarse.

La generación del access token incorpora identidad, roles y fechas antes de firmar:

```

public String generarToken(Usuario usuario) {
    Date ahora = new Date();
    Date expiracion = new Date(ahora.getTime() + expirationMs);

    List<String> roles = usuario.getRoles().stream()

```

```

        .map(rol -> "ROLE_" + rol.getNombre())
        .toList();

    return Jwts.builder()
        .subject(usuario.getUsername())
        .claim("id", usuario.getId())
        .claim("email", usuario.getEmail())
        .claim("roles", roles)
        .issuedAt(ahora)
        .expiration(expiracion)
        .signWith(clave)
        .compact();
}

```

T3.3 - Generar el refresh token

El refresh token se firma igual, pero tiene una vida distinta y un propósito específico. Conviene identificar su tipo o gestionar su uso en un servicio de revocación para impedir que se utilice como access token por accidente.

M6 implementa rotación: al refrescar se invalida el refresh anterior y se entrega uno nuevo. Esto reduce la ventana de reutilización.

El refresh token dura más y sólo sirve para renovar. Debe diferenciarse del access mediante claim/tipo y tratarse como credencial de alto valor. El diseño final rota refresh tokens: usar uno produce otro y el anterior deja de ser reutilizable.

Ese comportamiento reduce el riesgo de replay y debe quedar cubierto por un test: primera renovación válida, segundo intento con el mismo refresh rechazado.

Ejemplo.

```

return Jwts.builder()
    .subject(username)
    .signWith(clave, Jwts.SIG.HS256)
    .compact();

```

Pregunta de reflexión: ¿Por qué el refresh token no incluye los roles del usuario?

Respuesta razonada: El refresh token no se usa para autorizar peticiones de negocio, sólo para obtener una nueva pareja de tokens. Mantenerlo mínimo reduce exposición de datos y permite que, al refrescar, los roles actuales se vuelvan a obtener de la fuente de identidad en vez de perpetuar permisos antiguos.

Bloque 4 - El endpoint de login

T4.1 - El flujo de login

El cliente envía username y password al endpoint de login. AuthenticationManager valida las credenciales contra UserDetailsService y PasswordEncoder. Sólo después del éxito se generan access y refresh token.

Separar autenticación y emisión de token es importante: un JWT no “comprueba” la contraseña; simplemente representa una identidad que ya fue autenticada.

El login recibe credenciales, pide autenticación al AuthenticationManager y, sólo tras éxito, emite tokens. El controlador no decide si la contraseña coincide ni carga roles manualmente. Esta secuencia mantiene cada responsabilidad en su capa.

Una respuesta 401 no debe revelar si falló username o contraseña. El resultado externo es credenciales inválidas; el detalle interno puede registrarse de forma segura sin filtrar secretos.

El flujo debe tratar autenticación y emisión como una operación lógica: sólo un Authentication validado autoriza a crear tokens. Si la generación falla después de autenticar, la respuesta no debe dejar un estado parcial persistido que simule sesión. En nuestro diseño stateless no se crea sesión HTTP, por lo que un nuevo intento de login puede repetirse limpiamente.

El login no “crea un usuario autenticado para siempre”. Recibe username/password, delega la comprobación en Spring Security y, sólo si las credenciales son correctas, recupera la entidad necesaria para emitir tokens. El flujo puede resumirse como: DTO → AuthenticationManager → usuario válido → JwtService → respuesta con access y refresh.

Un fallo de credenciales debe terminar en 401 con un contrato estable y sin revelar si el username

existe. Tras el éxito, el servidor ya no necesita conservar una sesión HTTP del cliente: devuelve las credenciales temporales que se presentarán en las llamadas posteriores. Esta separación entre login inicial y autenticación con Bearer es el corazón de la transición desde HTTP Basic a JWT.

T4.2 - El AuthenticationManager en el login

AuthenticationManager recibe un UsernamePasswordAuthenticationToken no autenticado. Si el proveedor acepta las credenciales, devuelve otro objeto autenticado con el principal y sus authorities.

El servicio usa ese resultado como fuente de verdad para emitir tokens. No busca al usuario por su cuenta y compara la contraseña manualmente.

AuthenticationManager recibe un UsernamePasswordAuthenticationToken no autenticado con las credenciales. Si un proveedor lo valida, devuelve otro estado autenticado con principal y authorities. Esa salida es la fuente de información para emitir claims coherentes.

Emitir un JWT antes de autenticar o construir roles desde datos del request sería una vulnerabilidad directa. El orden del flujo es una propiedad de seguridad, no un detalle de estilo.

Después de autenticar, el objeto devuelto contiene las authorities efectivas. Emitir los claims desde esa identidad validada evita reconstruir roles desde el request o hacer una segunda interpretación potencialmente distinta. El test debe demostrar que un usuario con rol conocido obtiene un token cuyos claims conducen posteriormente a la misma autorización esperada.

AuthenticationManager sigue siendo útil aunque la API final use JWT. Su trabajo en /login es validar las credenciales iniciales contra UserDetailsService y PasswordEncoder. Se le entrega normalmente un UsernamePasswordAuthenticationToken no autenticado; el manager lo procesa y devuelve una autenticación válida o lanza una excepción.

Después del login, el AuthenticationManager deja de intervenir en cada request. El filtro JWT valida el token directamente con JwtService. Esta distinción evita un error conceptual frecuente: JWT no sustituye el proceso de verificar la contraseña en el momento de inicio de sesión; sustituye la necesidad de reenviar esa contraseña o mantener una sesión de servidor en cada petición posterior.

El login reutiliza el AuthenticationManager; no compara la contraseña manualmente:

```
@Service
public class AuthService {
    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UsuarioRepository usuarioRepository;

    public AuthService(
        AuthenticationManager authenticationManager,
        JwtService jwtService,
        UsuarioRepository usuarioRepository) {
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
        this.usuarioRepository = usuarioRepository;
    }

    public LoginResponseDTO login(LoginRequestDTO request) {
        try {
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                    request.getUsername(), request.getPassword()));
        } catch (BadCredentialsException e) {
            throw new NegocioException("Credenciales incorrectas");
        }

        Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new RecursoNoEncontradoException(
                "Usuario", request.getUsername()));

        String token = jwtService.generarToken(usuario);
        String refreshToken = jwtService.generarRefreshToken(usuario);
        return new LoginResponseDTO(
            token, refreshToken, jwtService.getExpirationMs());
    }
}
```

En el proyecto final, el `BadCredentialsException` se traduce mediante `CredencialesInvalidasException` a 401; el ejemplo se conserva para mostrar el flujo original.

T4.3 - El DTO de respuesta del login

La respuesta expone `access_token`, `refresh_token`, `token_type` y `expires_in`. Un DTO explícito estabiliza el contrato JSON y evita devolver objetos internos de Spring Security.

`token_type` vale `Bearer`, lo que indica al cliente cómo usar el access token en la cabecera `Authorization`.

El DTO de login debe devolver los tokens y metadatos necesarios sin exponer objetos internos de Spring. El cliente necesita saber qué usar como `Bearer` y, si procede, cuánto dura; no necesita el hash ni toda la entidad.

Mantener un contrato explícito también facilita OpenAPI y tests. Si la forma cambia, los tests de serialización detectan la regresión.

Ejemplo.

`LoginRequest` -> `AuthenticationManager` -> `JwtService` -> `LoginResponse`

La respuesta de login debe ser explícita y estable para el cliente. Incluye `access_token`, `refresh_token`, `token_type` y `expires_in`. El último valor suele expresarse en segundos, aunque internamente la configuración Java pueda usar milisegundos.

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

El DTO evita devolver entidades JPA y permite controlar exactamente el contrato JSON. Tampoco incluye contraseña, hash ni detalles internos del usuario. Mantener separados `access` y `refresh` es importante: tienen finalidades y duraciones diferentes y el servidor debe rechazar un `access` donde espera `refresh` y viceversa.

El DTO conserva la convención de nombres que recibirá el cliente:

```
public class LoginResponseDTO {
    @JsonProperty("access_token")
    private String accessToken;

    @JsonProperty("refresh_token")
    private String refreshToken;

    @JsonProperty("token_type")
    private String tokenType = "Bearer";

    @JsonProperty("expires_in")
    private long expiresIn;

    public LoginResponseDTO(
        String accessToken,
        String refreshToken,
        long expiresIn) {
        this.accessToken = accessToken;
        this.refreshToken = refreshToken;
        this.expiresIn = expiresIn;
    }
}
```

Pregunta de reflexión: ¿Qué pasos completos debe atravesar un login correcto antes de devolver tokens y qué respuesta corresponde a credenciales inválidas?

Respuesta razonada: El login recibe DTO validado, delega las credenciales al `AuthenticationManager`, recupera la identidad persistida y emite `access/refresh`. Credenciales incorrectas deben terminar en el error de autenticación acordado, no en tokens.

Bloque 5 - Refresh tokens y buenas prácticas

T5.1 - El endpoint de refresh

El refresh recibe un refresh token, comprueba que sea válido y utilizable, lo consume/rota y devuelve un nuevo par. No vuelve a pedir contraseña porque el refresh token es precisamente la credencial de renovación.

Si el token está expirado, revocado o ya usado, la operación debe fallar. No debe “arreglarse” aceptándolo de todas formas.

El endpoint de refresh recibe una credencial distinta del access token, valida firma, expiración, tipo y estado de uso, y emite un nuevo par según la política. No debe requerir un access válido, porque precisamente puede utilizarse cuando el access expiró.

Al mismo tiempo, refresh no puede quedar abierto a cualquier JWT firmado. Distinguir tipo y aplicar rotación evita que un access token se utilice como refresh o que el mismo refresh se reproduzca indefinidamente.

El endpoint de refresh permite obtener un nuevo par de tokens sin volver a enviar la contraseña. Primero valida que el token presentado está correctamente firmado, no ha expirado y además tiene tipo=refresh. Después identifica al usuario, genera nuevas credenciales y marca el refresh anterior como consumido cuando aplicamos rotación de un solo uso.

Esta propiedad evita que un refresh robado pueda reutilizarse indefinidamente después de que el cliente legítimo ya lo haya canjeado. El test correcto no termina con el primer 200: repite la misma petición y espera rechazo. También se prueba que un access token no sea aceptado por /refresh, porque compartir firma no significa compartir propósito.

T5.2 - Buenas prácticas con tokens

Los errores de tokens se traducen a respuestas coherentes, sin exponer la clave ni detalles criptográficos. Las duraciones se externalizan y el perfil de producción exige el secreto desde JWT_SECRET.

También conviene mantener los access tokens cortos y reservar el estado de revocación para los casos donde aporta valor, como logout y refresh de un solo uso.

Los tokens deben viajar por HTTPS y minimizar exposición en logs, URLs y almacenamiento inseguro. La elección de almacenamiento en cliente depende del tipo de cliente y amenaza; no existe una receta universal sin contexto.

En el servidor, secretos de firma y tokens revocados son cosas distintas. JWT_SECRET configura al emisor/verificador; la lista de revocación registra identificadores/tokens que ya no deben aceptarse.

Los access tokens deben durar poco y los refresh algo más, pero ninguna duración sustituye a una buena gestión de secretos. Se recomienda HTTPS, claims mínimos, distinguir tipos, usar jti, rotar refresh, no registrar tokens completos y no codificar la clave en el repositorio. El cliente debe tratar ambos tokens como credenciales sensibles.

La revocación en memoria del curso es deliberadamente educativa: permite demostrar logout y replay de refresh sin introducir Redis u otra infraestructura. En un sistema distribuido necesitaríamos un almacenamiento compartido o una estrategia diferente, porque cada instancia tendría su propia memoria. La guía debe dejar clara esa limitación para no presentar el ejemplo didáctico como arquitectura de producción.

T5.3 - Errores comunes

Entre los errores comunes están usar dependencias JJWT de versiones mezcladas, firmar con una clave String insuficiente, usar un algoritmo implícito, dejar un secreto de producción con fallback y no distinguir access de refresh.

Los tests de integración de JwtService son el lugar adecuado para capturar estos fallos antes de probar toda la API.

Ejemplo.

```
String token = Jwts.builder()  
    .subject(username)  
    .issuedAt(Date.from(ahora))  
    .expiration(Date.from(ahora.plusMillis(expirationMs)))
```

```
.signWith(clave, Jwts.SIG.HS256)
.compact();
```

En producción la clave se obtiene de JWT_SECRET; no se incluye una alternativa insegura en el propio fichero de configuración.

Errores habituales: secreto corto, dependencias JWT incompletas, aceptar refresh como access, no rotar refresh, abrir todo /auth/**, devolver 500 ante token expirado o registrar el token completo. Cada uno debe tener una ruta de diagnóstico y una prueba negativa.

La práctica moderna evita APIs de JWT antiguas; si un tutorial usa firmas deprecated, se entiende el concepto y se traduce a la API 0.13.0, no se copia literalmente.

Ejemplo.

```
refresh #1 -> válido -> se rota
refresh #1 reutilizado -> rechazado
```

Conviene reconocer los fallos concretos que aparecen al construir el servicio. Una clave demasiado corta para HS256 provoca WeakKeyException; la solución es usar material criptográfico con longitud suficiente y construir una SecretKey válida. Si falta el bean de clave, el JwtService ni siquiera puede inicializarse. Si se firma con una clave y se intenta verificar con otra, el token debe rechazarse: no se corrige desactivando la validación.

En el flujo de login, una BadCredentialsException sin traducir puede terminar como 500, cuando el contrato correcto es un 401 controlado. Si /login no está entre las rutas públicas, Spring Security lo bloqueará antes de que pueda autenticar a nadie. expires_in debe expresarse en la unidad que define el DTO; si internamente trabajamos en milisegundos y la respuesta promete segundos, hay que convertirlo. El refresh, además, no puede limitarse a “parsear un JWT”: debe comprobar firma, expiración y que el token sea realmente de tipo refresh. Estos errores se prueban de forma negativa, porque un camino feliz por sí solo no demuestra que el servicio rechace credenciales o tokens incorrectos.

Pregunta de reflexión: ¿Cómo demostrarías que un refresh es de un solo uso y que su replay no genera una segunda pareja de tokens?

Respuesta razonada: Un refresh de un solo uso se demuestra consumiendo el mismo jti dos veces: la primera petición genera una nueva pareja y la segunda se rechaza. Ese test prueba la protección frente a replay.

Resumen de la teoría

- JWT encapsula construcción, firma, parseo y validación de JWT.
- La clave de firma debe tener entropía suficiente y mantenerse fuera del repositorio.
- El access token es corto y transporta los claims necesarios; el refresh token dura más y cumple otra función.
- El login valida primero usuario/contraseña y sólo entonces emite tokens.
- Refresh, rotación y logout reducen el riesgo de reutilización de tokens robados.

Punto 6.7 - Filtro JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué JWT necesita un filtro personalizado en Spring Security.
2. Entender qué es OncePerRequestFilter y cuándo usarlo.
3. Implementar un filtro que extraiga el token de la cabecera Authorization.
4. Verificar el token con el JwtService y construir un UsuarioPrincipal.
5. Colocar el Authentication en el SecurityContextHolder.
6. Registrar el filtro en la cadena de seguridad.
7. Diagnosticar y resolver los errores más comunes al implementar un filtro JWT.

Bloque 1 - Por qué un filtro personalizado

T1.1 - El problema de la autenticación con JWT

Después del login el cliente ya no envía username y password en cada llamada. Envía un Bearer token. La aplicación necesita convertir ese token en una identidad que Spring Security pueda reconocer antes de evaluar cualquier regla de acceso.

Ese trabajo se realiza en un filtro. El filtro valida la credencial y, si es correcta, construye un Authentication que se almacena en el SecurityContext. A partir de ese punto, controladores y expresiones de autorización trabajan igual que con cualquier otro mecanismo de autenticación.

Una vez emitido el JWT, cada petición protegida debe convertirlo de nuevo en una identidad de Spring. Sin ese puente, SecurityFilterChain sólo ve una petición anónima aunque la cabecera Bearer sea correcta. El filtro personalizado realiza ese trabajo por petición.

El filtro no “crea permisos” arbitrariamente: valida el token y reconstruye un principal coherente con las authorities firmadas/usuario. La autorización se ejecuta después sobre ese contexto.

Después de emitir un access token, Spring Security no lo entiende automáticamente si hemos construido nuestro propio esquema con JWT. Una petición puede traer Authorization: Bearer ... y seguir siendo anónima si ningún componente extrae, verifica y transforma ese token en un Authentication.

El filtro cubre precisamente esa frontera. Se ejecuta antes del controlador, lee la cabecera, valida el token, reconstruye identidad y authorities y coloca la autenticación en el SecurityContext. A partir de ese momento las reglas de URL y @PreAuthorize funcionan con la identidad JWT igual que antes funcionaban con HTTP Basic. Sin filtro, el token sería sólo texto transportado por el cliente.

T1.2 - Por qué OncePerRequestFilter

OncePerRequestFilter garantiza una ejecución controlada por petición y ofrece una base adecuada para filtros propios de Spring. Evita tener que gestionar directamente detalles del ciclo servlet y hace más clara la implementación de doFilterInternal.

El filtro debe ser pequeño: extraer credencial, validarla, reconstruir identidad y continuar la cadena. La lógica de negocio no pertenece aquí.

OncePerRequestFilter ofrece una base de Spring que evita ejecutar el filtro varias veces dentro de una misma dispatch normal y proporciona una estructura clara para doFilterInternal. Es apropiado para autenticación Bearer por petición.

Aun así, el filtro debe siempre delegar cuando corresponde. Una petición sin Bearer no es automáticamente error si la ruta puede ser pública; se continúa y será la política de autorización quien decida si se exige identidad.

OncePerRequestFilter es una base de Spring pensada para ejecutar la lógica del filtro una vez por petición dentro del ciclo normal. Implementamos doFilterInternal(...), lo que evita tener que gestionar manualmente detalles del contrato Servlet y hace la clase más fácil de integrar como bean.

El filtro no debería almacenar estado específico de una petición en campos de instancia, porque el bean se reutiliza entre solicitudes. Todo dato temporal —token, claims, principal— permanece en variables locales o en el SecurityContext. La clase debe ser además tolerante: si la petición no tiene Bearer y la ruta puede ser pública, simplemente continúa la cadena; la autorización posterior decidirá si el acceso anónimo es válido.

OncePerRequestFilter concentra la implementación en un único método por petición:

```
@Override
protected void doFilterInternal(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain filterChain)
    throws ServletException, IOException {
    // ...
}
```

T1.3 - El flujo completo de una petición con JWT

La secuencia es: llega la petición, el filtro busca Authorization, extrae el Bearer token, valida firma y expiración, obtiene el username, carga el usuario, crea un UsernamePasswordAuthenticationToken autenticado y lo guarda en SecurityContextHolder. Después llama a filterChain.doFilter.

Si no hay token, el filtro no inventa una identidad; deja que la cadena continúe y que la política de seguridad decida si la ruta admite anónimo o exige autenticación.

El flujo completo es: llega petición, se lee Authorization, se extrae Bearer, se comprueba revocación/validez, se obtiene identidad, se construye Authentication, se guarda en SecurityContext y la cadena continúa. Más tarde las reglas autorizan o rechazan.

Descomponer el flujo ayuda a localizar fallos. Si no hay contexto, se investiga extracción/validación; si existe contexto con roles correctos y hay 403, se investiga la regla.

Ejemplo.

Bearer -> JwtAuthenticationFilter -> SecurityContext -> AuthorizationFilter

Pregunta de reflexión: ¿Por qué el filtro JWT debe ejecutarse en cada petición protegida y qué estado deja en SecurityContext cuando el token es válido?

Respuesta razonada: El token sólo tiene efecto si una pieza de la cadena lo lee y crea un Authentication. Sin el filtro, Spring Security ve una petición sin identidad aunque la cabecera Bearer esté presente.

Bloque 2 - Extraer el token

T2.1 - La cabecera Authorization

La forma estándar es Authorization: Bearer <token>. El prefijo y el espacio son parte del formato. El filtro debe comprobarlos antes de intentar parsear la cadena.

No conviene aceptar tokens desde parámetros query porque pueden quedar registrados en históricos, proxies o logs. La cabecera mantiene la credencial en el canal previsto por HTTP.

La sintaxis estándar es Authorization: Bearer <token>. El filtro debe verificar el prefijo y evitar operaciones de substring inseguras sobre cabeceras cortas o mal formadas. Espacios extraños y esquemas diferentes deben tratarse de forma predecible.

El token es una credencial: no se incluye completo en mensajes de error. Para diagnóstico se puede usar un identificador no sensible o un traceId.

Debe aceptarse exactamente el esquema previsto. Una cabecera Token ..., un Bearer sin valor o una cadena mal formada no se “corrigen” por intuición; se tratan como ausencia/credencial inválida según la política documentada. Ese comportamiento estable es importante para clientes y para evitar parsers excesivamente permisivos.

La convención Bearer usa una cabecera como:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

El filtro comprueba que la cabecera existe y comienza exactamente por Bearer antes de extraer el resto. No debe usar un replace indiscriminado que pueda eliminar texto en posiciones inesperadas. También conviene rechazar o ignorar de forma controlada una cabecera vacía o mal formada sin provocar excepciones 500.

La palabra Bearer recuerda que quien **porta** el token puede utilizarlo mientras siga siendo válido; no existe una segunda prueba de identidad en cada petición. Por eso el transporte HTTPS y la protección del token en el cliente son indispensables.

T2.2 - Qué hacer si no hay token

La ausencia de token no es automáticamente un error del filtro. Una ruta pública puede funcionar perfectamente sin él. Por eso el filtro simplemente continúa cuando la cabecera no existe o no empieza por Bearer.

Si la ruta es protegida, la fase de autorización detectará que no existe autenticación y el AuthenticationEntryPoint responderá 401. Esta separación evita codificar el mapa de rutas dentro del filtro.

Si no existe token, el filtro no inventa un 401 inmediatamente. Delegar permite que endpoints públicos funcionen y que AuthorizationFilter/entry point produzcan el 401 en rutas protegidas. Esto mantiene una única política de acceso.

La misma regla se aplica a rutas de login/registro: el filtro puede no autenticar y la allowlist permite continuar. Evitar excepciones especiales dispersas simplifica el diseño.

La delegación sin token es además lo que permite que CORS preflight y endpoints públicos atraviesen la cadena sin que el filtro JWT conozca cada ruta. El filtro permanece genérico; la allowlist sigue centralizada en SecurityConfig, reduciendo duplicación y riesgo de divergencia.

La ausencia de token no es por sí misma un error del filtro. Una petición a `/api/v1/public/**`, Swagger o un preflight CORS debe poder atravesar la cadena sin autenticación. El filtro llama a `filterChain.doFilter(...)` y deja el SecurityContext sin identidad propia.

Más adelante AuthorizationFilter evalúa la política. Si la ruta es `permitAll`, continúa; si exige autenticación, se activará el AuthenticationEntryPoint y el cliente recibirá 401. Esta separación evita duplicar las reglas de rutas dentro del filtro y dentro de SecurityConfig. El filtro resuelve autenticación cuando hay una credencial; la configuración decide si una ruta necesita esa autenticación.

T2.3 - No usar el filtro para autorizar

El filtro autentica, no decide si USER puede entrar en `/admin`. Las reglas de autorización deben permanecer en SecurityFilterChain o en `@PreAuthorize`.

Mezclar ambas responsabilidades convierte el filtro en un conjunto de `if` por URL y hace muy difícil mantener la política. El filtro sólo establece una identidad fiable.

Autenticar responde “quién es”; autorizar responde “puede hacer esto”. El filtro debe limitarse a la primera responsabilidad. No debe conocer que `/admin` exige ADMIN ni que un método concreto requiere GESTOR.

Separar responsabilidades evita que al cambiar una ruta haya que editar el filtro y la configuración. También permite testear el filtro con independencia de cada política de negocio.

Esta separación también mejora los mensajes de error: un token inválido es un problema de autenticación; un token válido con rol insuficiente es autorización. Si el filtro tomara ambas decisiones, sería fácil convertir todo rechazo en 401 y perder semántica, observabilidad y capacidad de prueba.

Ejemplo.

Authorization: Bearer eyJ...

Pregunta de reflexión: ¿Por qué el filtro JWT no debe autorizar? ¿Qué problema tendría si lo hiciera?

Respuesta razonada: El filtro debe autenticar: validar el token y construir la identidad. La autorización debe permanecer en las reglas de SecurityFilterChain y en las anotaciones de método. Si el filtro autorizara también, duplicaría políticas, mezclaría responsabilidades y sería más fácil obtener decisiones inconsistentes.

Bloque 3 - Construir el Authentication

T3.1 - Qué es un Authentication

Authentication representa la identidad de seguridad dentro de Spring. Antes de validar credenciales puede ser una solicitud no autenticada; después contiene principal, authorities y el indicador de autenticación.

En JWT no reutilizamos la credencial como contraseña. Una vez verificado el token, construimos un Authentication autenticado con el principal y sus authorities para el resto de la petición.

Authentication contiene principal, credenciales y authorities, además de su estado autenticado. Para una petición JWT ya validada se construye una autenticación sin necesidad de conservar la credencial secreta original.

Después de instalarla en SecurityContextHolder, el resto de Spring Security puede aplicar `hasRole`, inyectar principal y evaluar expresiones. Esa integración es el objetivo del filtro.

Las credenciales dentro de un Authentication ya autenticado suelen poder ser `null` porque el Bearer fue verificado antes. Conservar el token completo como credencial en el contexto aumentaría el riesgo de que apareciera por accidente en logs o serializaciones. El principal y sus authorities bastan para las decisiones posteriores del módulo.

Authentication es el objeto con el que Spring Security representa la identidad durante una petición. Contiene principal, authorities, detalles opcionales y el estado de autenticación. En el

flujo JWT construimos un UsernamePasswordAuthenticationToken usando un UsuarioPrincipal ya validado y una colección de authorities.

```
var auth = new UsernamePasswordAuthenticationToken(
    principal,
    null,
    principal.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
```

Las credenciales se pasan como null porque el token JWT ya fue la evidencia validada; no almacenamos la contraseña ni el token bruto como credencial en el contexto. Desde ese momento controladores y seguridad por método pueden consultar la identidad.

T3.2 - Usar UsuarioPrincipal como principal

UsuarioPrincipal adapta la entidad Usuario al contrato de Spring Security y puede exponer identificador, username, email y authorities sin revelar la contraseña. Esto mejora endpoints de perfil y expresiones SpEL que necesitan el id del usuario.

El principal debe ser inmutable desde el punto de vista del request. Cambiar roles en memoria durante una petición no debe sustituir la fuente persistente de verdad.

UsuarioPrincipal permite transportar el id interno junto con username y authorities. Esto habilita reglas de propiedad sin consultar por username repetidamente y evita usar la entidad JPA como principal vivo fuera de su contexto de persistencia.

Nunca debe incluirse la contraseña en serializaciones o logs. Aunque UserDetails exponga getPassword para autenticación, el objeto no se devuelve directamente al cliente.

UsuarioPrincipal permite que el principal tenga datos que el User estándar no expone, como id y email, además de username y authorities. Esto habilita expresiones como:

```
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
```

El principal se construye sólo después de verificar el access token. Sus valores proceden de claims firmados y el objeto implementa UserDetails para integrarse con el ecosistema Spring Security. getPassword() puede devolver null, porque este principal representa una identidad ya autenticada por token, no una entidad destinada a validar contraseña.

Una vez verificados los claims, el principal se reconstruye sin consultar de nuevo la base de datos:

```
Claims claims = jwtService.extraerClaims(token);
Long id = claims.get("id", Long.class);
String username = claims.getSubject();
String email = claims.get("email", String.class);
List<GrantedAuthority> authorities = jwtService.extraerAuthorities(token);
```

```
UsuarioPrincipal principal = new UsuarioPrincipal(
    id, username, email, authorities);
```

T3.3 - El UsuarioDetailsService en el flujo

Después de extraer el username del token, el filtro carga la identidad actual con UsuarioDetailsService. Esto permite usar los roles vigentes de la base de datos en lugar de confiar ciegamente en datos antiguos almacenados en el token.

La decisión también simplifica el diseño: el token identifica al usuario y la aplicación reconstruye el principal con la misma lógica utilizada por el login.

Hay dos estrategias comunes: confiar en claims del token o recargar el usuario desde base de datos. Recargar permite detectar cuentas desactivadas/roles cambiados antes de expirar el token, a costa de una consulta por petición. El diseño del curso utiliza sus componentes de forma coherente con el estado final documentado.

Lo importante es no mezclar estrategias accidentalmente. La documentación debe indicar de dónde salen las authorities efectivas y qué cambio de cuenta se refleja inmediatamente.

Ejemplo.

```
var auth = new UsernamePasswordAuthenticationToken(principal, null, principal.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
```

Pregunta de reflexión: ¿Por qué el filtro JWT no consulta la base de datos en cada petición?

Respuesta razonada: Porque los claims verificados ya contienen la identidad y las authorities necesarias para la autorización de esta arquitectura stateless. Consultar la base en cada petición eliminaría parte de la ventaja del token autocontenido y añadiría latencia; se consulta cuando una operación necesita estado de negocio actual.

Bloque 4 - Registrar el filtro

T4.1 - Añadir el filtro a la cadena

El filtro se registra con `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`. Así la identidad JWT existe antes de que las reglas de autorización necesiten el contexto.

Registrar un filtro no basta: también debemos retirar HTTP Basic del estado final para que la API no acepte dos mecanismos diferentes sin intención.

Registrar el filtro con `addFilterBefore` lo coloca en la cadena administrada por Spring Security. No se debe registrar además como filtro Servlet independiente si eso provoca doble ejecución. La configuración debe tener una sola ruta de instalación.

El ancla elegida sirve para garantizar que el contexto exista antes de autorización. El comportamiento se valida con un Bearer real en integración, no sólo comprobando que aparece una línea en `SecurityConfig`.

El filtro JWT debe situarse **antes** del filtro que tradicionalmente procesa username/password. La configuración moderna lo expresa con:

```
.addFilterBefore(  
    jwtAuthenticationFilter,  
    UsernamePasswordAuthenticationFilter.class)
```

La intención es que cuando llegue la fase de autorización el `SecurityContext` ya pueda contener la identidad reconstruida desde Bearer. Registrar el bean no basta: un filtro de Spring Security participa en la cadena sólo si la configuración lo añade en la posición correspondiente.

Al completar esta transición se desactiva HTTP Basic para el uso normal de la API. Login sigue aceptando usuario/contraseña en su DTO, pero las demás peticiones protegidas se autentican con access token.

T4.2 - El filtro como bean

`JwtAuthenticationFilter` es un componente administrado por Spring y recibe `JwtService`, `UsuarioDetailsService` y, cuando procede, el servicio de revocación. La inyección por constructor hace visibles sus dependencias y facilita tests.

No debe crear repositorios o parsers por su cuenta. La configuración y validación de tokens pertenecen a servicios dedicados.

Declarar el filtro como bean/componente permite inyectar `JwtService`, `UserDetailsService` y revocación. Las dependencias quedan visibles y testeables. En slices MVC, sin embargo, que el filtro sea bean implica que Spring puede intentar instanciarlo aunque `MockMvc` no aplique filtros.

Por eso los tests slice aportan `@MockitoBean` para los colaboradores necesarios. Desactivar filtros y sustituir dependencias son problemas diferentes.

El ciclo de construcción del contexto también importa. Un filtro con dependencias constructor-injected obliga a que esas dependencias existan al crear el bean, incluso si un test concreto desactiva su ejecución. Esa diferencia entre “bean creado” y “filtro aplicado a una petición” explica el fallo histórico de los slices MVC y por qué `@MockitoBean` es la corrección adecuada en esos tests.

Declarar `JwtAuthenticationFilter` como componente permite inyectarle `JwtService` y el servicio de revocación por constructor. Esa dependencia explícita facilita tests y evita búsquedas estáticas. El filtro no crea claves, no consulta configuración directamente y no implementa por su cuenta la lógica criptográfica.

Ser un bean también significa que su ciclo de vida es administrado por Spring y que puede aparecer durante la creación de contextos de test. Esta es la causa del antecedente con `@WebMvcTest`: aunque `MockMvc` no ejecute filtros, el contexto puede intentar instanciar el bean y necesitar sus colaboradores. Los tests slice deben proporcionar mocks cuando la capa probada no incluye esos servicios reales.

El filtro debe ser un bean para poder inyectar sus dependencias y registrarse en la cadena:

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    // ...
}

```

T4.3 - Orden de los filtros

Un orden incorrecto puede producir 401 aunque el token sea correcto o duplicar autenticaciones. La referencia a UsernamePasswordAuthenticationFilter ofrece un punto conocido en la cadena.

También conviene evitar volver a autenticar si SecurityContextHolder ya contiene una identidad válida. El filtro puede comprobarlo antes de cargar de nuevo el usuario.

El orden entre CORS, contexto, autenticación Bearer, traducción de excepciones y autorización determina resultados. Un preflight no debe quedar bloqueado como si fuera una petición de negocio; un Bearer debe establecer identidad antes de la regla protegida.

La mejor prueba del orden es funcional: OPTIONS autorizado según CORS, ruta protegida con token válido funciona y la misma ruta sin token produce el error JSON esperado.

Ejemplo.

```
http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

El orden de filtros no es un detalle cosmético. Si la autorización se ejecutara antes de que el Bearer construya el Authentication, una petición con token válido seguiría pareciendo anónima. Si el filtro se colocara varias veces, podríamos repetir parseo y efectos secundarios innecesarios.

Spring Security posee una secuencia predefinida de filtros. addFilterBefore y addFilterAfter permiten anclar uno personalizado respecto a un filtro conocido. En nuestro caso se usa UsernamePasswordAuthenticationFilter como referencia. Para depurar el orden pueden activarse logs DEBUG de Spring Security y observar qué cadena se construye y qué filtros procesan la petición.

> **Pregunta de reflexión:** ¿Qué pasaría si añadieras el filtro JWT después del AuthorizationFilter?

Respuesta razonada: La autorización se evaluaría antes de que el Bearer token hubiera creado un Authentication. Una petición con token válido podría verse como anónima y ser rechazada. Por eso el filtro de autenticación debe ejecutarse antes de las decisiones de autorización.

Bloque 5 - Diagnóstico y buenas prácticas

T5.1 - Depurar el filtro

Para depurar, observa tres datos: si la cabecera se detecta, si el parser valida el token y si el SecurityContext queda poblado. Los logs de DEBUG pueden ayudar, pero nunca deben imprimir el token completo ni la clave.

Un test de integración que accede a una ruta protegida con token real proporciona una evidencia más fiable que inspeccionar logs manualmente.

Para depurar el filtro conviene registrar eventos de alto nivel —cabecera presente, token rechazado por tipo/expiración, principal creado— sin imprimir credenciales. Un traceId permite correlacionar respuesta y log.

Breakpoints o tests pueden inspeccionar el SecurityContext después del filtro. Si contiene principal y authorities correctas, el siguiente paso es investigar autorización, no seguir modificando parsing de JWT.

Para depurar conviene activar temporalmente logging.level.org.springframework.security=DEBUG y añadir logs propios que indiquen **qué etapa** se ha alcanzado sin imprimir el token completo. Podemos registrar que existe cabecera Bearer, que la firma fue válida, qué subject se obtuvo o que la autenticación se colocó en contexto.

También es útil comparar tres casos: ruta pública sin token, ruta protegida sin token y ruta protegida con token válido. Si todos fallan igual, probablemente la política de rutas o el filtro estén mal registrados; si sólo falla el token válido, se investigan firma, expiración, tipo y authorities. El diagnóstico sistemático evita modificar varias capas a la vez.

Los logs de diagnóstico pueden incorporarse al propio filtro sin imprimir el token completo en el diseño final:

```
private static final Logger log =
    LoggerFactory.getLogger(JwtAuthenticationFilter.class);

@Override
protected void doFilterInternal(HttpServletRequest request, ...) {
    String authHeader = request.getHeader("Authorization");
    log.debug("Procesando petición {} {}",
        request.getMethod(), request.getRequestURI());
    // ...
}

logging.level.org.springframework.security=DEBUG
```

T5.2 - Errores comunes

Los errores habituales son cortar la cadena cuando no hay token, devolver 401 directamente desde el filtro para cualquier problema, olvidar el prefijo Bearer, cargar el usuario antes de validar la firma o registrar el filtro después del punto donde se necesita la autenticación.

También es un error autorizar por URL dentro del filtro; esa lógica debe permanecer declarativa.

Fallos típicos: prefijo Bearer mal extraído, token expirado tratado como 500, contexto no establecido, filtro registrado dos veces, token revocado ignorado o exception tragada que deja la petición como anónima sin explicación. Cada síntoma necesita un test negativo.

El filtro debe fallar de forma controlada. “Capturar Exception y continuar siempre” puede convertir tokens inválidos en comportamientos ambiguos.

Los errores típicos incluyen olvidar `filterChain.doFilter`, no registrar el filtro, aceptar refresh como access, no capturar `JwtException`, construir authorities vacías, rechazar peticiones públicas sin token, omitir STATELESS o ignorar la revocación. Otro fallo frecuente es lanzar un 500 al parsear una cadena mal formada en vez de convertirla en autenticación inválida.

El filtro tampoco debe autorizar rutas por sí mismo ni consultar roles contra paths concretos. Si empieza a contener `if (uri.startsWith("/admin"))`, se mezcla autenticación con política y se duplican reglas. El resultado correcto es sencillo: autenticar si hay un access válido; continuar sin autenticar si no hay credencial; y dejar la decisión final a Spring Security.

Hay varios fallos específicos del ciclo de un filtro. Olvidar `filterChain.doFilter(request, response)` impide que la petición continúe y el cliente puede quedarse sin respuesta útil. Construir un `UsernamePasswordAuthenticationToken` sin authorities puede dejar una autenticación que no sirve para las decisiones posteriores de autorización; por eso se usa el constructor que incluye los permisos verificados del principal.

El `SecurityContextHolder` se apoya en un contexto asociado al hilo durante la petición. Spring Security se ocupa de inicializarlo y limpiarlo mediante sus filtros, pero nuestro filtro no debe conservar identidades en campos compartidos ni intentar gestionar ese ciclo por su cuenta. Del mismo modo, una excepción de parseo o firma no debe escapar como un 500 accidental: debe convertirse en una autenticación no válida y continuar hacia el mecanismo configurado de 401, o rechazarse de forma controlada según la política elegida.

Finalmente, el filtro JWT no sustituye al `UserDetailsService` en todos los flujos. El login con usuario y contraseña todavía necesita cargar credenciales persistentes; el filtro de access token reconstruye una identidad ya autenticada a partir de claims verificados. Mezclar ambas responsabilidades conduce a consultas innecesarias o a autenticar antes de comprobar la firma.

T5.3 - Buenas prácticas

El filtro debe ser determinista, no conservar estado por petición y delegar criptografía en `JwtService`. Las excepciones esperables se traducen de forma coherente y las rutas públicas siguen funcionando sin credencial.

La responsabilidad puede resumirse en una frase: **si existe un token válido, establecer una identidad; en cualquier otro caso, no fabricar permisos.**

Ejemplo.

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
```

```

        filterChain.doFilter(request, response);
        return;
    }
    String token = header.substring(7);
    String username = jwtService.extraerUsername(token);
    UserDetails user = userDetailsService.loadUserByUsername(username);
    Authentication auth = new UsernamePasswordAuthenticationToken(
        user, null, user.getAuthorities());
    SecurityContextHolder.getContext().setAuthentication(auth);
    filterChain.doFilter(request, response);

```

La implementación real añade validación, revocación y manejo cuidadoso de errores.

El filtro debe ser pequeño: extracción, validación, creación de contexto y delegación. La lógica de emisión, refresh, roles de negocio y respuestas complejas pertenece a otros componentes. Menos responsabilidades reducen superficie de error.

También debe ser idempotente respecto al contexto: si ya existe una autenticación válida por una razón prevista, no debería sobrescribirla sin una política explícita.

Ejemplo.

sin token: delegar
 token válido: autenticar y delegar
 token inválido: rechazo controlado

Pregunta de reflexión: ¿Por qué el filtro no debe hacer consultas a base de datos?

Respuesta razonada: Porque su responsabilidad es convertir una credencial ya verificable en una identidad de Spring Security, no cargar estado de negocio en cada petición. Mantenerlo pequeño y stateless reduce acoplamiento, latencia y puntos de fallo; las consultas se reservan para operaciones que realmente las necesitan.

Resumen de la teoría

- El filtro JWT convierte la cabecera Bearer en una identidad de Spring Security.
- OncePerRequestFilter garantiza una ejecución controlada por petición.
- El filtro valida el token, construye el principal y deposita el Authentication en el SecurityContextHolder.
- La ausencia de token no debe inventar una identidad; la autorización posterior decide si la ruta exige autenticación.
- En el estado JWT final HTTP Basic desaparece y la sesión se configura como STATELESS.

Punto 6.8 - Protección de endpoints con JWT

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Configurar el SecurityFilterChain completo para una API con JWT.
2. Definir qué endpoints son públicos y cuáles privados.
3. Aplicar autorización por roles usando los claims del token.
4. Combinar reglas de URL con anotaciones de método.
5. Implementar el flujo completo: login → petición → refresh → logout.
6. Manejar la expiración del token en el cliente.
7. Diagnosticar y resolver los errores más comunes al proteger endpoints.

Bloque 1 - La configuración completa

T1.1 - El SecurityFilterChain con JWT

El estado final reúne todas las decisiones acumuladas: CSRF desactivado para la API Bearer, CORS habilitado, sesiones STATELESS, allowlist pública explícita, reglas por rol, manejadores de 401/403 y filtro JWT antes del filtro de usuario/contraseña.

La configuración ya no incluye HTTP Basic. Si una petición protegida no aporta un Bearer token válido, no existe un mecanismo alternativo silencioso que la autentique.

La cadena final combina STATELESS, CSRF acorde a API sin cookie de sesión, CORS existente, allowlist explícita, manejadores 401/403 y filtro JWT. El objetivo es que todas las decisiones temporales de 6.1-6.7 hayan evolucionado a una configuración única y legible.

Una revisión final busca anti-patrones: HTTP Basic aún activo, permitAll() global, wildcard /api/v1/auth/**, sesión IF_REQUIRED o segundo CORS contradictorio. El proyecto sólo se cierra si esos estados no sobreviven.

La cadena final combina todas las decisiones construidas durante el módulo: CSRF desactivado para la API stateless, CORS preservado, SessionCreationPolicy.STATELESS, rutas públicas explícitas, áreas ADMIN/GESTOR, manejadores JSON 401/403 y el filtro JWT antes del filtro de username/password. La última regla sigue siendo anyRequest().authenticated().

Lo importante es leer la configuración como una política completa, no como llamadas aisladas. Si se elimina STATELESS, puede aparecer una sesión inesperada; si se mueve anyRequest() antes de los matchers concretos, las reglas siguientes dejan de alcanzarse; si se omite el filtro, Bearer no crea identidad. Cada línea existe para una propiedad comprobable del sistema.

T1.2 - Por qué personalizar los manejadores de error

Los fallos de seguridad ocurren antes del controlador, de modo que un @RestControllerAdvice convencional no siempre puede construir la respuesta. AuthenticationEntryPoint atiende fallos de autenticación y AccessDeniedHandler atiende denegaciones de autorización.

Personalizarlos permite conservar el contrato ErrorResponse de M5, incluido traceId, en lugar de devolver formatos distintos según la capa que produjo el error.

Los fallos de seguridad ocurren antes del controlador, así que @ControllerAdvice no siempre controla 401/403 de la cadena. AuthenticationEntryPoint y AccessDeniedHandler son las fronteras correctas para producir el mismo ErrorResponse y traceId que el resto de la API.

La coherencia de errores es funcionalidad: clientes y observabilidad pueden interpretar respuestas uniformes sin adivinar si el fallo nació en MVC o en seguridad.

Los fallos de autenticación y autorización ocurren dentro de la infraestructura de Spring Security, a menudo **antes** de entrar en un controlador. Por eso un @RestControllerAdvice pensado para excepciones MVC no es suficiente para dar formato a todos los 401/403.

AuthenticationEntryPoint controla el caso 401: no existe autenticación válida para un recurso que la requiere. AccessDeniedHandler controla el 403: existe identidad, pero no posee permiso suficiente. Personalizarlos permite mantener el mismo contrato JSON de errores de M5, con timestamp, status, código, mensaje, path y traceId, en lugar de respuestas HTML o formatos inconsistentes.

T1.3 - Estructura del JwtAuthenticationEntryPoint

El entry point escribe una respuesta 401 con Content-Type JSON y el mismo esquema de error usado por el resto de la API. No revela si el username existe ni detalles de la firma.

La escritura se centraliza en SecurityErrorWriter para que 401 y 403 compartan estructura y no dupliquen serialización.

JwtAuthenticationEntryPoint responde cuando falta o falla autenticación en un recurso que la exige. Debe establecer status 401, content type JSON y cuerpo seguro, reutilizando el modelo común. No revela si un usuario existe ni el detalle criptográfico del token.

El traceId se devuelve y se registra para diagnóstico. El test debe comprobar status y estructura, no sólo que “hay algún JSON”.

Ejemplo.

```
.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

JwtAuthenticationEntryPoint implementa AuthenticationEntryPoint y su método commence(...) recibe request, response y la excepción de autenticación. Puede construir directamente una respuesta JSON o delegar en un escritor común que reutilice ErrorResponse.

La implementación debe establecer status 401, Content-Type: application/json y UTF-8 antes de escribir el cuerpo. No debe lanzar otra excepción que convierta el rechazo en 500. La misma estrategia se aplica al JwtAccessDeniedHandler para 403. Compartir una utilidad de escritura reduce duplicación y asegura que ambos errores de seguridad respeten el contrato transversal del proyecto.

El AuthenticationEntryPoint controla el caso 401 antes de que el controlador llegue a ejecutarse:

```

@Component
public class JwtAuthenticationEntryPoint
    implements AuthenticationEntryPoint {
    @Override
    public void commence(
        HttpServletRequest request,
        HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");

        String json = ""
            {
                "timestamp": "%s",
                "status": 401,
                "codigo": "NO_AUTENTICADO",
                "mensaje": "Se requiere autenticación para acceder a este recurso",
                "path": "%s"
            }
            """.formatted(
                Instant.now().toString(),
                request.getRequestURI());
        response.getWriter().write(json);
    }
}

```

Pregunta de reflexión: ¿Qué propiedades comprobables demuestran que la configuración final es realmente stateless y que HTTP Basic ya no participa?

Respuesta razonada: La cadena final debe desactivar CSRF para esta API stateless, configurar STATELESS, registrar el filtro JWT, establecer handlers de 401/403 y ordenar las reglas desde las más específicas hasta `anyRequest()`.

Bloque 2 - Endpoints públicos y privados

T2.1 - Clasificación de endpoints

Antes de escribir matchers conviene clasificar rutas: públicas, autenticadas, de gestión y administrativas. La clasificación evita reglas ad hoc que se acumulan sin una política general.

En M6 sólo son públicas las operaciones de registro/login/refresh declaradas, la zona `/api/v1/public/**`, H2 en desarrollo y la documentación OpenAPI/Swagger. El resto requiere identidad.

Clasificar endpoints obliga a declarar qué es verdaderamente público: registro, login, refresh, documentación necesaria y recursos explícitos. El resto cae bajo autenticación por defecto. Esta lista es parte del modelo de amenazas del sistema.

Una ruta pública no debe depender del orden casual de matchers. Debe existir una entrada concreta y un test anónimo que demuestre el comportamiento previsto.

Mantener una lista pública explícita también facilita revisión de seguridad. En una auditoría se puede enumerar cada endpoint anónimo y justificar por qué lo es. Un wildcard amplía la superficie de forma implícita: cualquier ruta futura bajo ese prefijo heredaría exposición sin que el autor de la ruta tenga que tomar una decisión consciente.

Conviene clasificar las rutas por intención: públicas, autenticadas, administrativas y de gestor. Registro, login y refresh son públicos porque permiten obtener o renovar credenciales; logout no tiene por qué abrirse de forma indiscriminada. Swagger/OpenAPI y recursos de desarrollo pueden exponerse según perfil; el resto exige autenticación.

Una regla amplia como `/api/v1/auth/**` puede parecer cómoda, pero también haría público cualquier endpoint futuro bajo ese prefijo. La edición actual prefiere matchers explícitos para `POST /registro`, `/login` y `/refresh`. Esta precisión convierte la configuración en una lista de decisiones revisables en vez de una excepción demasiado grande.

T2.2 - Orden de las reglas

Spring Security evalúa matchers en orden. Una regla amplia como `/**.permitAll()` colocada antes convertiría en irrelevantes las restricciones posteriores.

Por eso se escriben primero las excepciones públicas y las zonas con roles específicos, y se termina con `anyRequest().authenticated()`. El último matcher actúa como red de seguridad.

Los matchers se evalúan en orden; una regla amplia colocada antes puede eclipsar otra específica. La política se escribe de lo particular a lo general y termina en `anyRequest().authenticated()`.

Los tests deben incluir al menos un caso por frontera relevante. Así, reordenar accidentalmente matchers rompe un test antes de convertirse en exposición de seguridad.

Una forma práctica de comprobar el orden consiste en escoger dos rutas que coincidan con patrones diferentes y ejecutar la misma identidad contra ambas. Si una regla general se ha colocado demasiado pronto, el resultado de una de ellas cambiará. El test de regresión debe conservar esa pareja de casos: demuestra que la prioridad declarada sigue siendo la prioridad efectiva después de futuras ediciones de `SecurityConfig`.

Spring evalúa los matchers en orden, de modo que primero deben aparecer las excepciones más específicas y al final la regla general. Un esquema legible es: públicos → ADMIN → GESTOR/ADMIN → cualquier otra petición autenticada.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST, "/api/v1/auth/login").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
```

Si `anyRequest()` se declarase primero, ya habría una decisión terminal para todas las rutas y las reglas posteriores no cumplirían la intención del diseño.

T2.3 - Endpoints con permisos específicos

Las zonas `/api/v1/admin/**` exigen ADMIN y `/api/v1/gestor/**` admiten GESTOR o ADMIN. Otras reglas más cercanas al negocio pueden añadirse con `@PreAuthorize`.

Esta división muestra dos niveles: perímetro HTTP y autorización del caso de uso. Ambos se prueban de forma independiente.

Además de autenticación general, algunas rutas exigen GESTOR o ADMIN. La regla puede estar en URL, método o ambas según el caso. Lo importante es que un token válido con rol insuficiente produzca 403, no 401.

Esta distinción confirma que el filtro autentica correctamente incluso cuando la autorización rechaza. Es una observación muy útil al depurar.

Ejemplo.

públicos explícitos -> reglas de rol -> `anyRequest().authenticated()`

Las áreas con permisos específicos expresan políticas diferentes. `/api/v1/admin/**` requiere ADMIN; `/api/v1/gestor/**` admite GESTOR o ADMIN; un perfil normal sólo necesita autenticación salvo que un método añada una condición más fina.

Esta política debe corresponderse con las authorities que salen del JWT. Si el token contiene `ROLE_GESTOR`, `hasRole("GESTOR")` funcionará; si el filtro elimina el prefijo o construye authorities distintas, aparecerá 403 aunque el token sea válido. Por eso la protección de endpoints no se prueba sólo con “token sí/no”, sino con tokens de usuarios que poseen roles diferentes. > **Pregunta de reflexión:** ¿Por qué conviene enumerar de forma explícita las rutas públicas en vez de permitir `/api/v1/auth/**` completo?

Respuesta razonada: Una allowlist debe ser corta y explícita: registro, login/refresh y recursos realmente públicos/documentación de desarrollo. Un patrón demasiado amplio como `/api/v1/auth/**` puede abrir accidentalmente operaciones que deberían requerir token.

Bloque 3 - Autorización con JWT

T3.1 - Los roles vienen del token

En una implementación posible, los roles podrían viajar como claims. En M6, sin embargo, el token identifica al usuario y el filtro reconstruye el principal desde base de datos, de manera que las authorities actuales se obtienen del sistema.

Esto evita que un token conserve durante demasiado tiempo un rol retirado en la base de datos. La contrapartida es una consulta adicional para reconstruir identidad.

Las authorities utilizadas por autorización deben proceder de una fuente autenticada y verificable.

Si viajan como claims, sólo son confiables después de validar firma y tipo; si se recargan de base, deben transformarse con la misma convención de roles.

Nunca se leen roles de una query parameter o cabecera auxiliar enviada por el cliente. La integridad del token o la base de datos es lo que sostiene la decisión.

Si los roles cambian mientras un access sigue vigente, el efecto depende de si la aplicación confía sólo en claims o recarga usuario. Esa decisión debe conocerse porque determina la latencia de revocación de privilegios. El módulo documenta el comportamiento que realmente implementa y no atribuye al JWT una actualización instantánea que no exista.

En el flujo final los roles se leen de claims **sólo después** de verificar el access token. JwtService convierte cada string de la lista roles en SimpleGrantedAuthority, y esas authorities se insertan en el UsuarioPrincipal y en el Authentication.

Esto evita consultar la base en cada petición y conserva el carácter stateless, pero implica que los roles del token representan una fotografía del momento de emisión. Si un administrador cambia permisos en base de datos, un access token antiguo puede conservar sus claims hasta caducar o revocarse. Esa es otra razón para mantener access tokens cortos y no incluir en ellos información innecesariamente mutable.

T3.2 - hasRole y hasAuthority con JWT

Una vez creado el Authentication, la autorización no necesita saber que la identidad llegó por JWT. hasRole, hasAuthority y @PreAuthorize trabajan exactamente con las authorities del contexto.

Ésta es una de las ventajas de integrarse correctamente con Spring Security: el mecanismo de autenticación y la política de autorización quedan desacoplados.

Con JWT, hasRole y hasAuthority funcionan igual que con Basic una vez construido el SecurityContext. Esta es una ventaja arquitectónica: autorización no necesita saber cómo se autenticó la identidad.

Si cambiar de Basic a Bearer obliga a reescribir todas las reglas, probablemente las capas estaban demasiado acopladas. El módulo demuestra precisamente la independencia entre mecanismo y política.

La forma de la regla no cambia por usar JWT porque AuthorizationFilter trabaja sobre Authentication, no sobre el token original. Este desacoplamiento es una prueba arquitectónica: si mañana se sustituyera JWT por otro autenticador que construyera el mismo principal y authorities, las reglas de rol deberían seguir produciendo los mismos 200/403.

Con JWT no cambia la semántica de hasRole y hasAuthority: lo que cambia es el origen de las authorities. Antes procedían del UserDetailsService durante HTTP Basic; ahora las proporciona el principal reconstruido desde claims firmados.

hasRole("ADMIN") sigue buscando ROLE_ADMIN. hasAuthority("ROLE_ADMIN") busca exactamente ese texto. La consistencia de prefijos es por tanto transversal a todo el módulo: almacenamiento de roles, emisión del token, reconstrucción de authorities y reglas deben usar la misma convención. Un 403 con token válido suele ser una señal para revisar esa cadena de transformación.

Con UsuarioPrincipal como principal, las reglas pueden combinar rol e identificador del sujeto:

```
@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
public UsuarioResponseDTO consultarUsuario(@PathVariable Long id) {
    // ...
}
```

T3.3 - Combinar filtro, URL y método

El filtro valida credenciales, la cadena HTTP controla el perímetro y la seguridad por método protege casos de uso concretos. Si cada capa respeta esa responsabilidad, el diseño sigue siendo comprensible incluso cuando crece.

Los tests finales deben demostrar que estas capas cooperan: token válido sin rol produce 403, token válido con rol produce éxito y token inválido no crea autenticación.

Filtro, reglas URL y anotaciones de método forman una cadena de defensa. El filtro establece identidad; URL filtra superficie; método protege operaciones. Cada capa aporta una garantía diferente y todas deben coincidir en tests.

Una petición puede superar el filtro y aun así ser rechazada. Eso no es duplicación inútil: es la consecuencia de separar autenticación de autorización.

Ejemplo.

token verificado -> authorities -> hasRole/hasAuthority -> 200 o 403

Pregunta de reflexión: ¿Qué combinación de token y rol debe producir 401, 403 y 200 para demostrar que autenticación y autorización JWT están separadas?

Respuesta razonada: Las authorities del token deben reconstruirse sólo después de verificar firma, expiración y tipo. A partir de ahí hasRole/@PreAuthorize trabajan igual que con cualquier otra identidad de Spring Security.

Bloque 4 - El flujo completo**T4.1 - Diagrama del flujo**

Puede leerse el flujo de izquierda a derecha: cliente -> filtro JWT -> SecurityContext -> matchers HTTP -> proxy de método -> controlador/servicio. Una respuesta de seguridad puede producirse en varias etapas, pero siempre antes de ejecutar una operación no autorizada.

Esta visión evita depurar sólo el controlador cuando el rechazo ocurrió mucho antes.

El flujo final puede narrarse de extremo a extremo: cliente envía Bearer, filtro valida, contexto se llena, matcher autoriza, método puede reforzar, controlador/servicio ejecuta y respuesta vuelve. En fallo, entry point o denied handler generan contrato JSON.

Ser capaz de dibujar este flujo es una prueba de comprensión más valiosa que memorizar anotaciones. Cada componente tiene una entrada, una salida y una responsabilidad comprobable.

El flujo completo puede leerse de extremo a extremo:

```
credenciales -> /login -> AuthenticationManager -> access/refresh
access -> Authorization: Bearer -> JwtAuthenticationFilter
      -> JwtService verifica -> UsuarioPrincipal/authorities
      -> SecurityContext -> reglas URL -> @PreAuthorize
      -> controlador -> respuesta
```

Si falta token y la ruta es privada, el AuthenticationEntryPoint produce 401. Si hay identidad pero el rol no basta, el AccessDeniedHandler produce 403. Si todo es correcto, la petición alcanza MVC. Este diagrama ayuda a localizar fallos: cada síntoma corresponde a una frontera distinta y no tiene sentido modificar el controlador cuando la petición nunca superó la cadena de seguridad.

T4.2 - Manejo de la expiración en el cliente

El servidor debe rechazar un access token expirado. El cliente, al recibir 401 por expiración, puede usar el refresh token para renovar y repetir la operación. La lógica de renovación pertenece al cliente y al endpoint de refresh, no al filtro de cada recurso.

Un bucle infinito de refresh es un error de cliente; si el refresh también falla, la sesión lógica del usuario debe terminar.

El cliente debe tratar la expiración como estado normal, no como excepción catastrófica. Ante access expirado puede intentar refresh según política; si refresh falla o expiró, debe exigir autenticación de nuevo.

No se implementan bucles infinitos de refresh. Un 401 persistente después de renovar debe terminar el flujo y limpiar credenciales del cliente.

El cliente tampoco debe enviar el refresh como Bearer a recursos ordinarios. Separar usos reduce exposición y permite que el servidor aplique reglas distintas. En una aplicación real, el lugar de almacenamiento del refresh merece más protección que el access por su mayor duración y capacidad de obtener nuevas credenciales.

El cliente puede leer exp para anticipar la expiración, pero el servidor sigue siendo la autoridad final. Cuando un access caduca, el cliente usa un refresh válido para obtener un nuevo par y repite la petición original. No debe intentar prolongar localmente un token ni asumir que una fecha futura garantiza validez si el token fue revocado.

Una aplicación cliente necesita evitar bucles de refresh: si /refresh también falla, debe terminar la sesión lógica y solicitar credenciales de nuevo. También debe sincronizar peticiones concurrentes para no canjear varias veces el mismo refresh de un solo uso. Estas consideraciones muestran que el ciclo de vida JWT afecta tanto al servidor como al consumidor de la API.

T4.3 - Logout con JWT

Como no existe sesión HTTP que destruir, logout significa impedir que determinadas credenciales sigan siendo aceptadas. M6 mantiene una revocación didáctica en memoria y consume refresh tokens de un solo uso.

En sistemas distribuidos esta estrategia debería externalizarse o sustituirse por tokens muy cortos y un mecanismo de revocación compartido. El módulo documenta claramente el alcance de la implementación.

En JWT stateless, “logout” no destruye una sesión HTTP. Puede consistir en revocar el access/refresh relevante y hacer que futuras peticiones sean rechazadas. El proyecto usa revocación en memoria como mecanismo educativo, dejando claro que una arquitectura distribuida necesitaría persistencia compartida o una estrategia diferente.

El test esencial es temporal: token funciona antes de logout y deja de funcionar después. Sin esa prueba, un endpoint /logout que sólo responde 204 podría ser cosmético.

Ejemplo.

request -> filtro JWT -> contexto -> URL -> método -> controlador -> response

Conviene distinguir tres estrategias de logout porque un JWT emitido sigue siendo criptográficamente válido hasta su expiración. La primera es una **lista negra de tokens**: al hacer logout se registra el token invalidado y el filtro consulta esa lista antes de aceptarlo. Su ventaja es que permite revocación inmediata; sus costes son mantener ese estado, consultarlo en cada petición y eliminar periódicamente las entradas que ya han expirado.

La segunda es la **rotación de refresh tokens**. Cada refresh es de un solo uso: al utilizarlo se emite otro y el anterior queda invalidado. En logout se invalida el refresh actual. El access token que ya estaba emitido puede seguir funcionando hasta su exp, pero ya no podrá renovarse. Esta estrategia reduce el tamaño de una blacklist de access tokens, a cambio de aceptar una ventana limitada de validez.

La tercera es usar **access tokens de corta duración**. Si viven pocos minutos, el riesgo residual después del logout se acota aunque no se revoquen inmediatamente. En la práctica pueden combinarse tokens de acceso cortos con refresh rotativo y reservar una blacklist compartida para requisitos de revocación inmediata. La elección depende de cuánto estado se acepta mantener y de cuán rápido debe surtir efecto el cierre de sesión.

Pregunta de reflexión: ¿Cómo demostrarías de extremo a extremo que login, refresh, uso de access y logout forman un ciclo coherente y revocable?

Respuesta razonada: El flujo correcto enlaza autenticación inicial, emisión de tokens, uso del access token, renovación mediante refresh y revocación/logout. Cada etapa tiene un error probable distinto que debe estar cubierto.

Bloque 5 - Diagnóstico y buenas prácticas

T5.1 - Diagnóstico de problemas comunes

Ante un 401 inesperado, comprueba cabecera Bearer, firma, expiración, revocación y creación del SecurityContext. Ante un 403, la autenticación probablemente existe y el siguiente paso es revisar authorities y matchers.

Separar ambos diagnósticos ahorra tiempo. Un 403 no se corrige regenerando el mismo token si el usuario sigue sin rol.

Diagnosticar requiere localizar la fase: 401 suele apuntar a autenticación/token; 403 a authorities/reglas; fallo CORS a preflight/configuración; 500 a excepción no traducida. El traceId y logs de seguridad permiten seguir la petición sin exponer secretos.

Un procedimiento disciplinado evita cambiar varias reglas a la vez. Se reproduce, se observa contexto y matcher, se corrige la causa mínima y se repite la prueba.

Antes de activar DEBUG global conviene reproducir con una única petición y conservar su traceId. Así se puede comparar log y respuesta sin inundar el entorno ni exponer cabeceras. El diagnóstico termina restaurando el nivel de logging: una configuración temporal de trazas sensibles no forma parte del estado final del proyecto.

El diagnóstico empieza separando autenticación y autorización. Un 401 invita a revisar ausencia/formato Bearer, firma, expiración, tipo, revocación y construcción del contexto. Un 403 indica que ya existe identidad y conviene revisar roles, prefijos, orden de matchers o expresiones de

método.

Los logs DEBUG de Spring Security muestran la cadena seleccionada y decisiones de autorización. Para problemas CORS se inspecciona primero la petición OPTIONS; para refresh se comprueba el tipo y el jti; para logout se reutiliza el mismo access después del 204. Cada diagnóstico debe reproducirse con una petición mínima y un resultado esperado claro antes de cambiar código.

T5.2 - Errores comunes

Es fácil dejar httpBasic, usar permitAll demasiado amplio, crear sesión accidentalmente, tratar 403 como 401 o permitir cualquier ruta bajo /auth/**. También puede olvidarse CORS al reconfigurar HttpSecurity.

El estado final evita estos atajos y conserva la política CORS heredada mediante cors(Customizer.withDefault)

Errores finales críticos: permitir /auth/**, dejar Basic activo, usar sesión, duplicar CORS, devolver HTML en 401/403, ignorar revocación, no distinguir refresh y access, o mantener un permitAll() pedagógico. Son regresiones que los tests de seguridad deben detectar.

La configuración final se compara con el estado esperado y con tests de seguridad; compilar no es evidencia suficiente.

Como ejercicio de diagnóstico, cambia una sola condición cada vez y conserva una petición de control. Si al abrir una ruta desaparece el 403 pero también queda accesible para anónimos, la corrección ha roto otra propiedad. La meta no es obtener 200, sino obtener el status correcto para cada sujeto. Esa disciplina convierte los errores comunes en una herramienta de razonamiento y evita “arreglos” que sólo ocultan la causa.

Entre los errores más comunes están abrir /auth/** completo, olvidar STATELESS, no integrar CORS en seguridad, registrar el filtro en posición incorrecta, devolver 403 cuando corresponde 401, construir respuestas de error incompatibles con M5 o dejar HTTP Basic activo accidentalmente. También es fácil permitir un refresh token como credencial de recurso si no se valida su tipo.

Otro error es pensar que permitAll evita ejecutar todos los filtros: el filtro JWT puede seguir viendo la petición, por eso debe ser tolerante cuando no existe Bearer. Finalmente, una revocación sólo local no debe presentarse como solución distribuida; su alcance se documenta y se prueba como parte didáctica del módulo.

T5.3 - Buenas prácticas

Una configuración segura debe ser explícita, mínima y testeable. Las rutas públicas se enumeran, las credenciales de producción se externalizan, la sesión está desactivada y los errores usan un formato uniforme.

Además, cualquier simplificación didáctica —como la revocación en memoria— debe quedar identificada como tal y no presentarse como arquitectura distribuida de producción.

Ejemplo.

http

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST,
        "/api/v1/auth/registro", "/api/v1/auth/login", "/api/v1/auth/refresh").permitAll()
    .requestMatchers("/api/v1/public/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(deniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

Buenas prácticas: deny by default, allowlist pequeña, tokens breves, secretos externos, errores uniformes, revocación explícita, pruebas negativas y logs sin credenciales. La configuración debe poder leerse como una política, no como una acumulación histórica de parches.

También se documentan las limitaciones educativas —revocación in-memory, por ejemplo— para que el alumno no confunda una demostración con una solución distribuida completa.

Ejemplo.

401: identidad no aceptada
403: identidad válida sin permiso
traceId: correlación segura

Pregunta de reflexión: ¿Qué prueba conjunta de CORS, 401/403 y OpenAPI garantiza que endurecer seguridad no ha roto contratos transversales heredados?

Respuesta razonada: El diagnóstico debe separar 401 por identidad inválida/ausente de 403 por permisos, revisar orden de filtros/reglas y comprobar CORS. Los handlers deben devolver el mismo formato JSON de error del resto de M5/M6.

Resumen de la teoría

- La configuración completa integra stateless, filtro JWT, reglas por ruta y manejadores 401/403.
- Las rutas públicas se declaran explícitamente y las reglas específicas se colocan antes de `anyRequest()`.
- Los roles incluidos en el token alimentan la autorización sin consultar la base de datos en cada petición.
- El flujo completo comprende registro, login, petición autenticada, refresh y logout.
- Los errores de seguridad deben respetar el mismo contrato JSON que el resto de la API.

Punto 6.9 - Testing de seguridad**Objetivos de aprendizaje**

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar por qué la seguridad debe testearse como cualquier otra funcionalidad.
2. Usar `@WithMockUser` para simular usuarios autenticados en tests.
3. Usar `@WithUserDetails` para cargar usuarios desde un `UserDetailsService`.
4. Aplicar `SecurityMockMvcRequestPostProcessors` para simular autenticación por petición.
5. Testear los escenarios de autenticación, autorización y tokens JWT.
6. Verificar códigos de estado y estructura de respuestas de error de seguridad.
7. Diagnosticar y resolver los errores más comunes al testear seguridad.

Bloque 1 - Por qué testear la seguridad**T1.1 - La seguridad es funcionalidad**

Una política de acceso produce resultados comprobables y, por tanto, debe probarse como cualquier otra funcionalidad. El hecho de que una anotación esté presente no demuestra que esté activa ni que el matcher tenga el orden correcto.

Los tests convierten la política en una especificación ejecutable: quién puede llamar, con qué credencial y qué status debe recibir.

La seguridad es comportamiento funcional porque determina qué respuestas recibe cada actor. Una aplicación que devuelve datos a un usuario no autorizado está funcionalmente rota aunque todas las operaciones de negocio correctas pasen tests.

Por eso los tests de seguridad forman parte de la definición de terminado. No se relegan a una auditoría opcional posterior: acompañan cada política y protegen contra regresiones.

Tratar seguridad como funcionalidad implica incluirla en regresión cada vez que se toca el proyecto. Una actualización de CORS, un nuevo endpoint o una refactorización de controlador puede cambiar la superficie protegida sin modificar clases security. La suite final actúa como red de seguridad transversal sobre esos cambios.

La seguridad no es una configuración decorativa: 401, 403, permisos, expiración y revocación forman parte del comportamiento funcional de la API. Un cambio aparentemente inocente en un matcher puede abrir una ruta administrativa o bloquear a todos los usuarios. Por eso esas propiedades necesitan pruebas automatizadas igual que un cálculo de negocio.

Los tests convierten la política en un contrato ejecutable. "Sólo ADMIN puede listar usuarios" se traduce al menos en tres escenarios: anónimo rechazado, USER autenticado rechazado y ADMIN

aceptado. Si sólo se prueba el camino feliz, una apertura accidental puede pasar desapercibida.

T1.2 - Qué se puede testear

Podemos probar autenticación, autorización, formato de errores, login, refresh, logout, filtro JWT, rutas públicas y comportamiento de usuarios concretos. También podemos comprobar que un token manipulado o expirado no crea identidad.

La cobertura útil no consiste en contar anotaciones, sino en recorrer caminos de éxito y rechazo representativos.

Se pueden probar autenticación, autorización, contenido del SecurityContext, login, expiración, refresh, revocación, CORS y formato de errores. No todo requiere un servidor real: MockMvc cubre mucho; integración completa verifica la cadena real.

La estrategia elige el nivel mínimo que demuestre la propiedad. Un test unitario no puede acreditar que el filtro esté registrado en el orden correcto; ahí hace falta contexto de integración.

También es posible probar propiedades que atraviesan varias capas, como “un refresh rotado no vuelve a aceptarse”. Esas propiedades necesitan integración y estado secuencial. En cambio, una función pura de extracción de claims puede probarse de manera aislada. Elegir nivel por propiedad mantiene la suite rápida sin sacrificar evidencia donde realmente hace falta.

Se puede probar el servicio JWT de forma unitaria —firma, expiración, claims, tipos—; controladores y reglas con MockMvc; filtros con peticiones Bearer reales; login con credenciales válidas e inválidas; refresh y replay; logout y revocación; y flujos completos de integración.

También se prueban aspectos heredados que la seguridad podría romper: endpoint público, OpenAPI, CORS y respuestas de error. La suite debe cubrir tanto **permitir** como **denegar**. El objetivo no es maximizar el número de métodos de test, sino tener una matriz que represente las fronteras reales de confianza del sistema.

T1.3 - Herramientas de testing de seguridad

Spring Security Test se integra con MockMvc y el contexto de pruebas. Ofrece anotaciones de usuario simulado, post-processors para construir autenticaciones y matchers específicos de seguridad.

Para los flujos JWT reales, complementaremos esos helpers con peticiones al login y uso de los tokens generados por la aplicación.

Spring Security Test aporta anotaciones y request post-processors que permiten construir identidades de prueba sin inventar infraestructura propia. MockMvc permite afirmar status, cabeceras y JSON. JUnit organiza escenarios y fixtures.

Las herramientas facilitan el test, pero no sustituyen el diseño de casos. La matriz debe incluir rechazo, no sólo accesos permitidos.

Ejemplo.

propiedad de seguridad = sujeto + petición + resultado esperado

Spring Security Test complementa JUnit, Mockito y MockMvc con utilidades específicas: @WithMockUser, @WithUserDetails, request post-processors como user() o httpBasic(), y matchers de seguridad. Cada herramienta sustituye una parte distinta del flujo y por eso debe elegirse según la propiedad que se quiera demostrar.

Un @SpringBootTest puede ejecutar el sistema casi completo, mientras @WebMvcTest crea un slice MVC más rápido. El slice es adecuado para contratos del controlador, pero hay que proporcionar los beans de seguridad que el contexto necesite. En M6 usamos @MockitoBean para dependencias del filtro cuando el test no pretende cargar su implementación real. > **Pregunta de reflexión:** ¿Por qué una política de seguridad debe tratarse como funcionalidad regresable y no como configuración que basta con inspeccionar?

Respuesta razonada: Una regla de seguridad es comportamiento funcional: si cambia una URL o una anotación puede abrir datos sin producir errores visibles. Por eso debe existir un test que falle cuando una identidad no autorizada obtiene acceso.

Bloque 2 - @WithMockUser y @WithUserDetails

T2.1 - @WithMockUser

@WithMockUser crea una identidad sintética para el test. Es ideal para probar políticas de roles sin depender de la base de datos. Podemos declarar username, roles o authorities y verificar rápidamente 200/403.

No demuestra que UsuarioDetailsService cargue correctamente un usuario real. Por eso se combina con otros tipos de prueba.

@WithMockUser crea un usuario de prueba con roles/authorities sin consultar el UserDetailsService. Es ideal para aislar autorización de controlador/método. Si un test pretende verificar carga real de usuarios, esta anotación sería demasiado artificial.

Con roles="ADMIN" Spring crea ROLE_ADMIN; con authorities se especifica el texto exacto. La misma convención que en producción debe reflejarse en tests.

Un buen uso de @WithMockUser declara explícitamente username y roles cuando son relevantes para la política. Usar siempre el usuario por defecto puede ocultar que una expresión depende del nombre o de otra authority. También conviene evitarlo en tests que pretenden verificar el mapping Rol -> GrantedAuthority; ahí se estaría sustituyendo justamente la pieza que se quiere comprobar.

@WithMockUser crea directamente una identidad sintética en el SecurityContext. Permite indicar username, roles o authorities y resulta ideal para comprobar reglas de autorización sin depender de base de datos, password encoder ni login.

```
@Test
@WithMockUser(username="ana", roles="USER")
void adminDebeDevolver403ParaUser() { ... }
```

La anotación no verifica que ana exista, ni que su contraseña sea correcta, ni que JwtService pueda emitir un token. Precisamente por eso el test es rápido y focalizado. Se usa para aislar la política; otras pruebas cubren la autenticación real.

T2.2 - @WithUserDetails

@WithUserDetails pide al UserDetailsService del contexto que cargue una identidad concreta. Resulta útil cuando queremos verificar la integración con usuarios persistidos o con un servicio real de identidades.

El usuario debe existir antes de que se prepare el contexto de seguridad del test, lo que exige cuidar la inicialización de datos.

@WithUserDetails pide a un UserDetailsService real del contexto que cargue el usuario indicado. Es útil cuando queremos comprobar que los usuarios persistentes se transforman en el principal esperado.

Requiere fixtures de datos coherentes. Si el usuario no existe, el test falla antes de la petición, lo que ayuda a detectar una preparación incorrecta pero también exige entender el ciclo del contexto.

@WithUserDetails es especialmente útil para verificar que una cuenta desactivada o con roles persistidos se interpreta como esperamos. Sin embargo, no sustituye un login completo: carga directamente el usuario para preparar el contexto de test. La documentación del caso debe distinguir claramente esa preparación del camino de autenticación que se prueba en integración.

@WithUserDetails pide a Spring que cargue el usuario a través del UserDetailsService configurado en el contexto. Así añade una capa de integración respecto a @WithMockUser: la identidad debe existir y transformarse correctamente en UserDetails y authorities.

Es útil para comprobar que roles persistidos como ADMIN terminan convertidos en ROLE_ADMIN. A cambio, el test necesita datos de usuario preparados de forma determinista. No es la herramienta adecuada si lo único que queremos probar es una regla simple, porque introduciríamos base de datos y configuración sin necesidad.

T2.3 - @WithAnonymousUser

@WithAnonymousUser fuerza un escenario anónimo incluso si la clase de test define otra identidad por defecto. Permite documentar que una ruta pública funciona o que una ruta protegida rechaza explícitamente el acceso sin credenciales.

Los casos anónimos son esenciales para detectar permitAll accidentales.

@WithAnonymousUser fuerza explícitamente un contexto anónimo y hace más legible un test de rechazo, sobre todo cuando una clase de test define un usuario por defecto. Expresar el sujeto evita depender de configuración implícita.

Un caso anónimo debe diferenciarse de “usuario autenticado sin rol”; ambos pueden ser rechazados, pero con 401 y 403 respectivamente según la frontera probada.

Ejemplo.

```
@WithMockUser(roles = "ADMIN")
```

@WithAnonymousUser fuerza explícitamente el contexto anónimo. Es especialmente útil cuando una clase o método de test tiene una identidad por defecto y queremos verificar una excepción anónima sin depender de cómo se heredan anotaciones.

Un test anónimo prueba una propiedad distinta de un usuario sin permisos. En una ruta protegida, el primero debe conducir normalmente a 401 y el segundo a 403. Mantener esos escenarios separados ayuda a detectar configuraciones donde un permitAll accidental convierte un rechazo esperado en 200 o donde un handler incorrecto confunde ambos estados. > **Pregunta de reflexión:** ¿Qué demuestra @WithMockUser y qué no puede demostrar sobre el login, la base de datos o la firma JWT?

Respuesta razonada: @WithMockUser crea una identidad sintética sin consultar la base; @WithUserDetails carga un usuario mediante el UserDetailsService. El segundo es apropiado cuando también queremos verificar la integración con la fuente real de usuarios.

Bloque 3 - SecurityMockMvcRequestPostProcessors

T3.1 - Post-processors para peticiones

MockMvc permite añadir autenticación directamente a una petición mediante post-processors. Esto resulta más flexible que una anotación de método cuando diferentes requests del mismo test necesitan identidades distintas.

También facilita construir matrices de permisos sin duplicar clases de test.

Los request post-processors permiten añadir CSRF, usuario o JWT simulado a una petición MockMvc. Son especialmente útiles para probar cómo una ruta reacciona a una identidad sin tener que ejecutar todo el login en cada test.

No deben usarse para acreditar algo que están saltándose. Si el objetivo es probar JwtAuthenticationFilter, construir directamente una autenticación en el request no demuestra que el parser/filtro funcionen.

Antes de elegir un post-processor hay que escribir qué capa se quiere aislar. Si el objetivo es una regla de @PreAuthorize, simular un usuario es apropiado; si el objetivo es comprobar parsing de Bearer, hay que dejar actuar al filtro real. Esta decisión forma parte de la documentación del test: describe qué infraestructura se sustituye y qué infraestructura queda realmente bajo prueba.

Los request post-processors se aplican a una petición concreta de MockMvc y permiten expresar la identidad cerca del request:

```
mockMvc.perform(get("/api/v1/admin/usuarios")
    .with(user("admin").roles("ADMIN")))
    .andExpect(status().isOk());
```

Esto resulta cómodo cuando un mismo método ejecuta varias peticiones con usuarios diferentes. También existen httpBasic, csrf y otras utilidades. Como con las anotaciones, estos helpers pueden saltarse capas; por eso hay que documentar qué se simula y no usarlos para afirmar que el filtro JWT o el login real funcionan.

T3.2 - El post-processor jwt()

En aplicaciones que usan el soporte OAuth2 de Spring Security existe un post-processor jwt() que simula un JWT y sus claims. Nuestro proyecto implementa un filtro JWT propio, por lo que la prueba más representativa del filtro usa tokens generados por JwtService.

Aun así, conocer jwt() ayuda a distinguir pruebas de autorización basada en claims de pruebas del parser/filtro real.

El post-processor jwt() pertenece al soporte de resource server de Spring Security y puede ser útil en proyectos configurados para ello. Nuestro módulo implementa un filtro JWT propio con

JWT; por eso hay que distinguir un test de autorización con identidad JWT simulada de un test de nuestro filtro real.

La herramienta es válida si la propiedad probada está bien definida. La documentación debe decir qué capa está siendo sustituida por el test.

El post-processor `jwt()` está pensado para aplicaciones configuradas como OAuth2 Resource Server y permite construir una identidad JWT simulada con claims y authorities. Es una herramienta útil para probar autorización cuando ése es el stack real.

Nuestro M6 implementa un filtro personalizado con JWT. Por ello, usar `jwt()` no demostraría que `JwtAuthenticationFilter` extrae Bearer ni que `JwtService` valida la firma. Podemos conocer la herramienta y usarla en contextos apropiados, pero para el filtro propio generamos un token real de test o invocamos el filtro con un Bearer construido por nuestro servicio.

T3.3 - Verificaciones con `SecurityMockMvcResultMatchers`

Los matchers de seguridad permiten comprobar si una petición terminó autenticada, anónima o con una authority concreta. Combinados con `status()` y aserciones JSON, proporcionan una especificación precisa.

La verificación no debe limitarse al status: cuando sea relevante también comprobaremos cuerpo de error, `traceId` y datos del perfil autenticado.

`SecurityMockMvcResultMatchers` permite comprobar propiedades de seguridad además del status, como usuario autenticado o roles. Es útil cuando queremos demostrar que el contexto se construyó correctamente, no sólo que el endpoint devolvió 200.

Aun así, el resultado HTTP sigue siendo importante. Un test completo puede combinar contexto esperado con cuerpo/status para conectar seguridad y funcionalidad.

Estas aserciones son especialmente útiles para detectar un 200 accidental ejecutado como anónimo en una ruta que el test esperaba autenticada. Comprobar `authenticated()` y authorities junto con status ayuda a distinguir “la ruta respondió” de “la ruta respondió bajo la identidad y permisos que el escenario pretendía”.

Ejemplo.

```
mockMvc.perform(get("/api/v1/admin/usuarios"))
    .andExpect(status().isUnauthorized());
```

Pregunta de reflexión: ¿Cuándo usarías un post-processor de Spring Security Test en lugar de obtener un token real mediante el endpoint de login?

Respuesta razonada: Los post-processors permiten añadir usuario, CSRF u otras credenciales a una petición MockMvc concreta. Son útiles cuando el mismo test necesita expresar distintas identidades sin fijarlas a nivel de método.

Bloque 4 - Tests de autenticación y autorización

T4.1 - Tests de autenticación (login)

El test de login envía credenciales válidas y espera un par de tokens con tipo Bearer y expiración. También debe enviar una contraseña errónea y comprobar que no se emite ningún token.

Esto prueba la integración de `AuthenticationManager`, usuarios persistidos, password encoder y emisión JWT.

Los tests de login deben cubrir credenciales correctas e incorrectas, estructura de tokens y ausencia de información sensible. Un 200 que devuelve una cadena cualquiera no basta: la respuesta debe cumplir el DTO y los tokens deben ser verificables según la política.

También se prueba que el endpoint sea público sólo en el sentido correcto: no necesita Bearer previo, pero sí valida las credenciales recibidas.

Además del status se inspecciona que el JSON no contenga password ni detalles de excepción. Para credenciales incorrectas, la respuesta debe ser estable tanto si el username existe como si no; esa simetría reduce filtraciones de enumeración. El test no necesita comparar tiempos exactos, pero sí el contrato visible y la ausencia de datos sensibles.

El login se prueba enviando JSON con credenciales correctas e incorrectas. El caso exitoso espera 200 y un DTO con access, refresh, Bearer y expiración; el caso fallido espera 401 y no debe revelar si falló username o password.

La prueba puede validar además que el access es parseable por JwtService, que su sub corresponde al usuario y que el tipo es access. Así evitamos un falso positivo donde el controlador devuelve cualquier cadena. También verificamos que la respuesta no serialice contraseña ni hash y que el endpoint sea accesible sin un Bearer previo.

T4.2 - Tests de refresh y logout

Refresh debe aceptar un token utilizable, rotarlo y rechazar su reutilización. Logout debe provocar que las credenciales revocadas dejen de aceptarse según la estrategia didáctica del módulo.

Estos escenarios son importantes porque el camino feliz de login no revela errores de ciclo de vida.

Refresh necesita casos de éxito, expiración/tipo incorrecto y rotación; logout necesita demostrar revocación. Estas pruebas son secuenciales porque el estado cambia: un refresh usado una vez deja de ser válido y un token revocado cambia de aceptado a rechazado.

Ese carácter temporal debe reflejarse en el test. Si cada llamada se prueba aisladamente con mocks, se puede perder precisamente la propiedad de single-use.

Conviene comprobar también que los tokens nuevos son distintos de los consumidos y que conservan el sujeto esperado. En logout, el test no termina al recibir 204: reutiliza el access para demostrar 401. En refresh, el test no termina con el primer 200: reutiliza el refresh anterior para demostrar rechazo. Son secuencias, no peticiones aisladas.

Refresh y logout son pruebas de **secuencia**. Para refresh: obtener un refresh válido, canjearlo, comprobar el nuevo par y volver a usar el antiguo esperando rechazo. Para logout: obtener access, usarlo con éxito, hacer logout y reutilizar exactamente el mismo token esperando 401.

Estas pruebas verifican propiedades que un test aislado no puede demostrar: single-use y cambio de estado de revocación. También se prueba un access enviado a /refresh, un refresh mal firmado y, cuando sea viable, expiración. El resultado no es sólo un status; debe mantenerse el contrato de error con código y traceId.

T4.3 - Tests de autorización

La autorización se prueba con identidades que difieren sólo en sus roles. Un USER y un ADMIN pueden estar correctamente autenticados, pero obtener respuestas distintas al llamar a una operación administrativa.

Ese contraste demuestra que 403 pertenece a autorización, no a autenticación.

Los tests de autorización construyen la matriz 401/403/200 para rutas y métodos sensibles. La suite final incluye anónimo, USER, GESTOR y ADMIN donde corresponda y verifica contenido para evitar falsos 200 vacíos.

Una prueba de “ADMIN puede” no demuestra que “USER no puede”. Las políticas de seguridad se prueban por ambos lados de la frontera.

Ejemplo.

login OK/K0 | refresh OK/replay | logout antes/después | autorización 401/403/200

La matriz de autorización combina identidad y recurso. Para administración: anónimo → 401, USER → 403, ADMIN → 200. Para gestor: USER → 403, GESTOR → 200 y ADMIN → 200. Para perfil por id se añade la condición de propietario: usuario sobre su propio id → 200, sobre otro → 403, ADMIN sobre cualquiera → 200.

Comprobar sólo que ADMIN obtiene 200 deja sin testar el cierre de la frontera. Los tests negativos son igual de importantes y ayudan a detectar una regla demasiado amplia o una anotación perdida durante un refactor. > **Pregunta de reflexión:** ¿Qué escenarios negativos deben acompañar a cada camino feliz para que la suite detecte aperturas accidentales?

Respuesta razonada: La matriz debe incluir anónimo, credenciales inválidas, usuario autenticado sin rol y usuario autorizado, además de login/refresh/logout. Los status y el cuerpo de error deben verificarse, no sólo que la petición termine.

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas con tests de seguridad

Los tests deben ser pequeños, nombrar claramente el escenario y evitar depender unos de otros. Los datos se preparan de forma idempotente y cada prueba expresa su propia identidad.

Separar pruebas de política, de servicios JWT y de flujo end-to-end facilita diagnosticar el origen de un fallo.

Buenas prácticas: nombres de test que describen la propiedad, fixtures mínimos, separación entre autorización simulada e integración real, no compartir tokens mutables entre casos y comprobar cuerpos de error. Los tests deben ser deterministas y no depender del reloj de pared más de lo necesario.

La suite se ejecuta con el mismo Maven Wrapper del proyecto. El número de tests no sustituye a su calidad: no tiene sentido duplicar casos irrelevantes sólo para aumentar el contador.

Cada test debe expresar una propiedad concreta, preparar sólo los datos que necesita y ser independiente del orden de ejecución. Los tokens mutables o refresh de un solo uso no se comparten entre métodos. Los fixtures de usuarios y roles se construyen de forma idempotente y los nombres de test describen condición y resultado esperado.

Conviene separar tests unitarios de JwtService, tests MVC de política y tests de integración del flujo completo. Así, cuando algo falla, sabemos si mirar criptografía, configuración de Spring Security o persistencia. La suite debe ejecutarse con el Maven Wrapper y leer los resultados reales de Surefire, no inferir éxito por el simple hecho de que el comando terminó sin mostrar una excepción visible.

T5.2 - Errores comunes

Son comunes los falsos positivos causados por desactivar filtros en un `@WebMvcTest`, declarar un `@MockBean` que ya no corresponde a la versión actual de Spring Boot o contar tests sin leer el resultado de Surefire.

En M6 los slices MVC que preservan comportamiento heredado desactivan filtros cuando su objetivo no es seguridad, pero proporcionan mocks para las dependencias de seguridad que Spring aún necesita instanciar.

Errores comunes: usar `@MockBean` obsoleto en este stack en vez de `@MockitoBean`, desactivar filtros y creer que no se instancian sus beans, testear sólo 200, confundir rol/authority o mockear tanto que la política real nunca se carga.

El antecedente de `JwtAuthenticationFilter` enseña una lección concreta: `addFilters=false` afecta a la ejecución en `MockMvc`, no necesariamente al proceso de crear el `ApplicationContext`. Los colaboradores requeridos por el bean deben existir.

Los errores de testing también pueden producir una falsa sensación de seguridad: desactivar filtros en todos los slices, mockear `SecurityConfig`, usar `@WithMockUser` para “probar JWT”, olvidar escenarios negativos o mantener una anotación obsoleta que ya no corresponde al stack actual.

`@AutoConfigureMockMvc(addFilters=false)` impide que `MockMvc` ejecute los filtros en la petición, pero el `ApplicationContext` todavía puede crear el bean `JwtAuthenticationFilter`. Si sus colaboradores no están en el slice, el test ni siquiera arranca. La solución es proporcionar `@MockitoBean` para `JwtService` y `TokenRevocationService` cuando el objetivo del slice no es probar el filtro. Esta distinción forma parte del conocimiento práctico del módulo.

T5.3 - Estructura de un test de seguridad completo

Un test completo prepara datos, obtiene o construye la identidad, ejecuta la petición, comprueba status y cuerpo, y verifica el estado resultante cuando la operación modifica datos. Si prueba JWT real, genera el token con el mismo `JwtService` de la aplicación.

El módulo termina cuando esta batería demuestra conjuntamente autenticación, autorización, errores y continuidad con los módulos anteriores.

Ejemplo.

```
mockMvc.perform(get("/api/v1/admin/usuarios")
    .with(user("ana").roles("USER")))
    .andExpect(status().isForbidden());

mockMvc.perform(get("/api/v1/admin/usuarios")
    .with(user("admin").roles("ADMIN")))
    .andExpect(status().isOk());
```

La misma política debe verificarse también en el flujo JWT real, no sólo con identidades simuladas. Un test de seguridad completo declara precondition, sujeto, petición, status, contrato de error o

cuerpo y, cuando aporta valor, estado de autenticación. Para la ruta administrativa final, cinco escenarios permiten demostrar anónimo 401, roles insuficientes 403 y ADMIN 200 con contenido válido.

Este tipo de test funciona como especificación ejecutable. Si alguien abre accidentalmente una ruta o cambia el mapeo de roles, la suite falla antes de publicar.

Ejemplo.

```
@MockitoBean
JwtService jwtService;
```

Pregunta de reflexión: ¿Por qué es importante verificar el código de error en el JSON y no solo el estado HTTP?

Respuesta razonada: El estado HTTP distingue la categoría general del fallo, pero el código estable del cuerpo forma parte del contrato de la API y permite al cliente distinguir causas concretas sin depender del texto del mensaje. Probar ambos evita regresiones en el formato de error aunque el status siga siendo correcto.

Resumen de la teoría

- La seguridad es funcionalidad y necesita pruebas positivas y negativas.
- `@WithMockUser` permite crear identidades sintéticas; `@WithUserDetails` usa un usuario real del `UserDetailsService`.
- Los post-processors de Spring Security permiten preparar autenticación petición a petición con `MockMvc`.
- Los tests deben distinguir 401, 403 y 2xx y cubrir login, refresh, logout y filtro JWT.
- Una suite útil prueba también ausencia, expiración, manipulación y roles insuficientes, no sólo caminos felices.